



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

**ФАКУЛТЕТ ПО ИЗЧИСЛИТЕЛНА ТЕХНИКА И
АВТОМАТИЗАЦИЯ**

катедра

„Софтуерни и интернет технологии”

Дипломна работа

Тема:

**„Проектиране и реализация на
система за управление на здравословно
хранене и рецепти”**

Изготвил: Шебнем Хасан Мехмед,

Факултетен номер: 21621503

Специалност: Софтуерни и интернет
технологии

Курс: 4

Ръководител:

ас. Стефка Попова

Съдържание

Глава 1: Въведение.....	6
1.1. Увод.....	6
Глава 2: Анализ на проблема и избор на технологии	7
2.1. Анализ на проблема	7
2.2. Съществуващи решения и техните ограничения	8
2.3. Цели и изисквания	10
2.4. Цели и предимства на предложеното решение.....	11
2.5. Избор на технологии	12
2.5.1. React Native (Клиентска част, за крайните потребители)	12
2.5.2. React.js (Клиентска част, за администраторския панел)	13
2.5.3. Node.js (Сървърна част)	14
2.5.4. Express.js (Сървърна част)	15
2.5.5. Render.com	15
2.5.6. PostgreSQL (База данни)	16
2.6. Среда на разработка	17
2.6.1. Visual Studio Code (VS Code)	17
Глава 3: Проектиране на системата	18
3.1. Изисквания на програмната система.....	18
3.2. Архитектура на системата.....	19
3.3. Коммуникационен поток в системата	21
3.4. Логически модел.....	23
3.4.1. Основни логически модули	24
3.4.2. Основни логически обекти и връзки.....	25
3.4.3. Диаграма и описание на работния процес на потребителската част	29
3.4.4. Диаграма и описание на работния процес на админската част.....	33
3.5. Структурни скици на интерфейса (Wireframes)	35
3.6. Избор на цветова схема и визуални елементи	36
3.7. Организация на данните.....	37
Глава 4: Реализация на системата	42
4.1. Структура на данните	42

4.1.1. Аномалии и нормализации.....	42
4.1.2. Използвани специфични типове данни	43
4.1.3. Описание на таблиците от базата данни	44
4.2. Потребителски интерфейс и функционалности на системата	65
4.2.1. Регистрация на потребител	65
4.2.2. Потвърждение на имейл	66
4.2.3. Вход в системата.....	67
4.2.4. Първоначална настройка на профил	67
4.2.4. Добавяне на консумирана храна към дневник	71
4.2.5. Визуализиране на добавените храни в общия дневник	72
4.2.6. Преглед на добавени храни за изминали дни	73
4.2.7. Изтриване на добавени храни.....	74
4.2.8. Изчисляване на оставащи калории	75
4.2.9. Екран с рецепти	75
4.2.9. Екран с детайли за рецепта	76
4.2.10. Запазване на рецепта в любими	77
4.2.11. Преглед на любими рецепти	78
4.2.12. Начален екран – дневен преглед	78
4.2.13. Прием на вода	80
4.2.14. Редактиране на лични данни	81
4.2.15. Редактиране на основни физически данни	82
4.2.16. Редактиране на целево тегло	82
4.2.17. Редактиране на макронутриентно разпределение.....	84
4.2.18. Редактиране на диетичните предпочитания.....	85
4.2.19. Добавяне на персонализирани хранителни продукти	86
4.3. Административен панел. Функционалности на главен админ.....	89
4.3.1. Вход в системата за администратори	89
4.3.2. Активни потребители (Active Users – last 5 min)	89
4.3.3. Ръст на потребителската база (User Growth)	90
4.3.4. Ръст на потребителската база (User Growth)	91
4.3.5. Среден индекс на телесна маса на потребителите (Average BMI)	91
4.3.6. Управление на администратори (View & Manage Admins).....	92
4.3.7. Управление на потребители (View & Manage Users).....	93

4.3.8. Създаване на администратор (Create Admins)	93
4.3.9. Търсене и филтриране на потребители (User Search & Filtering).....	94
4.3.10. Честота на въвеждане на храни (Food Log Frequency)	96
4.3.11. Използваемост на функционалности (Feature Usage Breakdown)	97
4.4. Административен панел. Функционалности на админ-съдържание.....	98
4.4.1. Преглед на рецепти (View Recipes).....	98
4.4.2. Преглед на хранителни продукти (View Foods).....	99
4.4.3. Преглед на категории (View Categories)	99
4.4.4. Добавяне на категория (Add New Category).....	99
4.4.5. Добавяне на хранителни продукти от външен източник.....	100
4.4.6. Добавяне на рецепти	101
Глава 5: Тестове и резултати	103
5.1. Тестване на софтуерна система	103
5.2. Използвани методи при тестване	103
5.2.1. Тестване с Mocha и Chai	103
5.2.2. Тестване чрез черна кутия	106
Глава 6: Заключение и бъдещо развитие	112
6.1. Изводи и заключение.....	112
6.2. Бъдещо развитие	113
Списък с фигури	114
Използвани източници	117
Приложение – изходен код.....	118

Глава 1: Въведение

1.1. Увод

В съвременното общество все по-голямо внимание се обръща на здравословния начин на живот и правилното хранене. Благодарение на разпространението на информация в интернет и социалните медии, все повече хора се вдъхновяват да променят своите навици в посока на по-здравословно хранене. Платформи като Instagram, TikTok, YouTube изобилстват от съдържание, свързано с фитнес, диети и здравословни рецепти, което насърчава потребителите да бъдат по-осъзнати и ангажирани с хранителния си режим и личното си здраве.

Социалните мрежи играят важна роля не само като източник на информация, но и като двигател на промяна в нагласите към храненето. Все повече хора споделят свои постижения, рецепти и съвети, създавайки онлайн общности, които стимулират здравословния начин на живот.

С нарастващия интерес към персонализирано хранене възниква и сериозно предизвикателство: как да се адаптира информацията към конкретния потребител, отчитайки неговите физиологични параметри, цели и начин на живот. Много хора изпитват затруднения в правилното проследяване на калориите, съотношението между макронутриенти, напасването на менюто към дневния график и отчитането на прогреса във времето. Това води до нуждата от разработване на автоматизирана система, която да облекчи този процес.

Мотивацията за разработване на настоящата дипломна работа произтича от липсата на достатъчно интегрирани решения, които да съчетават всички ключови елементи: проследяване на хранене, изчисляване на основни здравни индикатори (като индекс на телесна маса – ВМІ, базов метаболизъм и дневен калориен баланс), търсене и филтриране на рецепти, както и поставяне и следене на цели (като намаляване, покачване или поддържане на тегло).

Представеният проект цели разработване на интерактивна диетологична система, която автоматизира значителна част от процесите, свързани с планиране и контрол на хранителния режим. Чрез внедряване на такава система се постига не само по-висока ефективност в управлението на диетичния режим, но и възможност за изграждане на устойчиви здравословни навици. В заключение, необходимостта от подобен тип автоматизирани решения е ясно изразена от тенденциите в обществото, и тяхната реализация представлява отговор на конкретна социална и здравна нужда.

Глава 2: Анализ на проблема и избор на технологии

2.1. Анализ на проблема

С нарастващия интерес към здравословния начин на живот все повече целенасочено търсят устойчиви стратегии за подобряване на своето хранене, поддържане на физическа активност и мониторинг на ключови здравни индикатори. Въпреки положителната тенденция, процесът по планиране и мониторинг на хранителния режим често се оказва предизвикателство. Той изисква последователност, време и достъп до надеждна, научно обоснована информация. Множество потребители остават изложени на митове и дезинформация – например погрешното схващане, че ефективното отслабване може да се постигне чрез изключване на цели групи храни (като мазнините) или чрез крайно ограничаване на приема. Такива подходи, включително гладуването, не само че са неустойчиви в дългосрочен план, но могат сериозно да навредят на организма. Важно е потребителят да осъзнае, че приемът на каквито и да било храни не трябва да бъде избягван, а по-скоро балансиран.

Наред с объркването, породено от недостоверна информация, потребителите често срещат и редица практически затруднения в ежедневиия процес на хранене. Сред тях се открояват:

- Трудност при изчисляване на калориите и съотношението на макронутриенти, съобразени с конкретна цел (покачване, сваляне или поддържане на тегло);
- Липса на яснота относно желания дневен прием и дали той покрива физиологичните нужди;
- Неудобство при дългосрочно проследяване на напредъка и изграждането на навици;
- Ограничен достъп до рецепти, адаптирани към хранителни режими или здравословни състояния (алергии, непоносимости);
- Липса на средства за анализ – потребителите често не знаят как отделна храна влияе на дневния им баланс;
- Невъзможност за проследяване на допълнителни фактори като воден прием, които също влияят върху здравословното състояние.

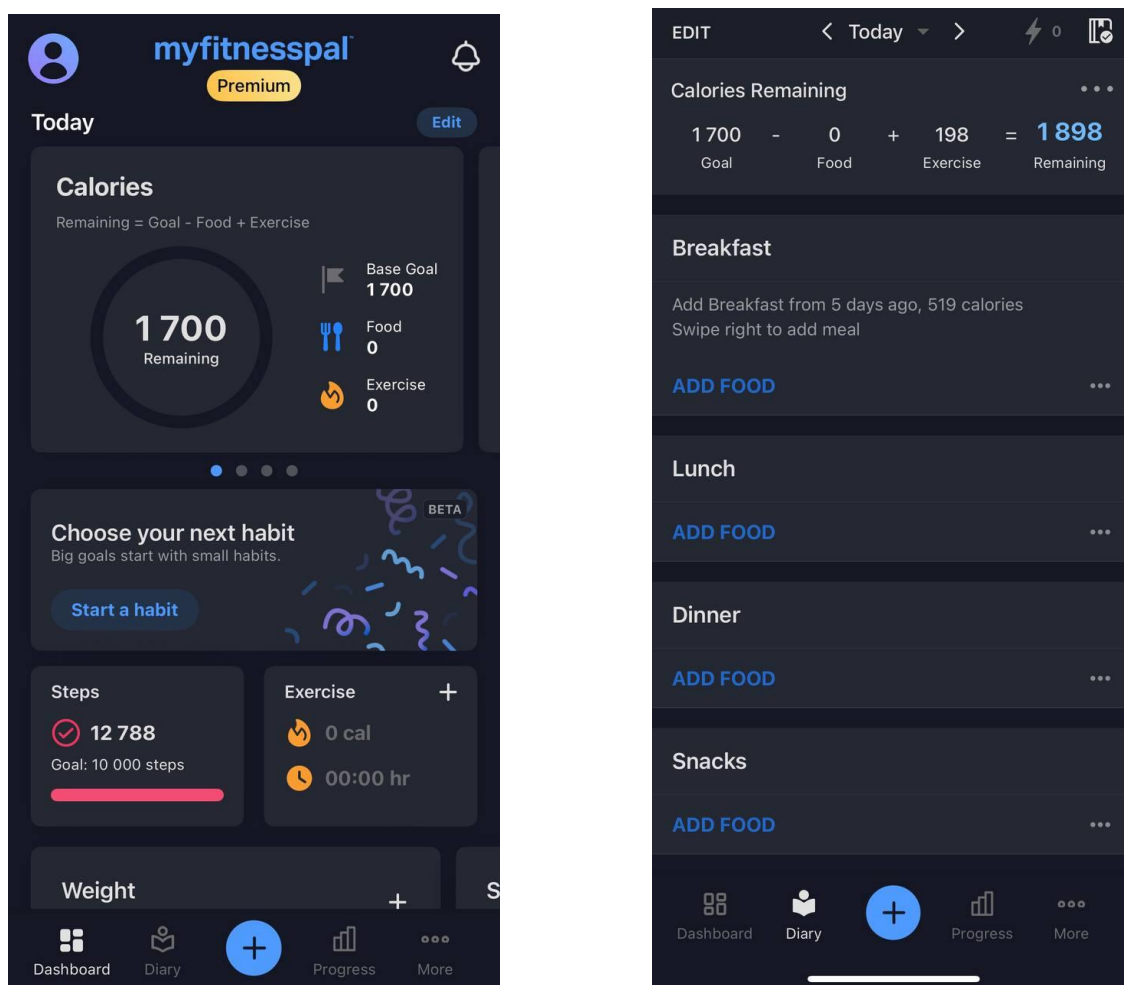
Всички изброени затруднения ясно очертават необходимостта от дигитално решение, което да облекчи потребителите в процеса на планиране, проследяване и оптимизиране на хранителния режим. За да бъде наистина полезна, една такава платформа трябва да бъде едновременно интуитивна, функционално богата и научно обоснована – с възможности за индивидуализиране според целите, физиологичните нужди и специфичните хранителни предпочитания на

потребителя. Освен това, тя следва да предлага локализирано съдържание, ясно визуализиране на напредъка и интелигентна обратна връзка, която не само да информира, но и да мотивира

2.2. Съществуващи решения и техните ограничения

На пазара съществуват редица мобилни и уеб приложения, предназначени за подпомагане на потребителите в процеса на проследяване на хранителни навици, калориен прием, макронутриенти и физическа активност. Тези приложения се използват широко както от хора, стремящи се към отслабване или покачване на тегло, така и от спортисти, фитнес ентузиаста и всички, които желаят да подобрят здравословното си състояние чрез по-информиран контрол върху своето хранене. Сред най-популярните от тях са:

1) MyFitnessPal



Фигура 1 - Изглед на MyFitnessPal - система за следене на здравословно хранене

MyFitnessPal е едно от най-популярните приложения за проследяване на калории и физическа активност. Позволява на потребителите да въвеждат храни, упражнения, да следят тегло и да получават обобщена информация за макронутриентите.

Предимства:

- Огромна база от данни, включително баркод скенер, което улеснява потребителите;
- Свързване с други фитнес приложения и устройства;
- Подробни дневни отчети за калории и макронутриенти;
- Възможност за създаване на персонализирани ястия;

Недостатъци:

- Много от полезните функции (като баркод скенер) са достъпни само в платената версия;
- Няма лесен филтър за алергии, тип диета или вид кухня;
- Не предлага визуализация как отделна храна влияе спрямо дневната цел;
- Интерфейсът е претоварен и неинтуитивен за начинаещи;
- Загуба на данни за добавени храни

2) Cronometer

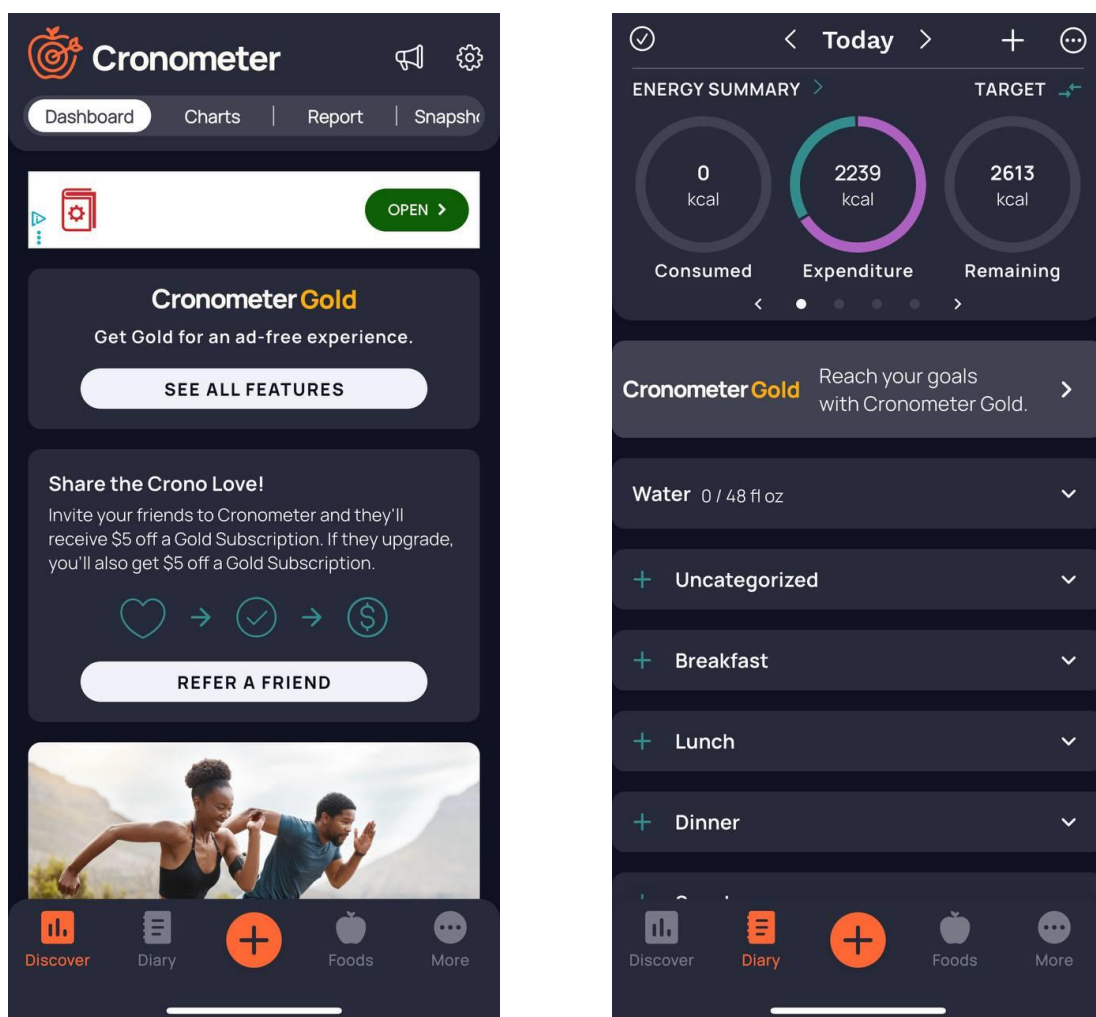
Cronometer се фокусира върху прецизното проследяване на калории, витамини и минерали и други макронутриенти, като е предпочитан от хора с по-специфични хранителни нужди.

Предимства:

- Изключително точни хранителни данни;
- Поддръжка на голям брой витамини и минерали;
- Функции за следене на BMI и физическа активност;
- Възможност за създаване на индивидуални цели.

Недостатъци:

- По-сложен за употреба от начинаещи потребители;
- Част от функциите са в платената версия;
- Интерфейсът не е достатъчно модерен и визуално ангажиран;
- Ограничена база с рецепти и липса на филтри по диети, алергии и кухни.
- Безплатната версия е пълна с реклами;



Фигура 2 - Изглед на Cronometer - система за следене на здравословно хранене

2.3. Цели и изисквания

Целта на дипломната работа е да предостави интуитивно и функционално приложение, което подпомага потребителите при планиране, проследяване и оптимизиране на техния хранителен режим, съобразно личните им цели и здравословни нужди. Приложението трябва да бъде подходящо за широк кръг потребители – от начинаещи до напреднали – и да предлага полезни инструменти за информирано хранене.

Изисквания:

Потребителят трябва да може да:

- се регистрира, влиза в системата и редактира профила си;
- избира цели като покачване на тегло (bulk), сваляне (cut), поддържане (maintenance);
- въвежда и преглежда консумирани храни;
- следи хранителната стойност и % от дневния прием на храните;

- филтрира рецепти по режим, алергии и кухни;
- проследява хидратация за деня
- добавя храни
- добавя рецепти

Администраторът трябва да може да:

- управлява потребителски профили;
- добавя/редактира/изтрива храни, рецепти, категории;
- наблюдава статистики и контролира съдържанието;

Системата трябва да има:

- интуитивен и модерен интерфейс;
- сигурност при вход и достъп до данни
- устойчивост и разширяемост – възможност за добавяне на нови функции;

2.4. Цели и предимства на предложеното решение

Предложеното решение има за цел да предостави на потребителите мобилно приложение, чрез което те да могат ефективно да следят и управляват своето хранене и начин на живот. За разлика от повечето съществуващи приложения, които са ограничени по функционалност или не отчитат индивидуалните потребности, настоящият проект е фокусиран върху висока степен на персонализация, достъпност и цялостно покритие на нуждите на различни типове потребители – от начинаещи до напреднали.

Чрез създаване на потребителски профили и задаване на конкретни цели – като покачване, намаляване или поддържане на тегло – системата автоматично изчислява ключови здравни индикатори, като индекс на телесна маса (BMI) и дневна калорийна нужда, базирани на индивидуални параметри. Това целево ориентирано планиране позволява на всеки потребител да получи персонализирани препоръки, съобразени с неговите цели и физиологични особености.

Сред основните предимства на системата се открояват:

- Интуитивен интерфейс, който улеснява навигацията и прави приложението достъпно за широка аудитория;
- Проследяване на дневния хранителен прием, като всяка въведена храна се анализира по калории и макронутриенти, показвайки какъв процент от дневните нужди покрива;
- Възможност за въвеждане на нови храни, с което се гарантира динамичност и адаптивност на базата данни;

- Поддръжка на различни хранителни режими (кето, веган, палео и др.), както и възможност за филтриране по вид кухня и наличие на алергии (глутен, лактоза, ядки и др.);
- Функция за следене на дневния прием на вода, която подпомага хидратацията – ключов, но често пренебрегван елемент от здравословния начин на живот;
- Преглед на хранителен дневник, който подпомага изграждането на дългосрочни и устойчиви навици чрез визуализиране на изминали хранения и напредък;
- Администраторски панел, чрез който може да се управлява съдържанието в приложението – включително рецепти, храни, категории и потребители.

Освен това приложението дава възможност за детайлен визуален анализ, който подпомага потребителите при вземане на информирани решения относно избора на храна – особено когато се стремят към специфичен нутриционен баланс. Тази съвкупност от функционалности превръща системата в цялостен инструмент за управление на хранителния режим и начин на живот. Със своето ориентирано към целите поведение, персонализиран подход и лесна употреба, решението се явява практичен отговор на нарастващото търсене за дигитални платформи, които обединяват здраве, хранене и технологии в едно.

2.5. Избор на технологии

При разработката на приложението бяха избрани съвременни, широко използвани технологии, които осигуряват стабилност, добра производителност и възможност за бъдещо разширяване. Изградена е двустранна архитектура, включваща отделни клиентски приложения за крайни потребители и администратори, свързани със съвместима сървърна част (backend) и централизирана база данни.

2.5.1. React Native (Клиентска част, за крайните потребители)

React Native е кросплатформена рамка за разработка на мобилни приложения, създадена от Meta. Тя позволява изграждането на мобилни приложения, разработени специално за iOS и Android с помощта на езика JavaScript и библиотеката React. React Native използва реални, вградени компоненти, което осигурява висока производителност и потребителско изживяване, близко до това на вградени за платформата приложения. Приложенията, създадени с React Native, могат да използват една и съща база от код за двете платформи (iOS и Android), като това значително намалява времето и ресурсите за разработка и поддръжка.

Възможности на React Native:

- Създаване на напълно функционални мобилни приложения от един общ кодов базис;
- Достъп до вградени хардуерни функции на устройствата (камера, GPS, акселерометър, файлове);
- Интеграция с Rest API и външни услуги (Firebase, Stripe, Google Maps);
- Работа с локални бази данни или сървърна синхронизация;
- Използване на готови компоненти и библиотеки за по-бърза разработка;

Основни предимства:

- Кросплатформеност – един код за две платформи;
- Висока производителност – за разлика от WebView-базирани приложения, React Native използва реални вградени компоненти;
- Голямо и активно общество – обширна документация и много готови решения;
- Бърза разработка и актуализации
- Лесна интеграция с бекенд – подходяща за приложения, които работят с динамични данни и потребителски акаунти;
- Гъвкавост при дизайн – възможност за изграждане на съвременен и адаптивен интерфейс за мобилни устройства;
- Поддръжка от Meta – рамката е стабилна и непрекъснато се развива;

2.5.2. React.js (Клиентска част, за администраторския панел)

React.js е популярна JavaScript библиотека с отворен код, разработена и поддържана от Meta, предназначена за създаване на динамични, компонентно-базирани потребителски интерфейси, главно за уеб приложения. React работи чрез виртуален DOM (Virtual DOM) – механизъм, който минимизира желаните манипулации с DOM структурата в браузъра, като по този начин повишава производителността и реактивността на интерфейса.

Възможности на React.js:

- Създаване на сложни, интерактивни и мащабируеми уеб интерфейси;
- Изграждане на повторно използваеми компоненти, което ускорява разработката и улеснява поддръжката;
- Възможност за създаване на едностранни приложения, които се зареждат бързо и без презареждане на страницата;
- Интеграция с библиотеки за визуализация на данни (Chart.js, Recharts), стилизиране (Tailwind CSS, Styled Components) и UI компоненти.

Предимства на React:

- Бързодействие и оптимизация – чрез виртуален DOM и изграждане на интерфейса чрез отделни компоненти;
- Лесна визуализация и обработка на данни – например списъци с рецепти, потребители, категории;
- Гъвкав дизайн;
- Модулност и добра структура – улеснява добавянето на нови функционалности в админ панела (като редакция на рецепти, одобрения, статистики и други);
- Широка общност и богата документация;
- Административните маршрути трябва да бъдат защитени чрез маршрутизиране и показване на съдържание според ролята на потребителя.

2.5.3. Node.js (Сървърна част)

Node.js (или само Node) е сървърно приложение, създаващо среда за изпълняване на JavaScript код (runtime environment), а не език за програмиране. Node.js прави възможно изпълнението на JavaScript код директно на сървъра, както обикновено това се случва в уеб браузъра, позволява разработка на бързи, мащабируеми и асинхронни уеб приложения, като използва събитийно-ориентирана архитектура и неблокиращ I/O модел, за да се избягват неща като синхронни нишки, които стартират отделна нишка за заявка и блокират целия процес. Това я прави изключително подходяща за приложения, при които е важно бързо обслужване на множество заявки.

Възможности на Node.js

- Създаване на RESTful API за комуникация между клиент и сървър;
- Обработка на HTTP заявки и отговори;
- Работа с бази данни;
- Реализация на аутентикация, авторизация и ролево управление;
- Изпращане на нотификации, хеширане на пароли и обработка на файлове;
- Изпълнение на скриптове и фонові задачи;
- Възможност за реализация на чат, статистики и логическа обработка в реално време;

Предимства на Node.js

- Единен език за клиент и сървър – JavaScript може да се използва както във фронтенд (React, React Native), така и в бекенд, което улеснява комуникацията и намалява сложността;
- Висока производителност – благодарение на неблокиращи I/O и асинхронна обработка;

- Голяма екосистема – чрез npm (Node Package Manager) има достъп до хиляди готови библиотеки и модули;
- Активна общност и богата документация, което улеснява разработката и намирането на решения;

2.5.4. Express.js (Сървърна част)

Express.js е уеб фреймуърк (технологична работна рамка), създадена на JavaScript, който улеснява създаването на сървърни приложения и RESTful API. Тя е една от най-популярните библиотеки в екосистемата на Node и предлага структуриран начин за дефиниране на маршрути (routes), обработка на заявки, работа с междинни слоеве (middleware), както и комуникация с бази от данни и външни услуги.

Express позволява разработването на мащабируеми, модулни и поддържани бекенд системи, като същевременно запазва висока производителност и контрол върху логиката.

Възможности:

- Дефиниране на HTTP маршрути (GET, POST, PUT, DELETE)
- Работа с middleware функции – за валидация, автентикация, логване и защита
- Поддръжка на мултимодулна структура, улесняваща разделянето на логиката по функции

Предимства:

- Лек и бърз;
- Добре документиран, много готови решения и примери
- Интеграция с Node.js

2.5.5. Render.com

Това е съвременна облачна платформа, предназначена за автоматизирано хостване на уеб приложения, бекенд услуги, статични сайтове, бази данни и фонові процеси. Тя предлага опростен начин за деплойване на проекти, като поддържа интеграция с популярни системи за контрол на версиите (GitHub, GitLab) и осигурява пълна автоматизация на процеса по изграждане, хостване и обновяване на приложения.

В рамките на настоящия проект Render.com се използва за хостване на бекенд частта на приложението за проследяване на хранене и рецепти. Бекендът осигурява достъп до базата от храни, рецепти и потребителски профили, обработва заявките от мобилния интерфейс и извършва изчисления, свързани с калориен прием, макронутриенти и индикатори.

Възможности:

- Автоматичен деплой – при всяка нова версия на кода, качена в GitHub/GitLab, Render автоматично изгражда и публикува актуализирана версия.
- Мониторинг на логове – в реално време могат да се наблюдават логове от сървъра;
- Базии данни – възможност за хостване на PostgreSQL база директно в Render, с връзка към приложението.

Предимства:

- Лесен за използване интерфейс
- Поддържане на различни езици и среди

Причина за избора на Render.com

Една от основните причини да бъде избрана тази платформа е избягването на необходимостта от фиксиране на IP адреси при свързване между мобилното приложение (фронтенд) и бекенд услугата. Render автоматично предоставя публично достъпен URL адрес за всяка уеб услуга, което елиминира нуждата от ръчно конфигуриране на IP и улеснява както разработката, така и тестването на системата в реална среда.

2.5.6. PostgreSQL (База данни)

Обектно-релационна система за управление на бази от данни с отворен код, известна със своята стабилност, мащабируемост, сигурност и съвместимост със SQL стандарта. Тя поддържа както класически релационен модел (таблици, релации, ключове), така и по-сложни структури от данни (JSON, масиви, географски координати). PostgreSQL е широко използвана в мобилни приложения и уеб платформи.

Възможности:

- Пълна поддръжка на SQL – подзаявки, тригери, изгледи, функции
- Сложни типове данни
- Ролево управление и права на достъп

Предимства:

- Отворен код – без лицензионни такси
- Висока сигурност
- Разширяемост – възможност за добавяне на модули, плъгини и собствени функции

2.6. Среда на разработка

2.6.1. Visual Studio Code (VS Code)

За създаване на проекта е използвана съвременна и широко разпространена среда за разработка – Visual Studio Code. Това е безплатен, лек и разширяем текстов редактор, разработен от Microsoft, който предлага пълна поддръжка на JavaScript, Node.js, React, React Native и множество други езици и технологии.

Изборът на тази среда е мотивиран от нейната богата екосистема, висока степен на персонализация и интеграция и инструменти за контрол на версиите, дебъгване и тестване.

Глава 3: Проектиране на системата

Настоящата глава представя логическото и технологичното проектиране на системата, използвано за създаване на ефективна, сигурна и разширяема платформа за управление на здравословно хранене и рецепти.

3.1. Изисквания на програмната система

Разработената програмна система има за цел да подпомогне потребителите в изграждането и поддържането на здравословни хранителни навици чрез мобилно приложение, както и да осигури администраторски контрол върху съдържанието и потребителските данни чрез отделен уеб базиран панел. Системата е двукомпонентна – клиентската част за крайни потребители е мобилна, а административният панел е реализиран като уеб приложение.

Използваните технологии (React Native за мобилно приложение и React за уеб интерфейса) гарантират бързодействие и интуитивен потребителски опит, а сървърната част, изградена с Node.js и Express, осигурява стабилна и мащабируема обработка на заявки.

Бизнес правилата са специфични насоки или ограничения, които диктуват поведението и работата на софтуерното приложение. Те са от съществено значение, за да се гарантира, че приложението отговаря на бизнес изискванията и е в съответствие с организационните политики, Някои от основните бизнес правила в приложението са:

- 1) Управление на потребителски профили (мобилно приложение)
 - Регистрация чрез имейл и парола с необходимо имейл потвърждение;
 - Вход само след потвърждение на имейл чрез шестцифрен код;
 - Първоначална настройка на профила – задаване на текущо тегло, ръст, избор на целево тегло, избор на физическа активност, задаване на хранителна цел (отслабване, покачване или поддържане);
 - Редакция на профил, цели, имейл и парола;
 - JWT токени за удостоверяване на заявките.
- 2) Администраторски панел (уеб приложение)
 - Достъпен само през уеб интерфейса;
 - Вход само за верифицирани администратори;
 - Достъп до списък с потребители, храни, категории, рецепти и хронология на активност;
 - Добавяне, редактиране и изтриване на съдържание;
 - Преглед на статистики

3) Управление на хранителен дневник (Diary)

- Добавяне на храна за деня, тип хранене, количество и единици;
- Автоматично изчисление на нутриентите (калории, протеин, мазнини, въглехидрати);
- Обновяване на дневната обобщена статистика;
- История на храненията с възможност за изтриване;

4) Достъп до съдържание

- Всички потребители имат достъп до публични рецепти и категории;
- Възможност за търсене и филтриране на рецепти и храни;
- Извличане на хранителни стойности от USDA API (за администраторите)
- Възможност за създаване на персонализирани храни от потребителя;

5) Сигурност и валидиране

- Всички входни данни се валидират;
- Всяка защитена заявка изисква валиден токен;
- При грешки се връщат подходящи HTTP статус кодове (400, 401, 403, 404, 405);

3.2. Архитектура на системата

Архитектура на системата означава структурата и организацията на цялото софтуерно приложение – това включва начина, по който различните му части (модули, компоненти, слоеве) си взаимодействат, как са разпределени отговорностите и как протичат данните между тях.

Архитектурата на проекта за следене на здравословно хранене и рецепти включва:

1) Клиентска част (Front – end)

- Това е потребителският интерфейс, чрез който потребителите взаимодействат със системата.

2) Сървърна част (Back – end)

- Отговаря за обработка на заявки, връзка с базата данни, логика за вход, регистрация, храни, рецепти;
- Изграден е с Node.js и Express;

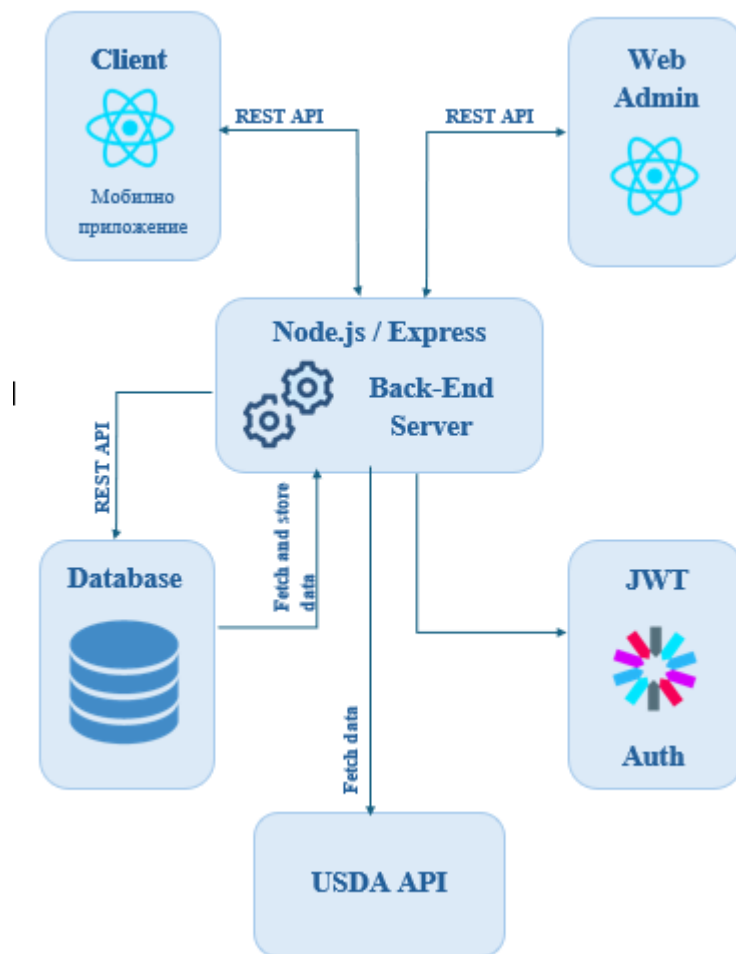
3) База данни (Database)

- Съхранява информация като потребители, рецепти, дневник на храни, прогрес;

4) API (Application Programming Interface)

- Мостът между фронтенда и бекенда;

5) Middleware сигурност



Фигура 3 - Архитектура на системата

Архитектурата на системата е изградена на принципа на клиент-сървър модел с използване на Node.js/Express като основа на сървърната логика (бекенд), което осигурява надеждна и гъвкава REST API комуникация между различните компоненти.

1) Клиент (Client – мобилно приложение)

Мобилното приложение е разработено с помощта на React Native и служи като основен интерфейс за крайните потребители. То комуникира със сървъра чрез REST API, като изпраща заявки (регистрация, вход, търсене на храни, дневен прием) и получава обратно отговори с нужната информация.

2) Web Admin панел

Уеб администраторският панел използва React.js и служи за управление на съдържание и потребителски данни. Подобно на мобилното приложение, комуникацията със сървъра се осъществява чрез REST API, като администраторът може да добавя, редактира и изтрива данни в системата.

3) Back-end Server (Node.js/Express)

Централният компонент на архитектурата е бекенд сървърът, реализиран чрез Node.js и Express.js. Той отговаря за:

- Обработка на входящи заявки от клиента и уеб админа;
- Изпълнение на логиката за валидиране, авторизация и обработка на данни;
- Комуникация с базата данни чрез REST API;
- Извличане на данни от външни източници като USDA API;
- Генериране и проверка на JWT токени за сигурност и удостоверяване на потребители.

4) База данни (Database)

Базата данни съхранява цялата ключова информация, включително:

- Данни за потребители;
- Хранителни записи и дневници;
- Административни настройки;
- История на консумация и резултати;

5) USDA API

USDA API е външен източник на информация за хранителни стойности. Бекенд сървърът изпраща заявки към USDA API, за да извлича детайлна информация за храни (като калории, мазнини, протеини, въглехидрати), която после предоставя на клиента.

6) JWT Auth (JSON Web Tokens)

За удостоверяване и авторизация се използва JWT. При вход или регистрация на потребител, сървърът генерира токен, който клиентът съхранява и използва при последващи заявки. Този токен се валидира от сървъра при всяка защитена операция.

3.3. Комуникационен поток в системата

Комуникацията между компонентите в системата се осъществява чрез REST API, като всеки участник има ясно дефинирана роля и отговорности. Потокът на данни следва логическа последователност, осигуряваща надеждна работа,

сигурност и ефективност. Пълният комуникационен процес между слоевете се реализира в следните стъпки:

1) Инициране на заявка от клиент или администратор

Комуникационният процес започва, когато мобилното приложение или уеб административният панел изпрати HTTP заявка към бекенд сървър. Тази заявка може да бъде за:

- Вход или регистрация на потребител;
- Извличане на дневно меню;
- Добавяне на хранителни записи;
- Преглед на статистики и анализи;
- Управление на данни от страна на администратор;

2) Обработка на заявката в Back-end сървър

След получаване на заявката, Node.js/Express сървърът:

- Валидира входните параметри;
- Проверява автентичността на потребителя чрез JWT токен, ако заявката изисква удостоверяване;
- Извиква съответните контролери и бизнес логика за обработка;

3) Авторизация чрез JWT (JSON Web Tokens)

При заявка, която изисква достъп до защитени ресурси, сървърът валидира предоставения JWT токен. Ако токенът е валиден:

- Сървърът позволява достъп до искания ресурс;
- В противен случай, заявката се отклонява със съответен HTTP статус код (например 401 Unauthorized)

4) Работа с базата данни

При необходимост от четене или записване на информация, сървърът изпраща заявка към базата данни чрез вътрешен REST API интерфейс. Това включва операции като:

- Съхраняване на нови записи (потребителски данни, консумирани храни)
- Извличане на статистика, история на хранения
- Актуализация или изтриване на съществуващи данни

5) Извличане на информация от USDA API

Когато потребител търси информация за определена храна , сървърът може да използва външна USDA API, за да:

- Извлече детайлна информация за хранителния състав;
- Съхрани резултатите в базата данни за бъдеща употреба;
- Изпрати обработените данни обратно към клиента;

6) Връщане на отговор към клиента

След като заявка е обработена – дали чрез достъп до базата данни, външен API или просто чрез бизнес логика – сървърът:

- Формира структуриран отговор (обикновено в JSON формат);
- Изпраща го обратно към клиента или уеб админа чрез REST API;
- Отговорът може да съдържа статус на операцията, резултати или грешки;

7) Визуализиране на резултата в интерфейса

Получената информация се визуализира в мобилното приложение или уеб панела чрез React компоненти. Това позволява:

- Интерактивна работа на потребителя със системата
- Актуализация в реално време при смяна на хранителни записи
- Представяне на анализи и препоръки за диета

Комуникационният поток е структуриран така, че да гарантира сигурност чрез JWT удостоверяване, ефективен достъп до данни чрез базата и външни източници (USDA API), както и удобен потребителски интерфейс чрез React базирани приложения. В резултат, системата постига балансирана и надеждна архитектура, способна да отговори на нуждите на реални потребители в контекста на управление на диети и хранене.

3.4. Логически модел

След като са определени основните технологии и архитектурата, които ще се използват при изграждане на дипломния проект, могат да се определят и функционалностите, които ще се разработят в системата.

Чрез изготвяне на логически модел на системата може да се представи абстрактно структурата, поведението и взаимодействието на компонентите на дадено приложение. Той описва как системата функционира и как различните части на системата си взаимодействат помежду си и с външни фактори. Логическият модел не се фокусира върху физическите или техническите детайли на системата, а по-скоро върху концептуалните аспекти.

3.4.1. Основни логически модули

1) Потребителски интерфейс (Client App – React Native)

Организиран е в директорията `app/` и съдържа логическо разделение по функционалности:

Модул	Поддиректории	Логическа роля
auth	<code>components, pages, services</code>	Отговаря за регистрация, вход и автентикация
home	<code>components, pages, services</code>	Начална информация, обобщение на консумирани калории и макронутриенти през деня, обобщена информация за прием на вода
diary	<code>components, pages, services</code>	Добавяне и преглед на дневни записи
meals	<code>components, pages, services</code>	Търсене и филтриране на рецепти
admin	<code>Dashboard, hooks, services, shared</code>	Функционалности за администраторски достъп
components	<code>shared</code>	Общи компоненти като цветове, UI елементи

2) Бекенд сървър (Node.js/Express)

Логически се разделя на следните групи:

Файл/Папка	Логическа роля
routes/	Съдържа всички API крайни точки, групирани по домейни – потребители, храни, рецепти, дневници, статистики и др.
middleware/	Съдържа логика за автентикация
db.js	Връзка с базата данни

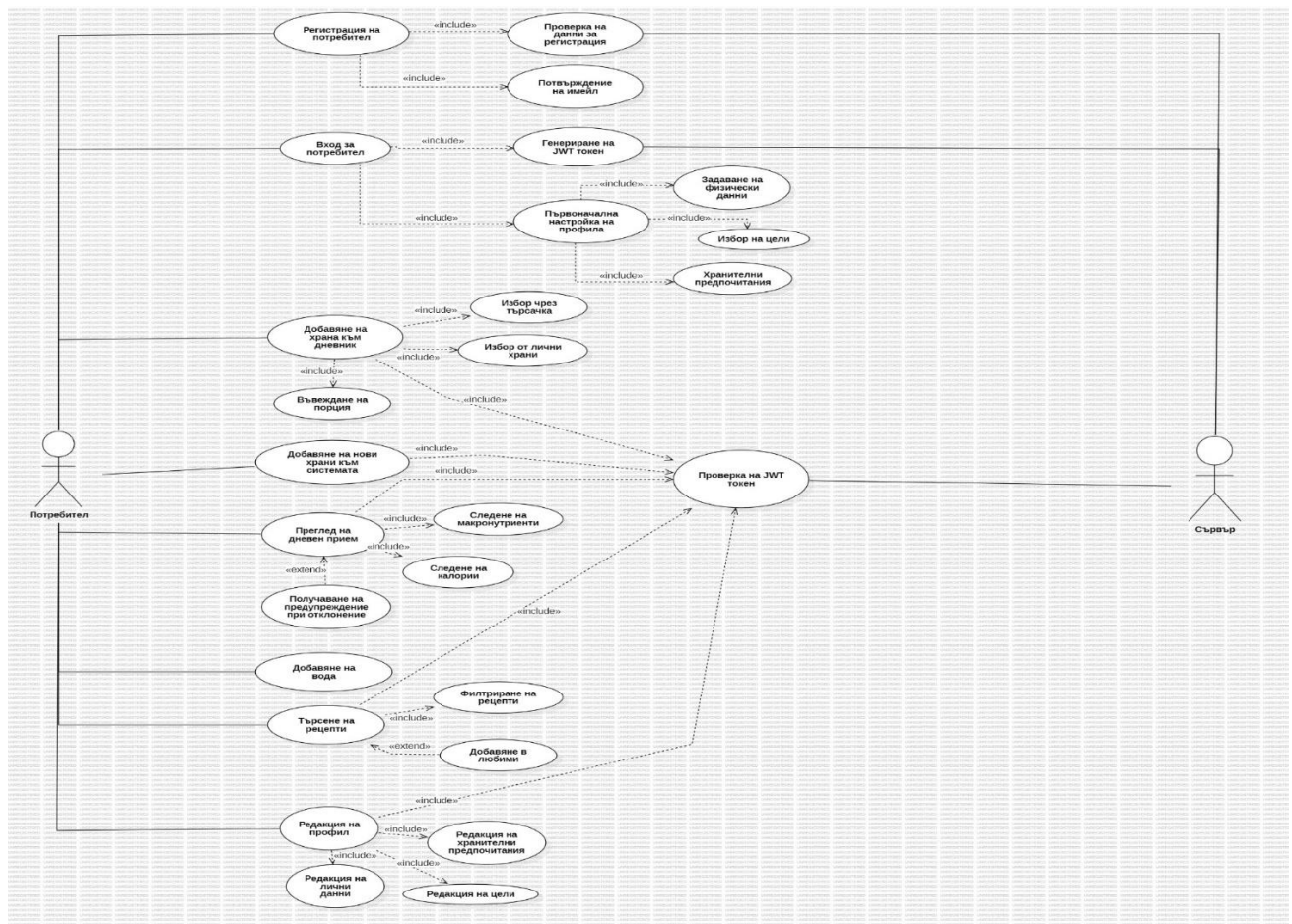
3.4.2. Основни логически обекти и връзки

1) Краен Потребител – основният обект в системата. Представява реално лице, което използва приложението, за да управлява личната си диета, да следи хранителния си прием и да анализира прогреса си.

Функционалностите, до които имат достъп обикновените потребители са:

- Регистрация в системата за обикновени потребители
- Потвърждаване на имейл след регистрация
- Вход в системата след потвърждение на имейл
- Настройване на личния профил при първо влизане в системата като администратор. Това настройване включва 7 стъпки:
 - Задаване на пол, физиологично състояние (ако полът е „женски“), година на раждане, ръст, тегло
 - Ниво на активност
 - Целево тегло
 - Целева норма
 - Преглед на зададените цели
 - Разделение на макронутриентите
 - Хранителни предпочитания – включва тип диета, алергии/непоносимости, вид кухня.
- Автоматично изчисляване на Body Mass Index (BMI)
- Добавяне на консумирана храна към дневен хранителен дневник, разделен по хранения – закуска (Breakfast), обяд (Lunch), вечеря (Dinner), междинно хранене (Snack)
- Добавяне на персонализирани храни
- Избор на храна чрез търсене
- Избор на храна от персонализирани храни на потребителя (Select from your foods)
- Въвеждане на порции, грамаж и тип хранене
- Преглед на консумирани храни през изминалите дни
- Следене на усвоени и оставащи макроси
- Следене на подробности по макронутриентите – разбивка по въглехидрат, мазнини, протеини и други витамини – изчисляват се автоматично въз основа на порцията
- Следене на предупреждения от системата ако калориите са под препоръчителния минимум или при превишаване на калорийния лимит
- Добавяне и следене на прием на вода
- Търсене/Филтриране на рецепти
- Запазване на рецепти в любими
- Редактиране на потребителско име, имейл, смяна на парола

- Редактиране на тегло, ниво на активност, разделение на макронутриентите в хранителния режим
- Редактиране на хранителните предпочитания и алергии



Фигура 4 - Use Case диаграма за потребителската част

2) Администратор – администраторът представлява потребител с разширени права, които му позволяват достъп до административния панел на системата. В архитектурата съществуват два подтипа:

2.1) Super Admin (Системен администратор) – това е администратор с пълен контрол над системата. Неговите отговорности обхващат управление на потребители, роли, анализи.

Основни функционалности на Системния администратор са:

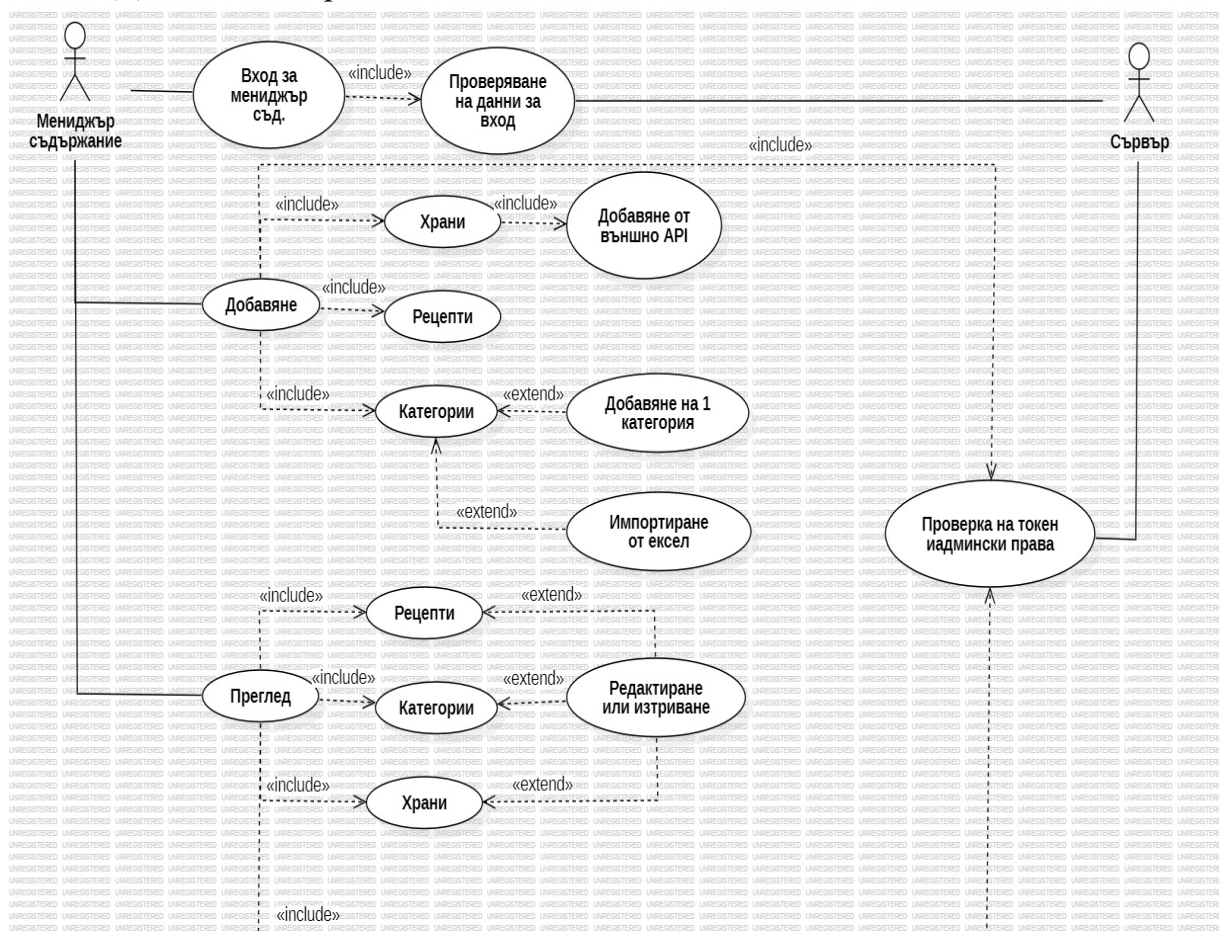
- Добавяне на нови админи
- Преглед на админи
- Редактиране на име или роля на админ
- Преглед на всички потребители
- Търсене/Филтриране на потребители
- Преглед на целите на потребителите

- Преглед на прогреса на даден потребител
- Преглед на честота на добавени храни
- Разбивка на потребителската активност

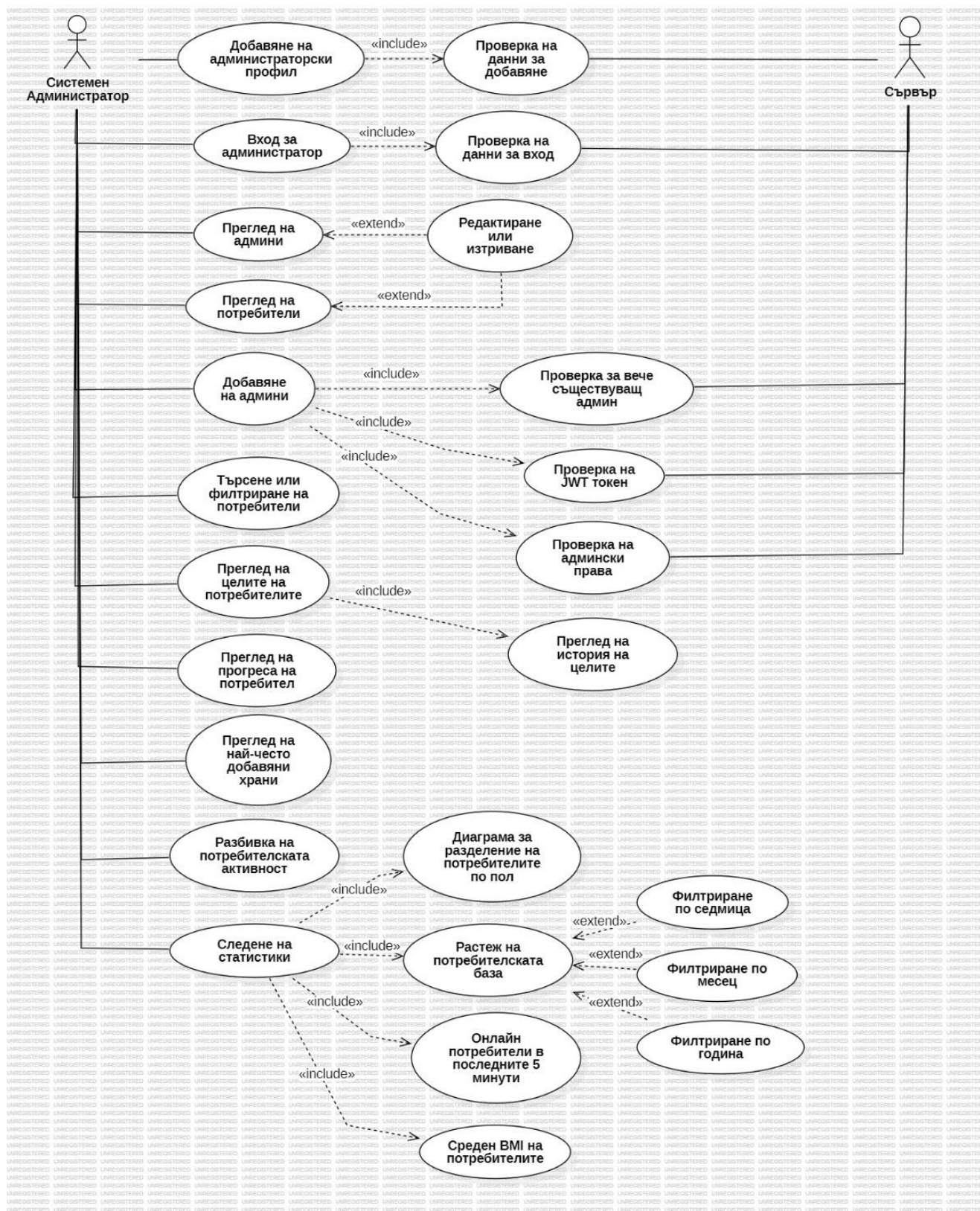
2.2) Content Manager (Мениджър съдържание) – администратор с фокус върху хранителното съдържание на системата. Няма достъп до потребителски акаунти, а работи само със съдържание, използвано от крайните потребители.

Основни функционалности на Мениджър съдържание са:

- Вход като мениджър съдържание
- Добавяне, редакция и премахване на:
 - Храни
 - Категории
 - Рецепти
- Добавяне на категории от Excel файл (Bulk import)
- Преглед на рецепти
- Преглед на храни
- Преглед на категории
- Добавяне на храни от външен източник



Фигура 5 - Use case диаграма за админ - мениджър съдържание

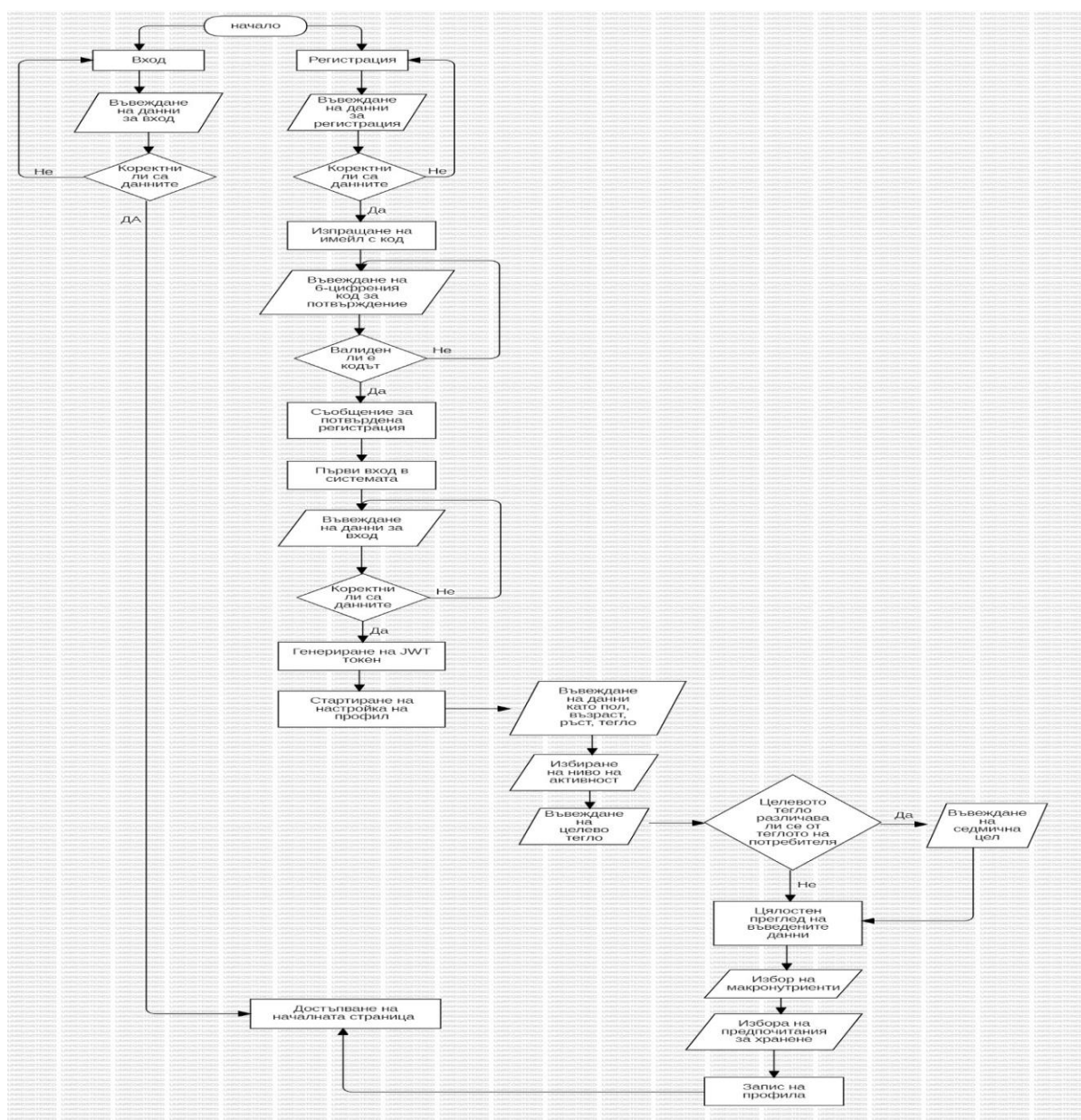


Фигура 6 - Use case диаграма за главния админ (Всички административни use case-ове изискват валиден JWT токен. Проверката се извършва автоматично при всяка заявка, дори ако не е визуално свързана в диаграмата.)

3.4.3. Диаграма и описание на работния процес на потребителската част

Диаграмата на работния процес е визуално представяне на стъпките или процесите, които са необходими за изпълнение на дадена задача или работа. Тя показва последователността от действията, потока на информация и взаимодействията между различните елементи и участници в процеса. Тези диаграми са полезни за разбиране, анализ и оптимизиране на работните процеси в различни контексти – бизнес, производство, софтуерно инженерство и други.

Работният процес на потребителската страница може да се представи чрез следната диаграма:



Фигура 7 - Flowchart диаграма за регистрация, вход и първоначална настройка на профила

След стартиране на приложението, потребителят има възможност да избере между два основни пътя – „Вход“ или „Регистрация“.

Ако потребителят избере регистрация, той попълва своите основни данни – имейл, парола, потребителско име. След попълването, системата проверява дали введените данни са коректни. Ако има грешка, потребителят се връща към формата за регистрация.

При валидни данни, системата изпраща 6-цифрен код на имейла, който потребителят трябва да въведе, за да потвърди самоличността си. След като кодът бъде въведен, системата проверява неговата валидност. Ако кодът не е правилен, потребителят е подканен да опита отново. При успешна валидация, регистрацията се счита за завършена и се показва съобщение за потвърден акаунт.

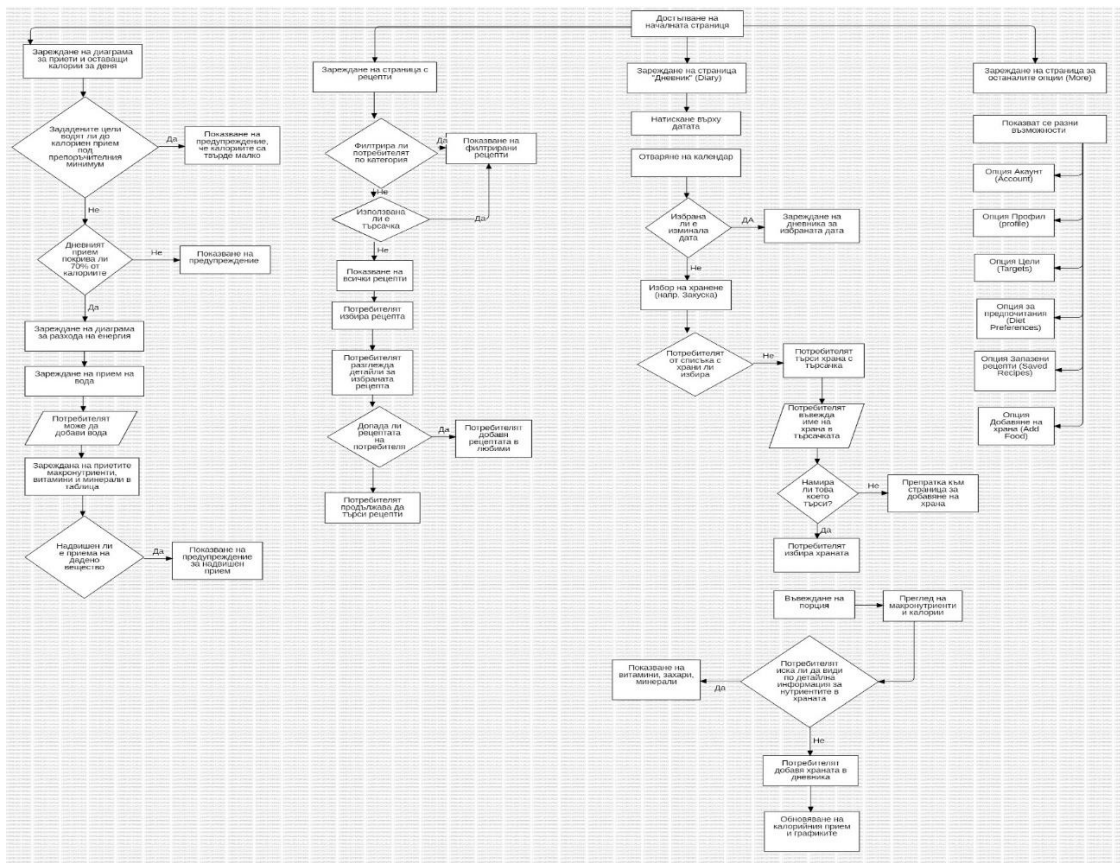
След потвърждение на имейла, потребителят извършва първи вход в системата, чрез въвеждане на имейл и парола. Системата отново проверява коректността на данните. При успех се генерира JWT токен, който удостоверява самоличността на потребителя.

След успешен вход, автоматично започва процесът по първоначална настройка на профила. Той включва:

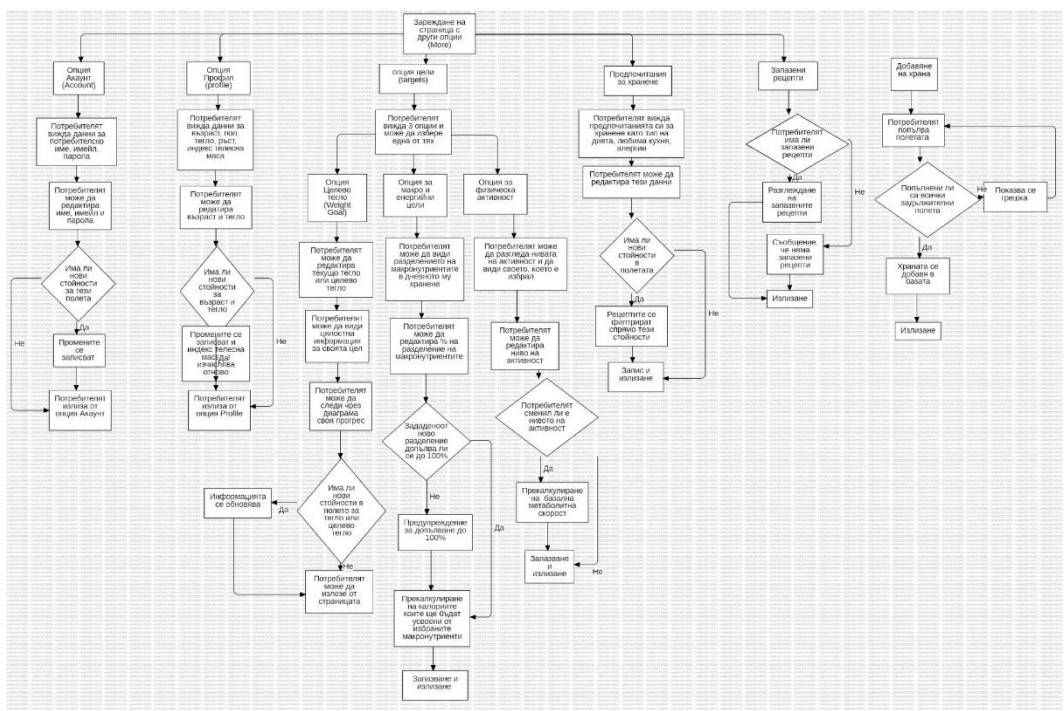
- Въвеждане на биометрични данни – пол, възраст, ръст и текущо тегло;
- Избор на ниво на физическа активност;
- Въвеждане на целево тегло – при разлика с текущото тегло се появява възможност за задаване на седмична цел (килограми, с които потребителят трябва да отслабне за една седмица);
- Преглед на введените данни – след въвеждане на горепосочените данни се изчислява индекс телесна маса и се предоставя прогнозна дата, кога потребителят може да се достигнал желаната цел, било то покачване на тегло или сваляне. Също така се генерира примерно макронутриентно разделяне. Потребителят преглежда тези данни;
- Избор на макронутриентно разделяне – какъв процент от калориите ще идват от въглехидрати, мазнини и протеини;
- Избор на хранителни предпочитания и алергии – например безглутенова диета, азиатска кухня и непоносимост към ядки;

След попълване на всички необходими данни, системата ги записва в базата, и потребителят получава достъп до началната страница на приложението, която е конфигурирана спрямо настройките които потребителят е задал.

При всеки следващ вход в системата потребителят трябва само да въведе имейл и парола. Извършва се проверка за валидност на данните и при успех, потребителят достъпва пак началната страница на системата.



Фигура 8 - Flowchart диаграма за работния процес на отделните страници



Фигура 9 - Flowchart диаграма за Други Опции (More)

Диаграмата на фигура 11 представя как потребителят взаимодейства с началната страница, разглежда рецепти и използва хранителния си дневник.

В началната страница (Home) потребителят вижда диаграма с приети калории, вода, витамини и минерали. Ако има някакво нарушение, като например калориите на потребителя да са под препоръчителния минимум, тогава потребителят вижда предупреждение.

В страницата с рецепти потребителят може да филтрира по категории или да търси с търсачката. След избор на рецепта, може да разгледа нейното съдържание, да добави рецептата към любими рецепти.

В Дневник (Diary) при натискане на датата се отваря календар. Потребителят може да избере изминал ден и да разгледа какви храни е консумирал.

Потребителят може да си избере тип хранене за деня (например Закуска). След това да потърси желаната храна, ако я открива може да разгледа детайлите за избраната храна, ако всичко е наред, да добави храната към закуската си. Ако има превишаване на цели системата извежда предупреждение. Ако потребителят не открие желаната храна при търсене може да добави свои персонализирани храни.

След добавяне на храната, усвоените и оставащите калории се преизчисляват спрямо калориите на добавената храна. Всички диаграми се актуализират.

В страница Други опции (More) потребителят вижда преки връзки към Профил, Акаунт, Цели, Предпочитания, Запазени рецепти, Добавяне на храна.

Диаграмата на фигура 12 описва действията на потребителя в раздел „Други опции“ (More), където той управлява личния си профил и хранителни цели.

В опция Акаунт потребителят може да редактира своите данни – име, имейл, парола. Ако има нови стойности, те се записват. Ако липсват или са некоректни – се показва съобщение и стойностите не се запазват.

Опция профил позволява редактиране на възраст и тегло. Ако има промяна, тя се записва, и потребителят може да види графика с прогреса си – промяна на теглото с времето.

Опция Цели (Targets) съдържа 3 подменюта:

- Целево тегло – може да се редактира или да се зададе желано тегло;
- Макронутриенти – позволява ръчна промяна на разпределението на протеини, въглехидрати и мазнини;
- Ниво на активност – позволява промяна на нивото на активност.

Ако потребителят първоначално е избрал цел „сваляне на килограми“, но след време реши да покачва, което твърде често се случва при хора, които се занимават с изграждане на тяло (bodybuilding), системата ще изисква от потребителя отново да въведе стойност за седмични килограми и спрямо това ще се прекалкулира всичко наново.

В Предпочитания за хранене (Diet Preferences) потребителят може да отбележи тип диета (веган, кето и др.), алергии и любими кухни. Ако всички необходими полета са попълнени, информацията се запазва и рецептите се филтрират според тях.

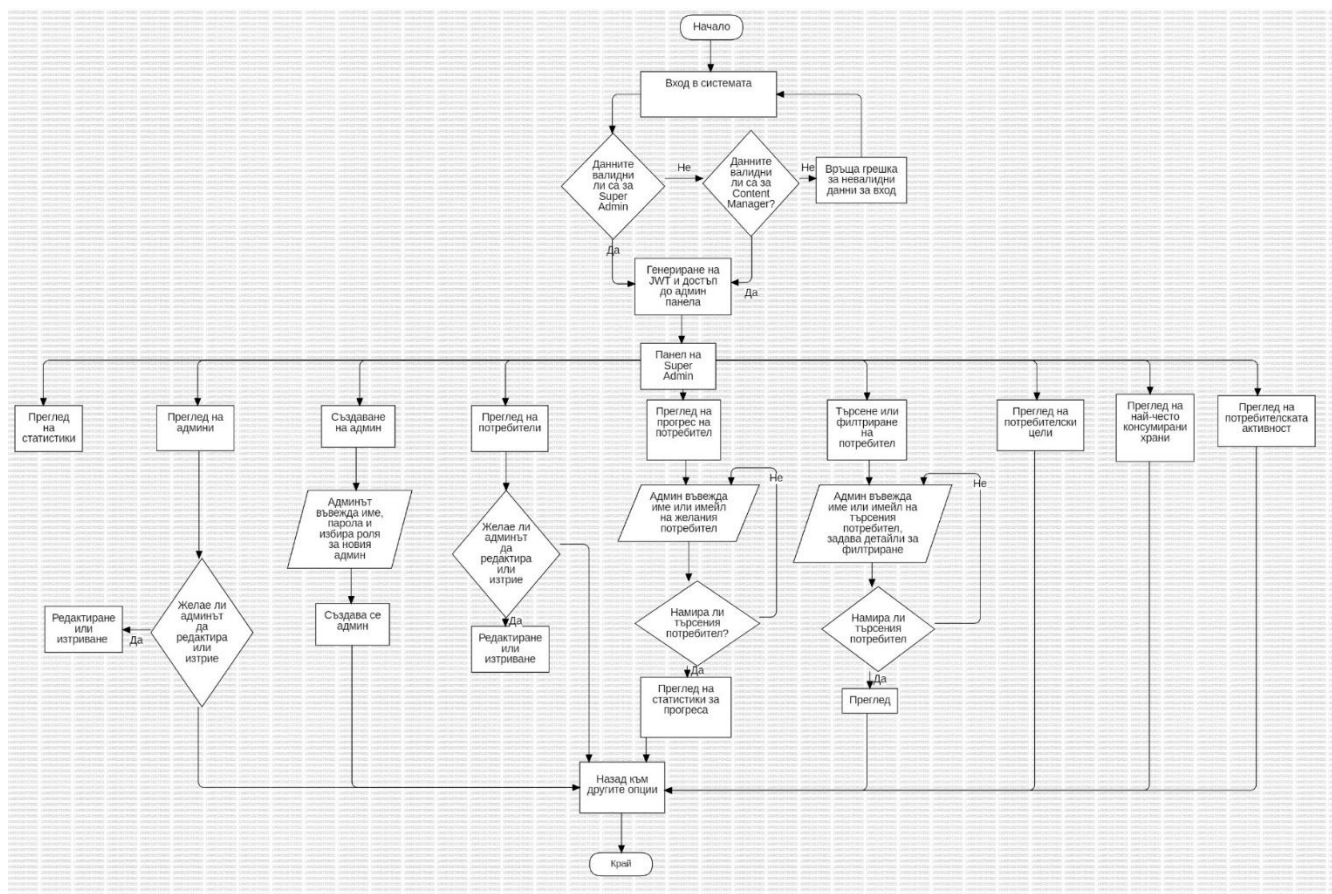
В Запазени рецепти (Saved Recipes) – ако потребителят има рецепти в списъка – те се показват, ако няма – извежда се съобщение, че няма запазени рецепти.

В опция Добавяне на храна (Add Food) потребителят може да добавя свои храни като попълни задължителните полета. Ако има пропуски, системата извежда съобщение за грешка. При пълна и валидна информация храната се записва успешно в базата и може да бъде използвана и от останалите потребители.

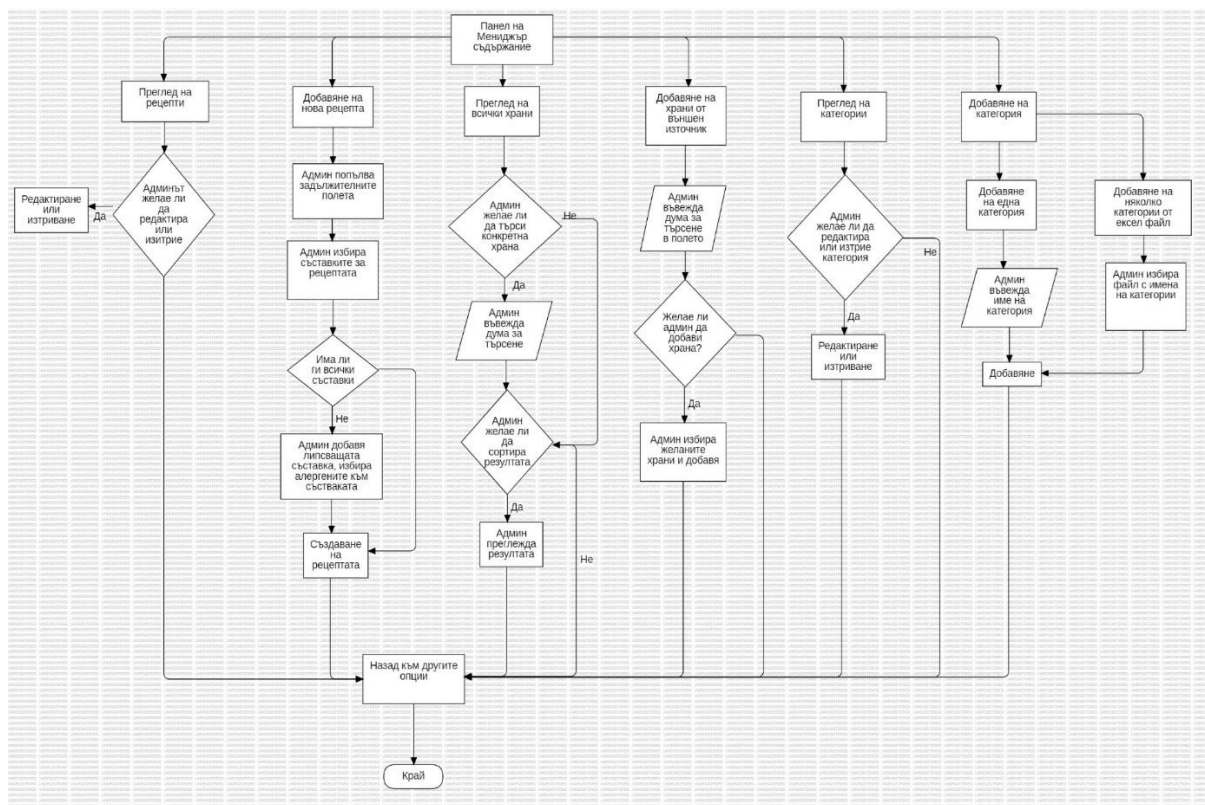
Описаният работен процес в потребителската част на системата осигурява цялостна и логична последователност от действия – от регистрацията и първоначална настройка до ежедневно използване на приложението. Всеки етап е съобразен с нуждите на крайния потребител и води до персонализирано изживяване, подпомагащо постигането на здравословни цели. Диаграмите ясно онагледяват начина, по който се въвеждат, валидират и обработват данните, както и как се визуализират резултатите и анализите. Така системата не само подпомага потребителя в реално време, но и изгражда навици чрез визуална обратна връзка и лесен достъп до функционалностите.

3.4.4. Диаграма и описание на работния процес на админската част

Административната част на системата е предназначена за потребители с повишени права – Системен администратор (Super Admin) и Мениджър съдържание (Content Manager). Тяхната роля е да поддържат системата актуална, надеждна и структурирана чрез управление на съдържание, потребители и рецепти.



Фигура 10 - Flowchart диаграма за функционалностите на главния администратор



Фигура 11 - Flowchart диаграма за функционалностите на администратор - мениджър съдържание

3.5. Структурни скици на интерфейса (Wireframes)

Структурните скици представляват първоначални визуални скици, които показват основната структура и функционалност на отделните екрани, преди да бъде приложен желаният графичен дизайн. Те са създадени на база потребителските потоци, описани в т. 3.4., и служат като междинен етап между анализа и финалния дизайн. Целта на тези скици е да дефинират разположението на елементите в интерфейса, да изяснят логиката и потребителското взаимодействие, да улеснят комуникацията между анализатор, дизайнер и програмист и да осигурят основа за UX тестване още преди изграждането на графичния облик.

Използвани са минимални визуални елементи – без цветове, шрифтове и детайли. Всеки екран съдържа основните компоненти.

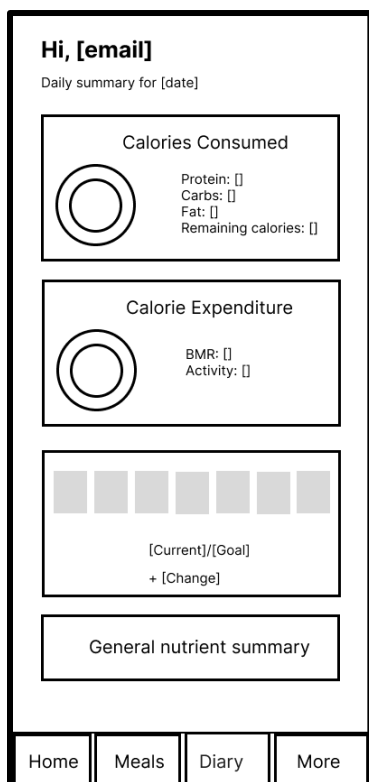
Създадени са следните wireframe-и:

- 1) Екран за вход и регистрация – полета за имейл, парола, бутони Sign in/Create Account
- 2) Екран за потвърждение на имейл – инструкции и поле за код
- 3) Начален екран – показва калориен баланс, диаграми
- 4) Дневник на храненията – секции по хранения, информация за макронутриенти
- 5) Екран с рецепти и търсене – списък с рецепти, филтри по категория

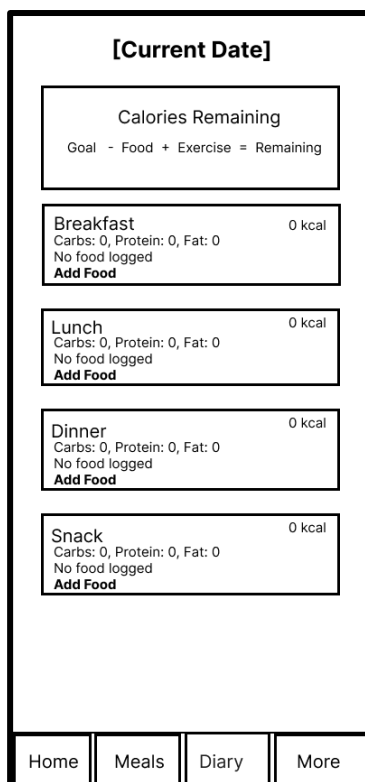
Фигура 12 - wireframe за вход

Фигура 13 - wireframe за регистрация

Фигура 14 - wireframe за потв. на имейл



Фигура 15 - wireframe за начална стр.



Фигура 16 - wireframe за дневник



Фигура 17 - wireframe за рецепти

3.6. Избор на цвetoва схема и визуални елементи

Цвetoвaтa пaлитрa нa систeмaтa e избpaнa c цeл дa пpeдизвикa ycecцaнe зa тoплинa, cпoкoйcтвиe и дoвepия, бeз дa нaтoвapвa зpeниeтo нa пoтpeбитeлeя. Изпoлзвaни ca мeки и кoнтpaстни тoнoвe, cъчeтaни c яcни икoни и лeceн зa чeтeнe шpифт.

Цвят	HEX	Използване
	#101326	Фон, текст, навигация
	#6B6E7E	Икони, вторични текстове
	#FFB766	Акценти, известия
	#FFA237	Бутони
	#EB3D00	Важни действия, предупреждения
	#FBF9F5	Главен фон на приложението
	#F7F0E6	Фон на карти като картите за рецепти

3.7. Организация на данните

Разглежданата система ще съхранява данни в база данни PostgreSQL. Начинът на съхранение на данните в базата се нарича модел. За да се определи как да бъде създаден този модел, е необходимо предварително да се дефинират целите на системата и нейните функционалности. Организацията и анализът на данните са ключови стъпки за изграждане на цялостна и ефективна система.

Моделът на представяне на данните в базата е релационен, тъй като данните са представени в таблична форма. Таблиците могат да бъдат логически свързани помежду си. Всяка таблица съдържа множество записи, а всеки запис се състои от полета (атрибути). Всеки запис притежава уникален идентификатор, наречен първичен ключ (primary key). Освен него съществуват и така наречените вторични ключове (foreign key), които се използват за връзки между таблиците, но не са задължителни. Тези връзки могат да бъдат от следните видове:

- 1:1 (едно към едно) – един запис от първата таблица се свързва с един запис от втората таблица
- 1:m (едно към много) – един запис от първата таблица се свързва с един или повече записи от втората таблица; един запис от втората таблица се свързва само с един запис от първата таблица.
- m:n (много към много) – един запис от първата таблица се свързва с един или повече записи от втората таблица; един запис от втората таблица се свързва с един или повече записи от първата таблица.

За съхранение на данните в разглежданата система е използвана NeonDB – облачна услуга, базирана на PostgreSQL. Това решение предлага напълно съвместима с PostgreSQL среда и дава възможност за мащабируемо и надеждно управление на данни.

NeonDB е избрана поради следните предимства:

- Автоматично мащабиране – базата се адаптира според натоварването, което е особено полезно при нарастващ брой потребители;
- Съвместимост с PostgreSQL – поддържат се всички стандартни релационни функционалности, включително заявки, индекси, транзакции и връзки между таблици.
- Облачна инфраструктура – няма нужда от локална поддръжка, прави се автоматичен backup, осигурява се достъп отвсякъде и лесна интеграция с други облачни услуги;

Използването на NeonDB улеснява както първоначалното изграждане на базата, така и поддръжката ѝ в дългосрочен план. Това я прави подходящ избор за съвременни уеб приложения с динамично разрастваща се потребителска база.

Следва да се създаде концептуална схема на базата данни. Това е начин на представяне на данните, без да се навлиза в детайли.

Проектираната база данни ще се състои от следните таблици и полета:

- users – таблица за регистрираните потребители в системата
- admins – съхранява информация за администраторските потребители в системата, те имат достъп до администраторски интерфейс, управление на съдържание, потребители и настройки
- activity_levels – таблица за ниво на активност на потребителя
- user_stats – съдържа измервания и изчисления, свързани с физическото състояние на потребителя
- goal_types – съдържа видовете цели, които един потребител може да избере като фокус на своя диетичен или фитнес режим. Всеки тип цел е свързан с определена стратегия за хранене
- user_goals – съхранява индивидуалните цели на потребителите, свързани с тяхното телесно тегло и здравословен прогрес.
- user_macros – съдържа информация за съотношението на макронутриенти (протеини, мазнини, въглехидрати), което е индивидуално зададено за всеки потребител. Данните в таблицата се използват при изчисление на дневните граници за макронутриенти, на база калориен бюджет на потребителя.
- user_preferences – съдържа индивидуални диетични предпочитания на потребителите. Тези предпочитания се използват при визуализиране на препоръки за рецепти
- user_allergies – съхранява информация за алергиите на потребителите към конкретни хранителни алергии. Всеки ред представлява една алергия на даден потребител. Един потребител може да има множество алергии към различни алергени. Тези данни се използват за предотвратяване на препоръчване на неподходящи рецепти
- user_favorite_cuisines – съхранява информация за любимите кулинарни стилове (национални кухни) на потребителите. Тези предпочитания се използват за визуализиране на рецепти в платформата
- user_favourite_recipes – таблицата съхранява информация за рецептите, които потребителите са отбелязали като любими. Използва се за персонализиране на препоръки, бърз достъп до предпочитани ястия. Един потребител може да има множество любими рецепти.
- diet_presets – съдържа предварително дефинирани диетични режими, базирани на разпределението на макронутриенти – протеини, мазнини и въглехидрати.

- `food_logs` – съхранява информация за това каква храна е консумирал потребителят, в какво количество, кога и за кое хранене. Използва се за проследяване на приема на калории и макронутриенти, както и за изграждане на дневни обобщения и анализ на хранителни навици.
- `daily_summaries` – съдържа обобщена информация за хранителния прием на потребителя за конкретна дата – включително калории и макронутриенти. Данните се изчисляват на базата на `food_logs` и служат за проследяване на напредъка спрямо дневните цели
- `foods` – съдържа информация за отделни хранителни продукти, включително техните наименования, хранителни стойности, източник, порция и категория. Тези данни се използват при създаване на хранителни дневници и анализи на приема
- `food_categories` – съдържа основни групи храни, които се използват за класифициране на продуктите в таблицата `foods`. Категориите улесняват търсенето, филтрирането и организирането на храни в системата
- `recipe_nutrients` – съдържа хранителни стойности, асоциирани с конкретни рецепти. Всеки запис представя стойността на един нутриент за дадена рецепта.
- `nutrient_categories` – дефинира основните групи, към които принадлежат различните нутриенти
- `nutrient_labels` – съдържа стандартизирани нутриенти имена. Всяко име е свързано с категория и единица за измерване
- `allergens` – съдържа списък с често срещани хранителни алергени, които могат да предизвикат нежелани реакции при чувствителни потребители. Тези стойности се използват при създаване на потребителски профили, рецепти
- `ingredient_allergens` - дефинира връзката между съставки и алергени, като указва кои съставки съдържат или могат да съдържат конкретни алергени.
- `recipes` – съдържа основна информация за рецепти, включително тяхното заглавие, изображение, описание, време на приготвяне, порции и здравословна оценка. Тя служи като централна таблица, свързана с други таблици като съставки, категории, инструкции.
- `ingredients` – съдържа информация за индивидуалните съставки, които се използват в рецепти.
- `instructions` – съдържа стъпка по стъпка инструкции за приготвяне на рецепти. Всяка рецепта може да има една или повече стъпки, които се подреждат по номер и описват последователността на действия.

- `recipe_ingredients` – дефинира връзката между рецепти и използваните съставки, включително оригиналното описание, точното количество и единицата.
- `recipe_categories` – представя много към много връзка между рецепти и категории, използва се когато една рецепта може да принадлежи към няколко категории
- `recipe_dish_types` – представя връзката между рецепти и типове ястия.
- `recipe_cuisines` – дефинира връзката между рецепти и национални или регионални кухни. Използва се за класифициране на рецептите според кулинарен произход
- `categories` – съдържа категории, които се използват за класифициране на рецепти според диетичния режим, особености и предпочитания.
- `dish_types` – съдържа типове хранения в рамките на деня. Използва се за предложения според времето на консумация на храненето.
- `meal_types` – дефинира основните типове хранения през деня, като се използва за класификация на приемите на храна. Помага за организиране на храненията по времеви и функционален принцип
- `cuisines` – съдържа списък с национални или регионални кухни, използвани за класифициране на рецепти според географски произход, кулинарен стил и вкусови характеристики.
- `feature_usage_logs` – записва действията на потребителите в приложението, като предоставя данни за това коя функция е използвана, от кого и кога. Използва се за анализ на поведение, подобрене на UX и проследяване на активността

Базата данни за системата за управление на здравословно хранене и рецепти е проектирана с висока степен на гъвкавост и персонализация, което я прави изключително подходяща за приложение, насочено към индивидуални нужди на потребителите. Основно предимство на тази база е възможността ѝ да съхранява и обработва детайлна информация за всеки потребител – включително лични параметри като тегло, ръст, активност, физиологично състояние, цели за тегло, алергии, диетични предпочитания и други. Това позволява изграждане на персонализирани препоръки, прецизно следене на дневен прием и анализ на хранителни навици във времето. В допълнение, базата е структурирана така, че рецепти, съставки и нутриенти да могат да се комбинират динамично, като поддържат калкулации и съобразяване с ограниченията на всеки потребител.



Глава 4: Реализация на системата

4.1. Структура на данните

В реализацията на системата е използвана релационна база от данни PostgreSQL, като данните са структурирани в множество свързани таблици. Структурата отразява концептуалния модел, описан в глава 3.7., но тук е реализирана технически чрез SQL команди.

4.1.1. Аномалии и нормализации

При използването на база от данни съществува риск от възникване на аномалии, които основно са четири вида:

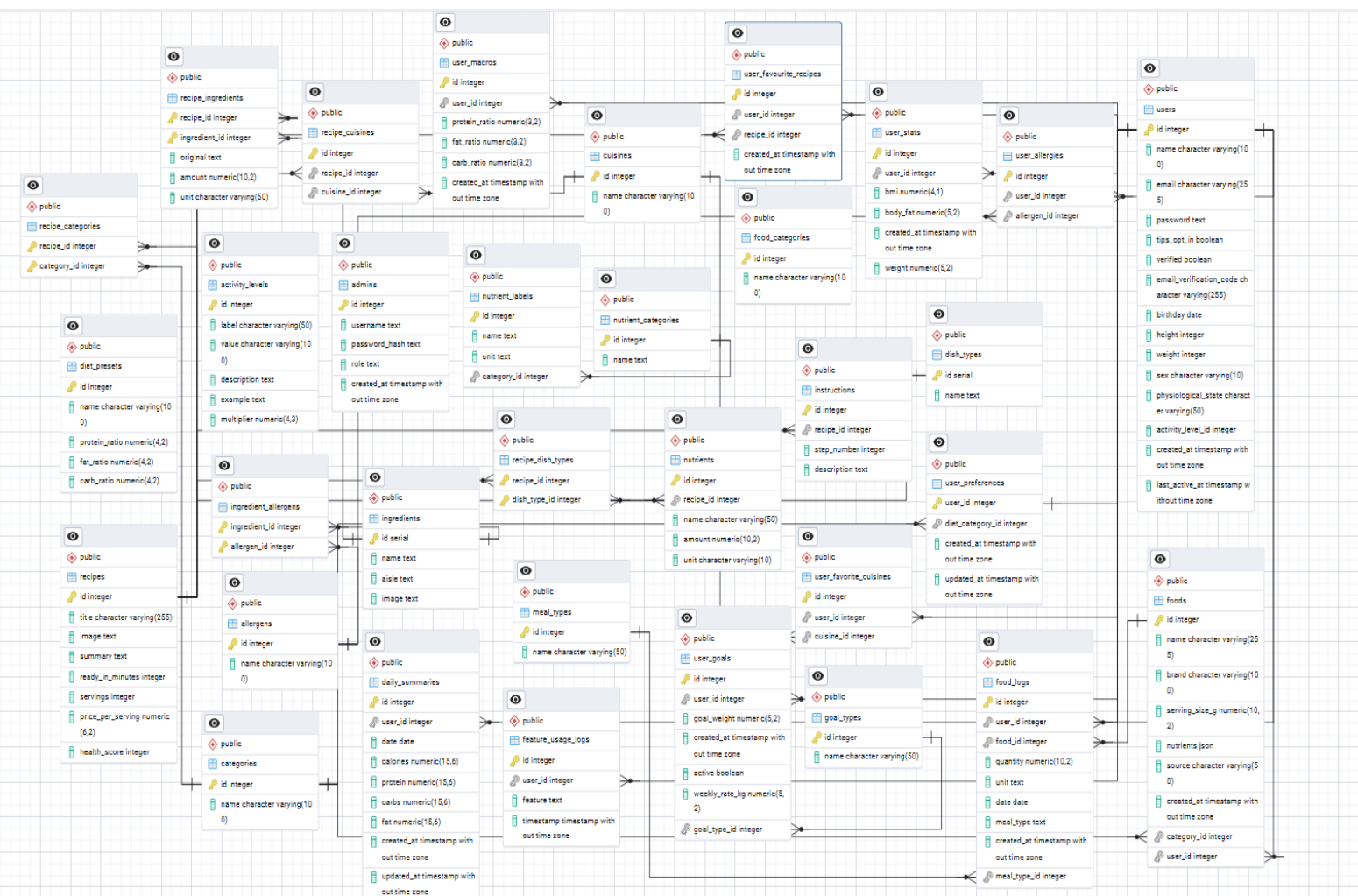
- Излишество – когато една и съща информация се съхранява на различни места в базата данни;
- Актуализация – при наличие на излишество, ако се актуализира запис, то има вероятност промяната да не се отрази навсякъде;
- Вмъкване при запис – когато се добавя запис с поле, което съдържа информация, която в момента не е налична;
- Изтриване на запис – когато изтриването на един запис налага изтриването на други свързани записи, което води до загуба на информация.

Избягването на тези аномалии се постига чрез така наречената нормализация на базата от данни. Това е процес на организиране на полетата в релационната база по начин, който отговаря на определени изисквания. Съществуват три нормални форми:

- Първа нормална форма (ПНФ) – за всяко пресичане на ред и колона съществува една единствена стойност и никога списък от стойности. Всяка клетка има само една стойност;
- Втора нормална форма (ВНФ) – изисква се всяка колона без ключ да бъде напълно зависима от целия първичен ключ, а не само от неговата част. Това са случаи, в които първичният ключ се състои от повече от една колона.
- Трета нормална форма (ТНФ) – изисква се не само всяка колона без ключ да зависи от целия първичен ключ, но и колоните без ключ да са независими една от друга.

Настоящата база данни е проектирана в съответствие с принципите на трета нормална форма. Това гарантира избягване на аномалии при обновяване, изтриване и вмъкване, подобрена цялост и валидност на данните, ясно

разграничение между логически обекти и техните връзки и улеснена поддръжка и разширяемост на системата.



Фигура 19 - релационна база от данни

4.1.2. Използвани специфични типове данни

В някои колони са използвани разширени типове данни с цел по-добра структура, ефективност и гъвкавост. Те включват:

- 1) ENUM – тип ENUM (enumerated type) е специален тип данни в релационните бази, който позволява дадена колона да съдържа само предварително дефинирани стойности. Той се използва, когато стойностите на едно поле са крайно много на брой и предварително известни. Това осигурява по-голяма валидност, четимост и оптимизация на съхранението. Предимствата на този тип е по-лесното валидиране на стойности и заема по-малко място от VARCHAR.

В проекта се използва ENUM за пол и физиологично състояние.

```
CREATE TYPE sex AS ENUM ('male', 'female');
```

```
CREATE TYPE physiological_state AS ENUM ('none', 'pregnant',  
'breastfeeding', 'menopausal');
```

- 2) JSONB (JSON Binary) – специален тип данни, който позволява съхранени на структурирани данни във формат JSON, но в бинарна, оптимизирана форма. Това го прави подходящ за полето, чието съдържание може да съдържа вложени структури, трябва да поддържа динамични ключове.

В проекта JSONB се използва в таблица `foods`, за колона нутриенти, тъй като всеки хранителен продукт съдържа различни нутриенти – един може да има само протеин, друг да има витамин Ц, цинк или Омега 3 и т.н. Ако се използват фиксирани колони за всеки нутриент, ще има много празни стойности и структурата ще е тромава. Също така JSONB позволява добавяне на нови нутриенти без промяна на схемата на таблицата. Няма нужда от `ALTER TABLE` всеки път, когато се появи нов нутриент.

4.1.3. Описание на таблиците от базата данни

Базата от данни съдържа 33 таблици, обединени чрез релационен модел. Всяка таблица има собствена функционална роля в системата, а връзките между тях осигуряват логическа цялост и нормализирана структура на данните.

1) Таблица **users**

Описание: Таблицата съхранява основна информация за всички регистрирани потребители в системата. Всеки потребител има уникален идентификатор и асоциирани лични, физиологични и поведенчески характеристики, които се използват за персонализиране на препоръките в системата.

Основни атрибути:

- `id` – тип `SERIAL` (цяло число с автоматично увеличаване), първичен ключ, уникален за всеки потребител, не може да бъде празен (`NOT NULL`), генерира се автоматично чрез последователност (`user_id_seq`)
- `name` – тип `TEXT`, задължително поле (`NOT NULL`), съдържа името на потребителя.
- `email` – тип `TEXT`, задължително поле (`NOT NULL`), съдържа имейл адреса на потребителя.
- `password` – тип `TEXT`, задължително поле (`NOT NULL`), съдържа хеширана парола.
- `tips_opt_in` – тип `BOOLEAN`, има стойност `false` по подразбиране, определя дали потребителят е съгласен да получава съвети

- `verified` – тип `BOOLEAN`, има стойност `false` по подразбиране, показва дали потребителският профил е потвърден
- `email_verification_code` – тип `VARCHAR(6)`, кратък код до 6 символа, използван за потвърждение на имейл адрес.
- `birthday` – тип `DATE`, първоначално може да е `NULL`, но при настройването на профила трябва да се добави стойност, за да може потребителят да достъпи началната страница. Съдържа датата на раждане на потребителя.
- `height` – тип `INTEGER`, първоначално може да е `NULL`, но при настройването на профила трябва да се добави стойност, за да може потребителят да достъпи началната страница. Съхранява ръста на потребителя в сантиметри.
- `weight` – тип `REAL`, първоначално може да е `NULL`, но при настройването на профила трябва да се добави стойност, за да може потребителят да достъпи началната страница. Съхранява теглото на потребителя в килограми.
- `sex` – тип `ENUM` (възможности – `male` и `female`), първоначално може да е `NULL`, но при настройването на профила трябва да се добави стойност, за да може потребителят да достъпи началната страница. Указва пола на потребителя.
- `physiological_state` – тип `ENUM` (възможности – `none`, `pregnant`, `breastfeeding` и `menopausal`), има стойност по подразбиране `'none'`, при настройването на профила може да се редактира в зависимост от състоянието на потребителя. Описва физиологичното състояние на потребителя.
- `activity_level_id` – тип `INTEGER`, първоначално може да е `NULL`, но при настройването на профила трябва да се добави стойност, за да може потребителят да достъпи началната страница. Външен ключ към таблицата `activity_levels(id)`. Представява нивото на физическа активност на потребителя.
- `created_at` – тип `TIMESTAMP`, има стойност по подразбиране – текущо време чрез `now()`, запазва датата и часа на създаване на профила.
- `last_active_at` – тип `TIMESTAMP`, има стойност по подразбиране, записва последната активност на потребителя в системата.

Скрипт за създаване на таблицата:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
```

password TEXT NOT NULL,
 tips_opt_in BOOLEAN DEFAULT false,
 verified BOOLEAN DEFAULT false,
 email_verification_code VARCHAR(6),
 birthday DATE,
 height INTEGER,
 weight REAL,
 sex sex_enum,
 physiological_state physiological_state_enum DEFAULT 'none',
 activity_level_id INTEGER REFERENCES activity_levels(id),
 created_at TIMESTAMP DEFAULT now(),
 last_active_at TIMESTAMP);

2) Таблица **admins**

Описание: Таблицата съдържа данни за административни потребители на системата. Всеки админ има уникално потребителско име, парола (в хеширан вид), роля и дата на създаване на акаунта.

Основни атрибути:

- **id** – тип INTEGER (SERIAL), задължително поле, първичен ключ, представлява уникален идентификатор на всеки администратор. Стойността се генерира автоматично чрез последователност (admins_id_seq).
- **username** – тип VARCHAR(50), полето е задължително, съхранява потребителското име на администратора. Трябва да бъде уникално за системата.
- **password_hash** – тип TEXT, задължително поле, съхранява хешираната парола на администратора, не се пази в чист текст.
- **role** – тип TEXT, задължително поле. Определя ролята на администратора в системата и в зависимост от ролята се визуализира административният панел.
- **created_at** – тип TIMESTAMP, съхранява дата и час на администраторския запис. Стойност по подразбиране – текуща дата и час (CURRENT_TIMESTAMP)

Скрипт за създаване на таблицата:

```
CREATE TABLE admins (  
    id SERIAL PRIMARY KEY,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    username VARCHAR(50) NOT NULL,  
    password_hash TEXT NOT NULL,  
    role VARCHAR(20) NOT NULL  
);
```

3) Таблица `activity_levels`

Описание: таблицата дефинира различните нива на физическа активност, които потребителите могат да изберат. Всяко ниво е описано чрез текстов етикет, стойност, коефициент за изчисления и допълнителна информация за описание и пример

Основни атрибути:

- `id` – тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всяко ниво на активност. Стойността се генерира автоматично чрез последователност (`activity_levels_id_seq`).
- `label` – тип `TEXT`, задължително поле, съхранява кратко текстово наименование на нивото на активност.
- `value` – тип `TEXT`, задължително поле, съдържа кодово име за активност, използва се при съхранение на изборите на потребителя.
- `description` – тип `TEXT`, полето може да е `NULL`, съдържа подробно описание на това какво означава конкретното ниво на активност.
- `example` – тип `TEXT`, не е задължително поле. Съдържа конкретен пример за това какво включва това ниво на активност.
- `multiplier` – тип `NUMERIC`, може да съдържа `NULL` стойности, съхранява коефициент, използван за изчисляване на дневния калориен разход на база активност.

Скрипт за създаване на таблицата:

```
CREATE TABLE activity_levels (  
    id SERIAL PRIMARY KEY,  
    label TEXT NOT NULL,  
    value TEXT NOT NULL,  
    description TEXT,  
    example TEXT,  
    multiplier NUMERIC);
```

4) Таблица **user_stats**

Описание – таблицата съхранява статистически данни за физическото състояние на потребителите във времето. Всеки запис отразява измервания като тегло, BMI към определен момент.

Основни атрибути:

- **id** – тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор на всяка статистика. Генерира се автоматично чрез последователност (user_stats_id_seq).
- **user_id** – тип INTEGER, външен ключ към таблицата users(id). Указва потребителя, за когото се отнасят измерванията.
- **bmi** – тип REAL, съхранява индекс на телесна маса, изчисляван на база ръст и тегло.
- **weight** – тип NUMERIC, съхранява теглото на потребителя в килограми към момента на измерване.
- **created_at** – тип TIMESTAMP, съхранява дата и час на създаване на записа.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_stats (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(id),  
    bmi REAL,  
    weight NUMERIC  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

5) Таблица **goal_types**

Описание: таблицата съдържа дефинирани типове цели, които потребителите могат да си поставят в системата – например за покачване на мускулна маса или намаляване на тегло. Тези стойности се използват във връзка с потребителските цели (user_goals).

Основни атрибути:

- **id** – тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки тип цел. Стойността се генерира автоматично чрез последователност (goal_types_id_seq).
- **name** – тип TEXT, задължително поле, съхранява името на целта, използва се както за визуализация, така и за логика в приложението.

Скрипт за създаване на таблицата:

```
CREATE TABLE goal_types (  

```

```
id SERIAL PRIMARY KEY,  
name TEXT NOT NULL  
);
```

6) Таблица **user_goals**

Описание: таблицата съдържа информация за цели, зададени от потребителите, свързани с промяна в телесното тегло – например покачване, сваляне или поддържане. Включва целево тегло, желано темпо на промяна и тип на целта.

Основни атрибути:

- **id** – тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор на всяка цел. Стойността се генерира автоматично чрез последователност (user_goals_id_seq).
- **user_id** – тип INTEGER, логически ключ към users(id). Указва потребителя, за когото е зададена целта.
- **goal_weight** – тип NUMERIC, указва целево тегло, към което потребителят се стреми.
- **created_at** – тип TIMESTAMP, съхранява дата и час на създаване на записа.
- **active** – тип BOOLEAN, стойност по подразбиране – true, указва дали тази цел в момента е активна за потребителя.
- **weekly_rate_kg** – тип NUMERIC, указва желаното темпо на промяна в теглото (в килограми на седмица)
- **goal_type_id** – тип INTEGER, външен ключ към goal_types(id), указва типа на целта.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_goals (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(id),  
    goal_weight NUMERIC,  
    created_at TIMESTAMP DEFAULT now(),  
    active BOOLEAN DEFAULT true,  
    weekly_rate_kg NUMERIC,  
    goal_type_id INTEGER REFERENCES goal_types(id));
```

7) Таблица **user_macros**

Описание: таблицата съдържа индивидуални макронутриентни съотношения, зададени от потребителя. Те определят какъв процент от

общия дневен калориен прием трябва да идва от протеини мазнини и въглехидрати.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор на всяка макро конфигурация. Стойността се генерира автоматично чрез последователност (user_macros_id_seq).
- **user_id** - тип INTEGER, логически ключ към users(id). Указва потребителя, за когото е зададена конфигурацията.
- **protein_ratio** – тип NUMERIC, процент от калориите, който трябва да идва от протеини.
- **fat_ratio** – тип NUMERIC, процент от калориите, който трябва да идва от мазнини.
- **carb_ratio** – тип NUMERIC, процент от калориите, който трябва да идва от въглехидрати.
- **created_at** – тип TIMESTAMP, съхранява дата и час на създаване на записа.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_macros (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  protein_ratio NUMERIC,  
  fat_ratio NUMERIC,  
  carb_ratio NUMERIC,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

8) Таблица `user_preferences`

Описание: съхранява предпочитанията на потребителя по отношение на определена диетична категория (веган, кето,..). Тази информация се използва за персонализиране на препоръки и филтри в страницата за рецепти.

Основни атрибути:

- **user_id** – тип INTEGER, задължително поле, външен ключ към users(id). Уникално идентифицира потребителя, за когото се записва предпочитанието. Всеки потребител може да избере една категория за даден момент.
- **diet_category_id** – тип INTEGER, външен ключ към categories(id). Представява избраната диетична категория от потребителя. Може да бъде празно ако потребителят няма изрично зададени предпочитания.

- `created_at` – тип `TIMESTAMP`, съхранява дата и час на създаване на записа.
- `updated_at` - тип `TIMESTAMP`, съхранява дата и час на последна промяна в записа.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_preferences (
  user_id INTEGER PRIMARY KEY REFERENCES users(id),
  diet_category_id INTEGER REFERENCES categories(id),
  created_at TIMESTAMP DEFAULT now(),
  updated_at TIMESTAMP DEFAULT now());
```

9) Таблица **user_allergies**

Описание: таблицата представлява свързваща таблица между потребителите и алергените. Използва се за отбелязване на това към кои алергени е чувствителен даде потребител.

Основни атрибути:

- `id` - тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (`user_allergies_id_seq`).
- `user_id` - тип `INTEGER`, външен ключ към `users(id)`. Указва потребителя, който има съответната алергия.
- `allergen_id` – тип `INTEGER`, външен ключ към таблицата `allergens(id)`, идентифицира алергена, към който е чувствителен потребителят.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_allergies (
  id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(id),
  allergen_id INTEGER REFERENCES allergens(id));
```

10) Таблица **user_favorite_cuisines**

Описание: таблицата отразява връзка много към много между потребителите и типовете кухни, които те предпочитат. Използва се за персонализиране на предложения и филтриране на рецепти.

Основни атрибути:

- `user_id` - тип `INTEGER`, задължително поле, външен ключ към `users(id)`. Указва потребителя, който има предпочитание към дадена кухня.

- `cuisine_id` – тип `INTEGER`, задължително поле, външен ключ към `cuisines(id)`. Идентифицира конкретния тип кухня, който потребителят харесва.

Комбинацията от `user_id` и `cuisine_id` формира съставен първичен ключ, гарантиращ, че един потребител не може да избере една и съща кухня два пъти.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_favorite_cuisines (
    user_id INTEGER NOT NULL REFERENCES users(id),
    cuisine_id INTEGER NOT NULL REFERENCES cuisines(id),
    PRIMARY KEY (user_id, cuisine_id));
```

11) Таблица **user_favourite_recipes**

Описание: таблицата съхранява информация за любимите рецепти, избрани от потребителите. Това позволява на системата да предлага бърз достъп до предпочитани рецепти и да анализира потребителски вкусове

Основни атрибути:

- `id` - тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (`user_favourite_recipes_id_seq`).
- `user_id` – външен ключ към `users(id)`. Идентифицира потребителя, който е маркирал рецептата като любима.
- `recipe_id` – външен ключ към `recipes(id)`. Идентифицира рецептата, която е добавена в любими.
- `created_at` - тип `TIMESTAMP`, съхранява дата и час на създаване на записа.

Скрипт за създаване на таблицата:

```
CREATE TABLE user_favourite_recipes (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    recipe_id INTEGER REFERENCES recipes(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

12) Таблица **diet_presets**

Описание: таблицата съхранява предефинирани макро-съотношения (протеини, мазнини, въглехидрати), които се използват като шаблон при създаване на диетични планове. Това улеснява потребителя при избора на хранителен режим, базиран на общи цели.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (diet_presets_id_seq).
- name – тип TEXT, задължително поле, съхранява името на диетичния етикет, който описва типа на хранителния режим по отношение на разпределението на макронутриентите. Стойности на полето са Moderate (умерено разпределение между протеини, мазнини и въглехидрати), Low-Carb (диета с намалено количество въглехидрати и повишени мазнини/протеини), High-Carb (диета с преобладаващ дял на въглехидратите, подходяща за висока физическа активност).
- protein_ratio – тип NUMERIC, задължително поле, указва процент от дневния калориен прием, който трябва да идва от протеини.
- fat_ratio - тип NUMERIC, задължително поле, указва процент от дневния калориен прием, който трябва да идва от мазнини.
- carb_ratio - тип NUMERIC, задължително поле, указва процент от дневния калориен прием, който трябва да идва от въглехидрати.

Скрипт за създаване на таблицата:

```
CREATE TABLE diet_presets (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL  
    protein_ratio NUMERIC NOT NULL,  
    fat_ratio NUMERIC NOT NULL,  
    carb_ratio NUMERIC NOT NULL;);
```

13) Таблица **food_logs**

Описание: таблицата съхранява информация за конкретни храни, които потребителят е консумирал в определен ден и хранене. Използва се за следене на дневния прием, калории и макронутриенти.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (food_logs_id_seq).
- user_id - външен ключ към users(id). Идентифицира потребителя, който е направил записа – добавил храна в дневника си.
- food_id - външен ключ към foods(id). Идентифицира консумираната храна.
- quantity – тип REAL, полето е задължително, указва количество от храната, което е било консумирано. Интерпретира се спрямо полето unit

- unit – тип TEXT, единица мярка за количество храна.
- date – тип DATE, задължително поле, указва датата на консумация. Стойност по подразбиране – текуща дата/ю.
- meal_type – тип TEXT, текстова версия на типа хранене, използвана като алтернатива на meal_type_id.
- created_at - тип TIMESTAMP, съхранява дата и час на създаване на записа.
- meal_type_id – тип INTEGER, външен ключ към meal_types(id). Указва вида на храненето

Скрипт за създаване на таблицата:

```
CREATE TABLE food_logs (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    food_id INTEGER REFERENCES foods(id),
    quantity REAL NOT NULL,
    unit TEXT DEFAULT 'serving',
    date DATE NOT NULL DEFAULT CURRENT_DATE,
    meal_type TEXT
    created_at TIMESTAMP DEFAULT now(),
    meal_type_id INTEGER REFERENCES meal_types(id));
```

14) Таблица `daily_summaries`

Описание: таблицата съхранява агрегирана информация за дневния хранителен прием на всеки потребител, включително калории и основни макронутриенти. Тя служи за проследяване на напредъка спрямо цели и препоръки.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (daily_summaries_id_seq).
- user_id - външен ключ към users(id). Идентифицира потребителя, за когото се отнася дневното обобщение
- date – тип DATE, задължително поле, указва конкретната дата, за която се отнасят събраните стойности, един потребител има по един запис на ден
- calories – тип NUMERIC, общо приети калории за деня, стойност по подразбиране - 0
- protein - тип NUMERIC, общо приети протеини (в грамове) за деня, стойност по подразбиране - 0

- carbs - тип NUMERIC, общо приети въглехидрати (в грамове) за деня, стойност по подразбиране - 0
- fat - тип NUMERIC, общо приети мазнини (в грамове) за деня, стойност по подразбиране - 0
- created_at - тип TIMESTAMP, съхранява дата и час на създаване на записа.
- updated_at – тип TIMESTAMP, указва дата и час на последна актуализация. Стойност по подразбиране now().

Скрипт за създаване на таблицата:

```
CREATE TABLE daily_summaries (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id),
  date DATE NOT NULL,
  calories NUMERIC DEFAULT 0,
  protein NUMERIC DEFAULT 0,
  carbs NUMERIC DEFAULT 0,
  fat NUMERIC DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  updated_at TIMESTAMP DEFAULT now());
```

15) Таблица **foods**

Описание: - таблицата съдържа данни за хранителни продукти, включително име, нутриенти, източник и възможен създател (потребител). Използва се както за вградени храни, така и за ръчно добавени от потребители.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (foods_id_seq).
- name – тип TEXT, задължително поле, съдържа име на храната
- brand – тип TEXT, полето може да съдържа NULL стойности. Съхранява марка на продукта, ако е приложимо.
- serving_size_g – тип REAL, размер на една порция в грамове. Стойност по подразбиране – 100 (грама)
- nutrients – тип JSONB, съдържа нутриентните стойности на храната в структура от тип ключ-стойност.
- Source – тип TEXT, указва източника на записа
- created_at - тип TIMESTAMP, съхранява дата и час на създаване на записа.

- `category_id` – тип `INTEGER`, външен ключ към `food_categories(id)`, определя към коя хранителна категория принадлежи продуктът
- `user_id` – тип `INTEGER`, външен ключ към `users(id)`. Ако е зададено, указва, че храната е била създадена от конкретен потребител (ръчно добавена)

Скрипт за създаване на таблицата:

```
CREATE TABLE foods (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  brand TEXT,
  serving_size_g REAL DEFAULT 100,
  nutrients JSONB NOT NULL,
  source TEXT DEFAULT 'manual',
  created_at TIMESTAMP DEFAULT now(),
  user_id INTEGER REFERENCES users(id),
  category_id INTEGER REFERENCES categories(id));
```

16) Таблица **food_categories**

Описание: таблицата съдържа различни категории за класифициране на храни, като например зърнени, млечни, месни, зеленчуци и други. Тези категории се използват за филтриране, сортиране и анализ на хранителни продукти.

Основни атрибути:

- `id` - тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (`food_categories_id_seq`).
- `name` – тип `VARCHAR(50)`, задължително поле, указва име на категория.

Скрипт за създаване на таблицата:

```
CREATE TABLE food_categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL);
```

17) Таблица **recipe_nutrients**

Описание: таблицата съхранява единични хранителни стойности (нутриенти) за дадена рецепта. Всеки ред представлява отделен нутриент като калории, протеин, витамин Ц и т.н., за определена рецепта.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (recipe_nutrients_id_seq).
- **recipe_id** – външен ключ към recipes(id), посочва към коя рецепта се отнася нутриентната стойност.
- **name** – тип TEXT, задължително поле, съдържа името на нутриента.
- **amount** – тип NUMERIC, съдържа стойността на нутриента – например 15 (грама, милиграма), според контекста на полето unit.
- **unit** – тип TEXT, съдържа единица мярка за стойността на нутриента.

Скрипт за създаване на таблицата:

```
CREATE TABLE recipe_nutrients (  
    id SERIAL PRIMARY KEY,  
    recipe_id INTEGER REFERENCES recipes(id),  
    name TEXT NOT NULL,  
    amount NUMERIC,  
    unit TEXT);
```

18) Таблица nutrient_categories

Описание: дефинира групи от нутриенти, използвани при описанието на хранителни продукти. Категориите служат за структуриране на нутриентните данни в смислени групи като „Общи“, „Витамини“, „Минерали“.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (nutrient_categories_id_seq).
- **name** – тип TEXT, задължително поле, указва името на категорията.

Скрипт за създаване на таблицата:

```
CREATE TABLE nutrient_categories (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL);
```

19) Таблица nutrient_labels

Описание: съдържа списък с допустими нутриенти, които могат да бъдат асоциирани с хранителни продукти. Всеки нутриент има име, мярка и принадлежност към категория.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (nutrient_labels_id_seq).
- **name** – тип TEXT, съхранява името на нутриента.
- **unit** – тип TEXT, единица мярка за нутриента.
- **category_id** – тип INTEGER, външен ключ към таблицата nutrient_categories(id). Определя категорията към която принадлежи нутриентът.

Скрипт за създаване на таблицата:

```
CREATE TABLE nutrient_labels (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL,  
    unit TEXT,  
    category_id INTEGER REFERENCES nutrient_categories(id));
```

20) Таблица allergens

Описание: съдържа списък с възможни алергени, които могат да бъдат свързани с потребители или съставки. Използва се за персонализиране на хранителни препоръки и за предотвратяване на излагане на алергични съставки.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (allergens_id_seq).
- **name** – тип TEXT, съхранява името на алергена, например „Gluten“, „Lactose“, „Soy“

Скрипт за създаване на таблицата:

```
CREATE TABLE allergens (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL);
```

21) Таблица ingredient_allergens

Описание: представлява свързваща таблица между съставки и алергени. Използва се, за да се определи кои съставки съдържат кои алергени и позволява системата да предупреждава потребителите с алергии.

Основни атрибути:

- `ingredient_id` – тип `INTEGER`, задължително поле, външен ключ към `ingredients(id)`. Представява конкретна съставка, която съдържа алерген.
- `allergen_id` – тип `INTEGER`, задължително поле, външен ключ към `allergens(id)`. Представява алергена, който е свързан със съставката. Комбинацията от `ingredient_id` и `allergen_id` формира съставен първичен ключ, за да се предотврати дублиране на записи.

Скрипт за създаване на таблицата:

```
CREATE TABLE ingredient_allergens (  
    ingredient_id INTEGER NOT NULL REFERENCES ingredients(id),  
    allergen_id INTEGER NOT NULL REFERENCES allergens(id),  
    PRIMARY KEY (ingredient_id, allergen_id));
```

22) Таблица **recipes**

Описание: съдържа основни данни за кулинарни рецепти, включително заглавие, време за приготвяне, брой порции, цена на порция, описание и изображение. Тя служи като централен обект, който се свързва с инструкции, съставки, кухни, типове ястия.

Основни атрибути:

- `id` - тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (`recipes_id_seq`).
- `title` – тип `TEXT`, задължително поле, съдържа име или заглавие на рецептата.
- `image` – тип `TEXT`, път или URL към изображение на ястието.
- `summary` – тип `TEXT`, кратко описание на рецептата
- `ready_in_minutes` – тип `INTEGER`, време за приготвяне на рецептата в минути
- `servings` – тип `INTEGER`, брой порции, които рецептата приготвя
- `price_per_serving` – тип `NUMERIC`, цена на една порция
- `health_score` – тип `INTEGER`, оценка за здравословност на рецептата

Скрипт за създаване на таблицата:

```
CREATE TABLE recipes (  
    id SERIAL PRIMARY KEY,  
    title TEXT NOT NULL,  
    summary TEXT,
```

image TEXT,
ready_in_minutes INTEGER,
servings INTEGER,
price_per_serving NUMERIC,
health_score INTEGER);

23) Таблица **ingredients**

Описание: съдържа списък със сурови съставки, които могат да бъдат използвани в рецепти. Това включва продукти като „домати“, „масло“, „ориз“ и други.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (ingredients_id_seq).
- name – тип TEXT, задължително поле, съдържа името на съставката.

Скрипт за създаване на таблицата:

```
CREATE TABLE ingredients (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL);
```

24) Таблица **instructions**

Описание: съхранява поетапни описания за приготвяне на рецепти. Всяка стъпка има номер, описание и е свързана с конкретна рецепта.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (instructions_id_seq).
- recipe_id – тип INTEGER, външен ключ към recipe(id). Определя към коя рецепта принадлежи съответната стъпка
- step_number – тип INTEGER, номер на стъпката в последователността.
- description – тип TEXT, текстово описание на конкретната стъпка – какво да направи потребителят на този етап на приготвянето

Скрипт за създаване на таблицата:

```
CREATE TABLE instructions (  
    id SERIAL PRIMARY KEY,  
    recipe_id INTEGER REFERENCES recipes(id),
```



```
step_number INTEGER,  
description TEXT);
```

Всяка стъпка `step_number` трябва да бъде уникална в рамките на дадена рецепта. За целта е създаден уникален индекс върху комбинацията от `recipe_id` и `step_number`. Това предотвратява дублиране на номера на стъпки в рамките на една и съща рецепта и осигурява валидна логическа последователност при изпълнение на инструкциите.

```
CREATE UNIQUE INDEX idx_unique_recipe_step ON instructions(recipe_id,  
step_number);
```

25) Таблица `recipe_ingredients`

Описание: представлява свързваща таблица между рецепти и съставки. За всяка комбинация се съхраняват конкретното количество, единица мярка и оригинален текст от рецептата.

Основни атрибути:

- `recipe_id` – тип `INTEGER`, задължително поле, външен ключ към `recipes(id)`. Определя към коя рецепта принадлежи съответната съставка
- `ingredient_id` – тип `INTEGER`, задължително поле, външен ключ към `ingredients(id)`. Определя коя съставка участва в рецептата.
- `amount` – тип `NUMERIC`, количеството на съставката, използвано в рецептата
- `unit` – тип `TEXT`, единицата мярка за количеството.
- `original` – тип `TEXT`, текстово описание, изведено директно от източника на рецептата

Комбинацията от `recipe_id` и `ingredient_id` трябва да бъде уникална за да се избегне дублиране на една и съща съставка в рамките на една рецепта.

Скрипт за създаване на таблицата:

```
CREATE TABLE recipe_ingredients (  
    recipe_id INTEGER NOT NULL REFERENCES recipes(id),  
    ingredient_id INTEGER NOT NULL REFERENCES ingredients(id),  
    amount NUMERIC,  
    unit TEXT,  
    original TEXT,  
    PRIMARY KEY (recipe_id, ingredient_id));
```

26) Таблица **recipe_categories**

Описание: тази таблица реализира много към много връзка между рецепти и диетични категории. Използва се за филтриране на рецепти по предпочитан хранителен режим.

Основни атрибути:

- **recipe_id** – тип INTEGER, задължително поле, външен ключ към recipes(id). Посочва рецептата, която е асоциирана с дадена диетична категория.
 - **category_id** – тип INTEGER, задължително поле, външен ключ към categories(id). Посочва диетичната категория, към която принадлежи рецептата
- Комбинацията от **recipe_id** и **category_id** формира съставен първичен ключ, за да не се допуска дублиране на една и съща категория за дадена рецепта.

Скрипт за създаване на таблицата:

```
CREATE TABLE recipe_categories (  
    recipe_id INTEGER NOT NULL REFERENCES recipes(id),  
    category_id INTEGER NOT NULL REFERENCES categories(id),  
    PRIMARY KEY (recipe_id, category_id));
```

27) Таблица **recipe_dish_types**

Описание: реализира връзка между рецепти и типове ястия. Използва се за класифициране и филтриране на рецепти по контекст на хранене.

Основни атрибути:

- **recipe_id** – тип INTEGER, задължително поле, външен ключ към recipes(id). Посочва рецептата, която е класифицирана
 - **dish_type_id** – тип INTEGER, задължително поле, външен ключ към dish_types(id). Посочва типа на ястието.
- Комбинацията от **recipe_id** и **dish_type_id** формира съставен първичен ключ, за да се избегне дублиране.

Скрипт за създаване на таблицата:

```
CREATE TABLE recipe_dish_types (  
    recipe_id INTEGER NOT NULL REFERENCES recipes(id),  
    dish_type_id INTEGER NOT NULL REFERENCES dish_types(id),  
    PRIMARY KEY (recipe_id, dish_type_id));
```

28) Таблица **recipe_cuisines**

Описание: реализира връзка между рецепти и кухни. Това позволява класифициране на рецепти по кулинарен произход.

Основни атрибути:

- **recipe_id** – тип INTEGER, задължително поле, външен ключ към recipes(id). Посочва рецептата, която се асоциира с конкретна кухня.
- **cuisine_id** - тип INTEGER, задължително поле, външен ключ към cuisines(id). Посочва кухнята към която принадлежи рецептата. Комбинацията от recipe_id и cuisine_id формира съставен първичен ключ, което гарантира, че една и съща рецепта не се свързва два пъти с една и съща кухня.

Скрипт за създаване на таблицата:

```
CREATE TABLE recipe_cuisines (  
    recipe_id INTEGER NOT NULL REFERENCES recipes(id),  
    cuisine_id INTEGER NOT NULL REFERENCES cuisines(id),  
    PRIMARY KEY (recipe_id, cuisine_id));
```

29) Таблица **categories**

Описание: съдържа диетични режими на хранене. Използва се за класифициране на рецепи и за настройване на потребителски предпочитания.

Основни атрибути:

- **id** - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (categories_id_seq).
- **name** – тип TEXT, име на диетичната категория.

Скрипт за създаване на таблицата:

```
CREATE TABLE categories (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL);
```

30) Таблица **dish_types**

Описание: съдържа видовете ястия, които определят ролята на дадена рецепта в едно хранене, например закуска, основно, супа, десерт. Използва се за класификация и по-лесно търсене на рецепти.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (dish_types_id_seq).
- name – тип TEXT, име на типа ястие.

Скрипт за създаване на таблицата:

```
CREATE TABLE dish_types (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL);
```

31) Таблица meal_types

Описание: дефинира типовете хранения, които се използват за категоризиране на дневния хранителен прием. Използва се в таблица food_logs, за да се отбележи по кое време на деня е прието даденото хранене.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (meal_types_id_seq).
- name – тип TEXT, име на типа хранене.

Скрипт за създаване на таблицата:

```
CREATE TABLE meal_types (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL);
```

32) Таблица cuisines

Описание: съдържа списък с кулинарни кухни по национален или регионален произход. Използва се за класифициране на рецепти и за предпочитания на потребителя.

Основни атрибути:

- id - тип INTEGER (SERIAL), задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (cuisines_id_seq).
- name – тип TEXT, име на кухня.

Скрипт за създаване на таблицата:

```
CREATE TABLE cuisines (  
    id SERIAL PRIMARY KEY,  
    name TEXT NOT NULL);
```

33) Таблица `feature_usage_logs`

Описание: служи за логване на действия на потребителите в системата – коя функционалност е използвана, от кого и кога. Това служи за анализ на поведение, подобряване на UX и откриване на неработещи или неизползвани части от приложението

Основни атрибути:

- `id` - тип `INTEGER (SERIAL)`, задължително поле, първичен ключ, уникален идентификатор за всеки запис. Стойността се генерира автоматично чрез последователност (`feature_usage_logs_id_seq`).
- `user_id` – тип `INTEGER`, задължително поле, външен ключ към `users(id)`, посочва кой потребител е използвал функционалността.
- `timestamp` – тип `TIMESTAMP WITHOUT TIME ZONE`, записва точния момент на използване на функционалността.
- `feature` – тип `TEXT`, задължително поле, име на използваната функционалност

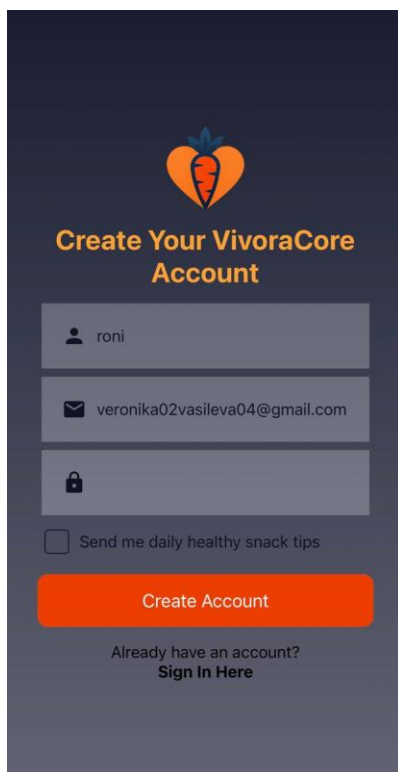
Скрипт за създаване на таблицата:

```
CREATE TABLE feature_usage_logs (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL REFERENCES users(id),  
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    feature TEXT NOT NULL);
```

4.2. Потребителски интерфейс и функционалности на системата

4.2.1. Регистрация на потребител

Потребителят може да създаде нов акаунт чрез началния екран за регистрация. Процесът изисква въвеждане на име, имейл и парола, както и (по избор) отбелязване на опция за получаване на здравословни съвети. При успешно изпращане на данните, системата създава нов потребителски запис и пренасочва към потвърждение на имейл.



Фигура 20 - страница за регистрация

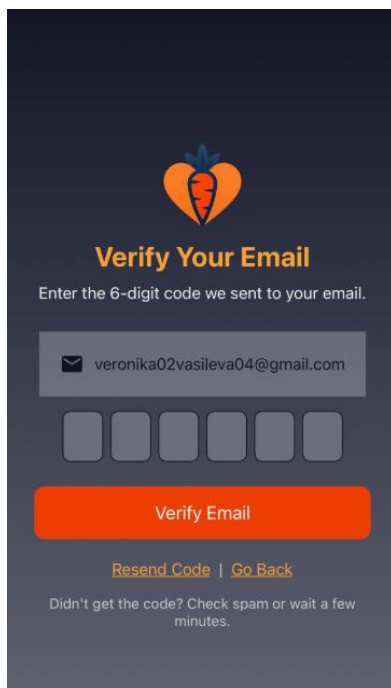
На фиг. 20 е показан екранът за създаване на акаунт. Формата включва икони и полета за – име, имейл и парола, checkbox „Send me daily healthy snack tips“, бутон „Create Account“ и препратка за вече съществуващ акаунт – “Sign in here”.

След натискане на Create Account, въведените данни се валидират (празни полета и формат на имейл). Изпраща се POST заявка към бекенд сървиса за регистрация на потребител.

Тялото на заявката изглежда по следния начин

```
{  
  "name": "testuser",  
  "email": testemail@test.com,  
  "password": "*****",  
  "tipsOptIn": false  
}
```

Ако регистрацията е успешна имейлът се запазва локално чрез SecureStore. Потребителят се пренасочва към страница за потвърждение на имейла.



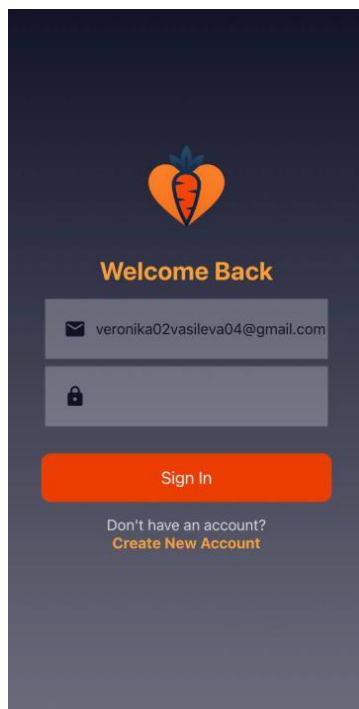
Фигура 21 - страница за потвърждение на имейл

4.2.2. Потвърждение на имейл

След успешна регистрация, системата изпраща 6-цифрен код за потвърждение на въведения от потребителя имейл адрес. Потребителят въвежда този код в полето, за да активира профила си. Проверява се дали подаденият имейл съществува. Сравнява се въведеният код с този в базата. Ако кодът съвпада, полето verified в таблица users се маркира като true. Полето с кода се занулява, за да не може да се използва повторно. Ако потребителят не получи код, може да поиска повторното му изпращане. При този иск се генерира нов 6-цифрен код, проверява се дали потребителят стои непотвърден, записва се новият код в базата и се изпраща имейл чрез SendGrid с шаблон. Ако потребителят иска да промени въведения имейл преди потвърждение, може да актуализира имейла.

Проверява се дали старият имейл съществува и е все още непотвърден, проверява се дали новият имейл не е вече зает, обновява се имейлът и се изпраща нов код за потвърждение. След успешно потвърждение на кода потребителят се препраща към екран за вход.

4.2.3. Вход в системата



Фигура 22 -стр. за вход в системата

След успешно потвърждение на имейла, потребителят може да влезе в системата чрез екран за вход (Login). Интерфейсът съдържа две полета – имейл и парола, както и бутон “Sign In”. След въвеждане на имейл и парола, клиентът изпраща POST заявка към сървиса за вход в системата. Преди да се извърши каквато и да е операция, се проверява дали има въведен имейл или парола и дали потребителят съществува и е потвърдил своя имейл.

На сървърната страна се извлича потребителят по имейл от таблицата users. Извършва се проверка дали съществува запис, профилът е потвърден (`verified=true`) и дали въведената парола съвпада (сравнение чрез `bcrypt.compare()`). При успех се генерира JWT токен с валидност 1 ден. В отговора от сървиса се изпращат и допълнителни потребителски данни като възраст, пол, ръст, тегло, хранителни цели, макроси, информация дали има запазени предпочитания, алергии и любими кухни.

Тези данни първоначално са със стойност `null` и тепърва ще бъдат добавени при настройването на профила.

4.2.4. Първоначална настройка на профил

След като потребителят влезе за първи път в системата, се стартира процес по първоначална настройка на профила, състоящ се от 7 стъпки. Целта на този процес е да се съберат лични и здравословни данни, необходими за индивидуализиране на калорийния прием и макронутриентите.

В Стъпка 1 (фиг. 23) потребителят попълва основни данни като пол (мъж/жена), дата на раждане, височина (в сантиметри), тегло (в килограми), физиологично състояние (появява се само ако е избран пол „жена“. След въвеждане на данните се натиска бутон “Next”, който валидира информацията и я изпраща към сървъра за съхранение. Клиентът изпраща PATCH заявка към бекенд маршрута за актуализация на потребителския профил. Бекенд логиката валидира всяко поле, създава динамична SQL заявка, която обновява само предоставените

полета, използва ID на удостоверения потребител (взема се от токена) и връща потвърждение при успех или съобщение за грешка.

В Стъпка 2 (фиг. 24) потребителят избира описание, което най точно съответства на ежедневната му физическа активност. Всеки избор е придружен с примери за по-голяма яснота. Опциите за нивата на активност се визуализират чрез GET заявка към таблица `activity_levels`. Изборът се запазва чрез заявка към сървъра, където се обновява полето `activity_level_id` в профила на потребителя.

В Стъпка 3 (Фиг. 25) потребителят задава желаното си тегло. То се използва при изчисленията на калориен прием и създаване на препоръки за хранене, базиран на целта – отслабване, поддържане или качване на тегло. След въвеждане на целевото тегло и натискане на Next, се изпраща PATCH заявка към бекенда със стойността на целевото тегло, ако вече е зададено, стойността се презаписва с новата.

PATCH /api/user-goals/weight

Headers: Authorization: Bearer <token>

Body: { goal_weight: 65 }

Сървърът валидира входа – стойността трябва да е положително число. Извлича текущото тегло на потребителя от таблицата `users`. Определя типа на целта – ако целевото тегло е по-ниско от текущото, то целта е отслабване (`cut`), ако е по-висока, целта е покачване (`bulk`), ако съвпада – поддържане (`maintenance`). Извлича се ID на целта от таблицата `goal_types`. Съхранява се нов запис в таблицата `user_goals` с ID на потребителя, целево тегло и тип на целта. Успешният отговор съдържа типа на целта:

```
{ "message": "Goal weight created", "goalType": "cut" }
```

В Стъпка 4 (Фиг. 26) потребителят определя своята седмична цел, тоест колко бързо иска да достигне целта си, дали иска да отслабне с 0.5 кг/седмица или да качва с 0.2 кг/седмица.

Ако целта на потребителя е отслабване, тогава ще може само да намалява скоростта на постигане на целта чрез „-“ бутона. Ако целта е покачване, тогава ще може само да увеличава скоростта чрез „+“ бутона. Допускат се само стойности в реалистични и здравословни граници. За отслабване границите са от (-1.0) до (-0.1) кг/седмица. За покачване – от (+0.1) до (+0.5) кг/седмица. При поддържане на тегло имаме фиксирана стойност 0. След натискане на бутона “Next” се изпраща PATCH заявка към сървъра с избраната стойност.

PATCH /api/user-goals/weekly-rate

Authorization: Bearer <token>

Body: { "weekly_rate_kg": -0.5 }

PATCH /api/user-goals/weekly-rate приема въведената стойност и валидира да е число и да попада в допустимия диапазон според типа на целта. Ако стойността е валидна, се актуализира запис в таблицата user_goals.

{ "message": "Weekly rate updated successfully", "goalType": "cut" }

В Стъпка 5 (Фиг. 27) потребителят получава обобщение на своя профил, калориен бюджет и макроразпределения, изчислени на базата на въведената до момента информация. Потребителят вижда обобщена информация за профила, информация за целта – тип, седмична скорост, прогнозна дата за достигане на целта, калориен дневен таргет (в kcal), калориен дефицит или излишък спрямо BMR. Потребителят вижда и информация за алтернативни разпределения на белтъчини, мазнини и въглехидрати, базирани на предварително дефинирани diet_presets.

Алгоритъмът в бекенда изчислява възраст спрямо датата на раждане, индекс телесна маса (BMI), базална метаболитна скорост (BMR). За изчисляването на BMR се използва формулата Harris-Benedict, това е количеството енергия (в kcal), което тялото изразходва в покой за поддържане на жизнените функции (дишане, циркулация и други.). Формулата за BMR по Harris-Benedict изглежда по следния начин:

- за мъже – $BMR = 10 \times \text{тегло (кг)} + 6.25 \times \text{ръст (см)} - 5 \times \text{възраст (години)} + 5$
- за жени - $10 \times \text{тегло (кг)} + 6.25 \times \text{ръст (см)} - 5 \times \text{възраст (години)} - 161$

След BMR, се изчислява общият дневен енергоразход (TDEE), това са количеството калории, които човек изгаря за едно денонощие, като се включват BMR, физическа активност, храносмилане и други ежедневни дейности. TDEE се изчислява с формулата: $TDEE = BMR \times \text{коэффициент на активност}$.

След като имаме BMR и TDEE се изчисляват калориите за дефицит или излишък (спрямо целта) – използва се $\text{weekly_rate_kg} \times 7700 / 7$, за да изчисли дневната разлика. Стойността се сумира с TDEE, за да се получи таргет калории. Прогнозната дата се изчислява въз основа на текущо тегло, целево тегло и скорост.

В Стъпка 6 (Фиг. 28) – потребителят избира какъв процент от дневния си калориен прием иска да бъде разпределен между протеини, мазнини и въглехидрати според своите предпочитания. Въвеждат се стойност в % (например 30% протеин, 20% мазнини, 50% въглехидрати). Визуализира се обща сума на

процентите, която трябва да бъде точно 100%, иначе системата показва предупреждение. След валидна стойност се активира бутон „Next“.

Една заявка за запазване на разпределението на макросите изглежда по следния начин:

```
await updateMacroPreferences(token, {  
  protein: 30, fat: 20, carbs: 50  
});
```

Пропорциите се преобразуват от проценти в стойности между 0 и 1 и се изпращат с PATCH заявка към /api/user-macros.

Тялото на заявката изглежда по следния начин:

```
{  
  "protein_ratio": 0.3,  
  "fat_ratio": 0.2,  
  "carb_ratio": 0.5  
}
```

Сървърът проверява дали $\text{protein_ratio} + \text{fat_ratio} + \text{carb_ratio} === 1$. Ако е вярно – данните се записват в таблицата `user_macros`. Ако вече има запис – той се актуализира.

В Стъпка 7 (Фиг. 29) потребителят задава предпочитан тип диета, избира алергени, които да се избягват, и любими кухни. Тази информация се използва за персонализиране на рецептите и препоръките в системата. Опциите в полетата се зареждат чрез GET заявки към бекенда. Избраните предпочитания се изпращат към /api/user/preferences

PATCH /api/user/preferences

Authorization: Bearer {token}

Content-Type: application/json

Тялото на заявката изглежда по този начин:
{ "diet_category_id": 3, "allergen_ids": [1, 2], "cuisine_ids": [4, 5] }.

Типа диета се записва или актуализира в таблицата `user_preferences`. Алергени – предишните записи в `user_allergies` се изтриват и се добавят новите. Любими кухни – аналогично в `user_favorite_cuisines`.

След успешно записване на предпочитанията, потребителят се пренасочва към началната страница (Home) на приложението, където се визуализират данните спрямо направената настройка – за калории, разпределение на макронутриенти, предпочитани рецепти и т.н. (Фиг. 30)

Фигура 23 - стъпка 1

Фигура 24 - стъпка 2

Фигура 25 - стъпка 3

Фигура 26 - стъпка 4

Фигура 27 - стъпка 5

Фигура 28 - стъпка 6

Фигура 29 - стъпка 7

Фигура 30 - начална страница

4.2.4. Добавяне на консумирана храна към дневник

Потребителят може да добави определена храна към дневника си за текуща дата и желано хранене (закуска/обяд/вечеря/междинно хранене). При добавянето се изчисляват и записват макронутриентите и калориите, като се актуализира и общият дневен прием.

В интерфейса (Фиг. 31) се въвежда порция и брой порции. Избор на тип хранене от падащ списък. За всеки хранителен продукт се показват макроси чрез интерактивна диаграма. Има индикация за надвишаване на дневните цели. Виждаме и бутон за потвърждение, който изпраща заявка за записване POST /api/food-logs. Изисква се авторизация (Bearer Token), а тялото на заявката изглежда по този начин:

```
{
  "food_id": 123,
  "quantity": 1.5,
  "unit": "100",
  "date": "2025-06-29",
  "meal_type": "Lunch",
  "meal_type_id": 2
}
```

Фигура 31 - интерфейс за добавяне на храна

Добавя се запис в таблица `food_logs` – записва се избраната храна с количество и дата. Извличат се хранителни стойности от таблицата `foods` (от JSON поле `nutrients`). Изчисляват се макроси и калории. Калории = (Въглехидрати * 4) + (Протеин * 4) + (Мазнини * 9). Актуализира се таблица `daily_summaries`, където се съхраняват обобщените стойности за деня.

4.2.5. Визуализиране на добавените храни в общия дневник

Предоставя на потребителя дневен преглед на записаните храни, разбити по хранения. Изчислява се текущият калориен прием спрямо дневната цел и се показват макросите.

Когато потребителят отвори дневника се изпраща GET заявка към сървъра с избраната дата

GET /api/food-logs?date=2025-06-29

Authorization: Bearer <token> по подразбиране е текущата дата.

Сървърът извлича всички записи за тази дата от таблицата `food_logs`, като ги свързва с данните за храната от таблицата `foods`. Всяка храна съдържа JSON с

хранителни стойности, които се използват за изчисление на макроси. След като се вземат записите, сървърът изчислява калории, протеини, въглехидрати и мазнини.

```
const totalGrams = quantity * unitGrams;

const multiplier = totalGrams / serving_size_g;

const protein = (nutrients['Protein']?.value || 0) * multiplier;

const fat = (nutrients['Total lipid (fat)']?.value || 0) * multiplier;

const carbs = (nutrients['Carbohydrate, by difference']?.value || 0) * multiplier;

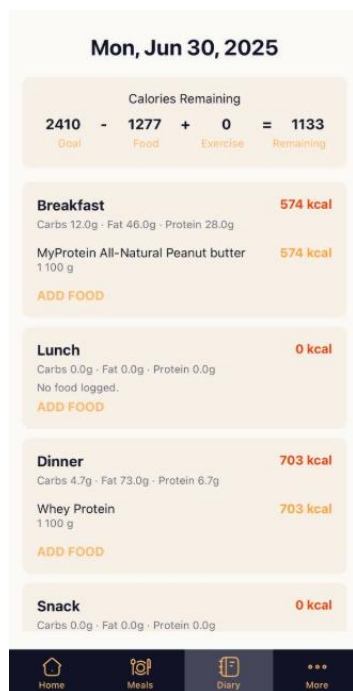
const calories = (protein * 4) + (carbs * 4) + (fat * 9);
```

Връща се обект със стойностите за всяка храна.

В интерфейса записите се групират по meal_type (Закуска, Обяд..)

```
const logs = foodLogs.filter(log => log.meal_type === meal.name);
```

Всеки хранителен запис се визуализира в контейнер за съответното хранене.



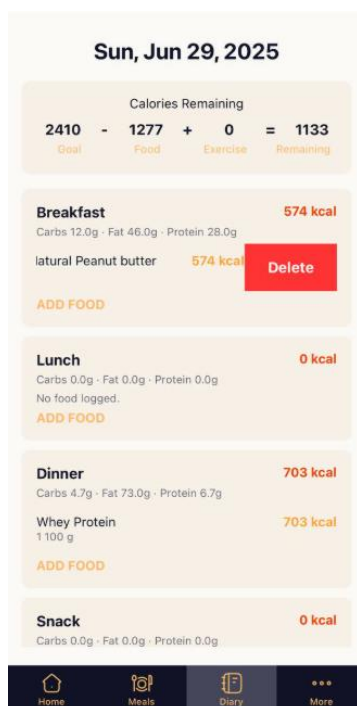
Фигура 32 - интерфейс за хранителния дневник

4.2.6. Преглед на добавени храни за изминали дни

Потребителят може да избере конкретна дата, за която иска да види записаните хранителни продукти. Календарът (Фиг. 33) дава възможност за навигация назад във времето, за да се проследява храненето по дни.

На екрана се визуализира заглавие с текущата дата. При натискане върху него се показва вграден календар. Маркираната дата е избраната `selectedDate`. Показват се месец и година с възможност за смяна. Изборът на новата дата затваря календара и презарежда дневника за нея. Компонентът, който управлява визуализацията и логиката на календара приема `selectedDate` – избраната дата, `setSelectedDate` – актуализира стойността при избор, `showCalendar` – булева стойност, която контролира дали календарът да е видим и `setShowCalendar` – функция за превключване на видимостта. Когато потребителят избере нова дата `setSelectedDate()` актуализира състоянието. Това задейства `useEffect()` в основния компонент `DiaryScreen`. Извиква се функцията `fetchData()`, която изпраща заявка към сървъра с новата дата `?date=YYYY-MM-DD`. В резултат се зареждат хранителните цели, типовете хранения и всички записани храни за деня.

Фигура 33 - визуализация на календар



Фигура 34 - изтриване на храни

4.2.7. Изтриване на добавени храни

Позволява на потребителя да премахне грешно въведена или ненужна храна от дневника за дадена дата. Това автоматично актуализира общите стойности на макронутриенти и калории за деня.

Всеки хранителен запис е обвит в компонент `<SwipeableRow />`. Потребителят може да плъзне вляво върху елемент от списъка и да види бутон за изтриване. При натискане на бутона се изпраща `DELETE` заявка към сървъра, чрез функцията `getToken()` се получава токен, за да се удостовери потребителят и ако храната се изтрие успешно списъкът с добавени храни за деня се обновява. В сървърната част маршрутът използва `authenticateToken middleware`, която гарантира, че потребителят е вписан и има право да достъпва ресурси, свързани с неговия акаунт. След това сървърът търси съответния запис в базата, заедно с хранителната

информация от таблицата `foods`. Ако записът съществува, сървърът изчислява колко протеин, мазнини и въглехидрати и калории съдържа този запис според количеството и порцията. Тези стойности се изваждат от обобщената дневна статистика в `daily_summaries`, за да се коригира общият прием за деня. Накрая записът се изтрива и промените се записват с транзакция. Ако има грешка, всичко се отменя и се връща съобщение за неуспех.

4.2.8. Изчисляване на оставащи калории

Изчисляването на `remaining calories` (оставащи калории) в компонента `Diary Screen` става чрез проста аритметика, базирана на целевия калориен прием и вече консумираните калории за избрания ден.



Фигура 35 - оставащи калории

```
const macros = calculateMacros(foodLogs);
```

```
const remainingCalories = energyTarget - macros.calories;
```

Функцията `calculateMacros(foodLogs)` сумира калориите и макронутриентите от всички записани храни за дадения ден. `energyTarget` е стойността, получена от сървъра – целевият дневен калориен прием на потребител според неговите цели. Оставащите калории са разликата от целта и консумираните до момента калории за деня. Ако потребителят е надвишил дневната цел (стойността е отрицателна), числото става червено, за да го предупреди визуално.

4.2.9. Екран с рецепти

Тук се визуализира списък с рецепти, персонализирани според вкусовите предпочитания и ограничения на потребителя. При зареждане на екрана се извършват няколко заявки към сървъра – извличане на категории рецепти `getCategories(token)` – зареждат се всички налични категории. След категориите се извличат потребителските предпочитания – с функцията `getUserPreferences(token)` се получават избран хранителен режим, предпочитани кухни и алергени, които да бъдат избегнати. Чрез тези функции на екрана се визуализират само рецепти, отговарящи на всички изисквания.

На екрана има поле за търсене на рецепти. След като се направи заявка към сървъра за всички рецепти (или рецепти по категория), те се съхраняват в променлива `data`.

```
if (searchTerm) {  
    filteredData = filteredData.filter(item =>
```



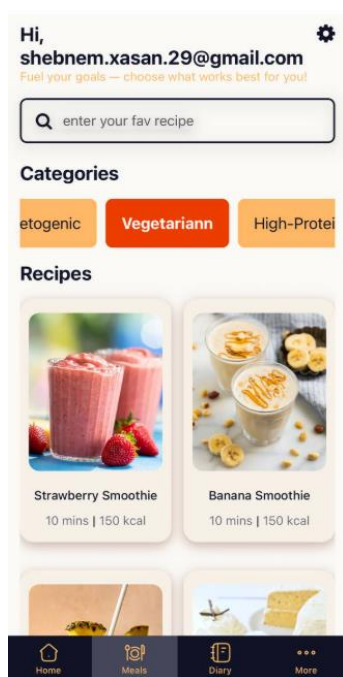
```
item.title?.toLowerCase().includes(searchTerm.toLowerCase()));}
```

searchTerm е текстът, въведен от потребителя в полето за търсене. Всяка рецепта (item) има поле title (заглавие). toLowerCase() се използва за сравнение без значение главни или малки букви. Ако заглавието на рецептата съдържа ключовата дума остава във филтрирания списък.

След прилагането на всички филтри (категория, търсене, кухня, алергени), на екрана се визуализират резултатите под формата на карти с рецепти (RecipeCard компоненти). Всяка карта съдържа:

- Изображение на рецептата – зарежда се предоставен URL
- Заглавие на рецептата – показва се до два реда центрирано
- Време за приготвяне – в минути (поле ready_in_minutes)
- Калории – извлечени от recipe_nutrients, ако са налични

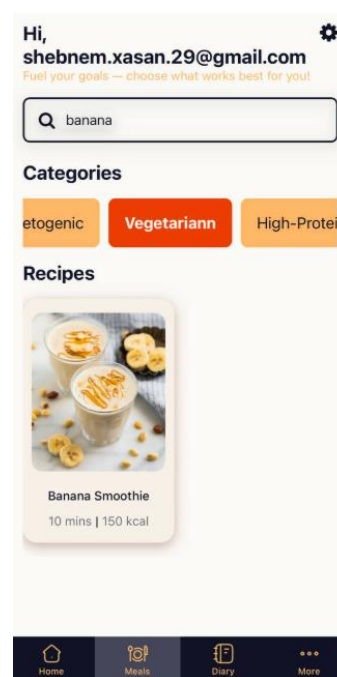
Картите са разположени едно до друго в две колони, като всяка втора има отместване, за да се запази виртуален ритъм.



Фигура 36 - филтрирани рецепти



Фигура 37 - страница с рецепти

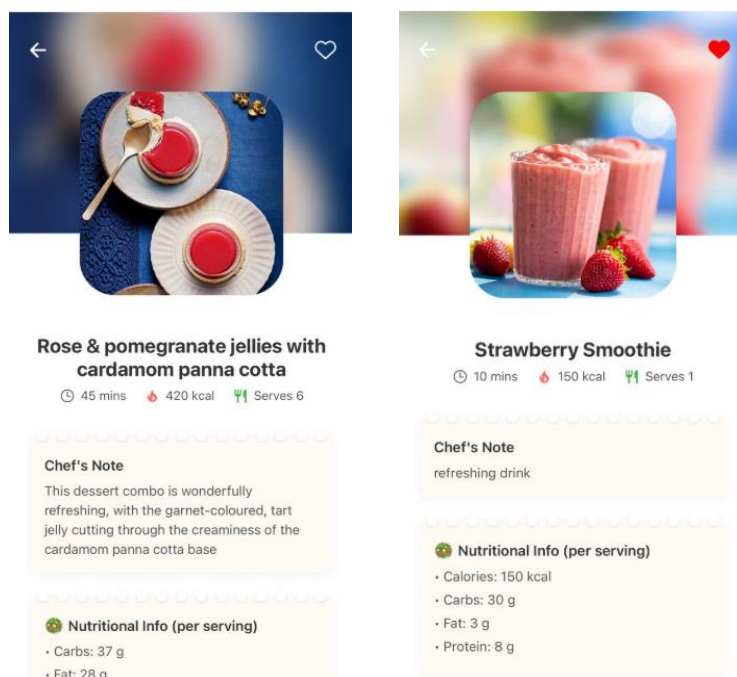


Фигура 38 - търсене на рецепта

4.2.9. Екран с детайли за рецепта

При натискане върху дадена рецепта се извиква функцията onPress, която пренасочва потребителя към екран с подробности за избраната рецепта. Основната цел на този екран е да предостави ясна и визуално привлекателна информация относно избраната рецепта – нейна снимка, хранителни стойности, нужните съставки и инструкции за приготвяне.

Още при зареждане на екрана се извършват няколко заявки към сървъра, като всяка използва идентификатора на рецептата (id). Чрез GET `/:id/ingredients` се получава списък със съставки включващ име на съставка (name), оригинално описание (original), количество (amount) и мерна единица (unit). Чрез GET `/:id/instructions` се зареждат стъпките на приготвяне, подредени по номер (step_number), всяка съдържа описание (description). Хранителните стойности се подават чрез `params.nutrients`, декодират се и се визуализират като списък от елементи с името и количеството.



Фигура 39 - страници с детайли за рецепта

4.2.10. Запазване на рецепта в любими

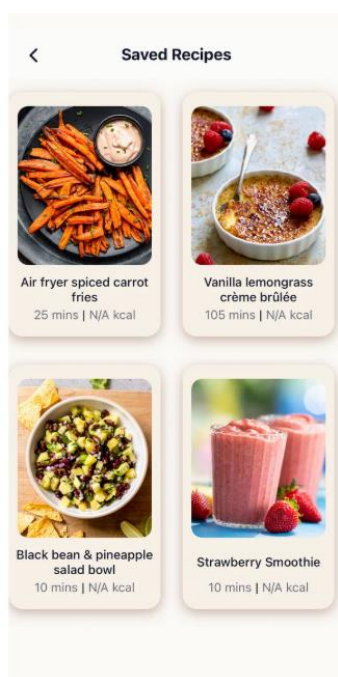
Потребителят може да маркира рецептата като любима чрез иконката с форма на сърце в горния десен ъгъл (Фиг.39). Когато потребителят натисне иконката със сърце в екрана за детайли на рецептата, приложението стартира функцията `addFavorite`, ако рецептата не е добавена в любими. Функцията `addFavorite(recipeId, token)` изпраща POST заявка към сървъра, с тяло `recipeId` – идентификатор на рецептата, която ще се добавя. В сървърната част се извлича `userId` от токена и `recipeId` от заявката. Прави се вмъкване в таблицата `user_favourite_recipes`, която пази връзки между потребители и рецепти. За да се предотвратят дубликати, заявката използва `ON CONFLICT (user_id, recipe_id) DO NOTHING`, тоест ако връзката вече съществува – няма да се направи нищо. Ако операцията е успешна, се връща отговор `{ "message": "Recipe favorited" }`.

Преди добавянето на рецепта в любими, се прави проверка чрез `checkIfFavorited(recipeId, token)`. Тази заявка връща `{ "isFavorited": true/false }`. На

база това се сменя иконката на сърцето – празна или запълнена. Ако рецептата вече е в списъка и потребителят натисне сърцето, се изпраща: DELETE /favorites/:recipeId. Така рецептата се премахва от таблицата user_favourite_recipes.

4.2.11. Преглед на любими рецепти

След добавянето на рецепти в любими, потребителят може да ги намери в екрана “Saved Recipes”. Компонентът Saved Recipes визуализира `<Recipes favoritesOnly={true} />`, тоест компонентът Recipes трябва да зареди само любимите рецепти. Компонентът Recipes проверява дали е зададен favoritesOnly. Ако да, извиква функцията getFavorites(token). Тази функция прави GET заявка до сървъра GET /favorites. Сървърът връща списък с рецепти, които текущият потребител е маркирал като любими.



Фигура 40 - екран с любими рецепти

4.2.12. Начален екран – дневен преглед

Това е началната страница на приложението, нейното основно предназначение е да предостави дневен преглед на хранителния прием, енергийните стойности, BMR, TDEE, вода и макронутриенти.

Първоначално при фокусиране на екрана (useFocusEffect) се взима токен от SecureStore, който се използва за декодиране на потребителските данни с jwtDecode. Зарежда се потребителското име и ID, което се използва при останалите заявки.

След това се извършват няколко заявки към сървъра:

- `getEnergySummary(token)` – извлича метаболитни данни като BMR (базов метаболизъм), TDEE (общ дневен енергоразход), целеви калориен прием, както и съотношения за макронутриенти според потребителската цел.
- `getFoodLogs(token, date)` – зарежда въведените от потребителя храни за деня. Тези данни се използват за изчисляване на приетите калории и макронутриенти чрез функцията `calculateMacros()`.
- `getNutrientSummary(token, date)` – предоставя детайлна разбивка на приетите витамини, минерали и микроелементи за същата дата, базирано на състава на записаните храни. Тези данни се намират в таблица `foods`, колона `nutrients`.

На основата на тези данни се визуализира калорийна кръгова диаграма (DonutChart) чрез компонента `CaloriesCard`, който показва какъв процент от приетите калории идват от протеин, въглехидрати и мазнини. Ако калорийният прием е под 70% от препоръчания или под 1200kcal, се появява предупредително съобщение (Фиг. 41) за рисковете от недостатъчно хранене.



Фигура 41 - предупреждение

Компонентът `EnergyExpenditure` показва стойностите на BMR и TDEE на потребителя, изчислени динамично спрямо възраст, пол, тегло, ръст и ниво на активност.

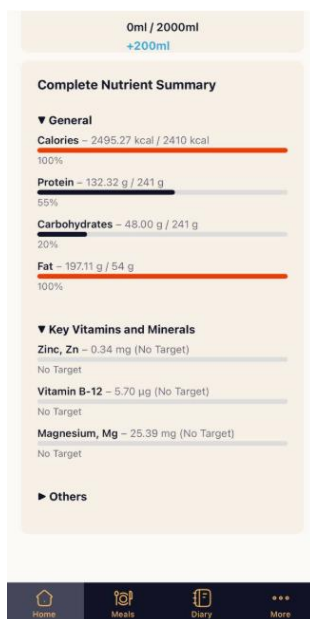
В края на екрана се визуализира пълният отчет (витамини, минерали и други.), разделен е в категории “General”, “Key Vitamins and Minerals” и “Others”. Всяка категория може да бъде разгъната или скрита чрез потребителски контрол. За всяко хранително вещество се визуализира стойността му, целта (ако има такава) и прогрес лента, която показва какъв процент от дневната цел е постигната. В раздела за витамини и минерали се показват стойностите на приетите микроелементи за деня, но за тях не е зададен конкретен таргет, защото определянето на оптимални стойности за всеки витамин и минерал зависи от много индивидуални фактори като възраст, здравословно състояние, физиологично състояние и др.

За разлика от тях, при макронутриентите е зададена ясна дневна цел и ако потребителят я надвиши, в екрана се визуализира предупредително съобщение.

Това цели да подпомогне потребителя да поддържа балансиран и безопасен хранителен режим.



Фигура 42 - начален екран



Фигура 43 - начален екран - обобщение на нутриенти и микронутриенти



4.2.13. Прием на вода

Целта на добавянето на вода в приложението е да насърчи потребителя да следи и поддържа хидратационен режим през деня. Приемът на достатъчно количество вода е ключов за редица физиологични процеси – включително метаболизма, храносмилането, концентрацията и общото здравословно състояние. Добавянето на вода в приложението се реализира чрез компонента WaterTracker, който визуализира седем бутилки, всяка разделена на 2 нива по 100 ml. При всяко натискане на бутона „+200ml“, в системата се добавят още 200ml към текущото количество приета вода за деня. Приетото количество вода не се съхранява в сървъра или базата данни, а се записва локално в устройството на потребителя чрез AsyncStorage. Причината за това е , че водата е относително неформален показател, не е свързана с хранителни логове или анализи и не изисква трайно съхранение. Водните данни се съхраняват само до края на текущия ден (до 00:00 часа). След това автоматично се изчистват при смяна на датата.



Фигура 44 - добавяне на вода

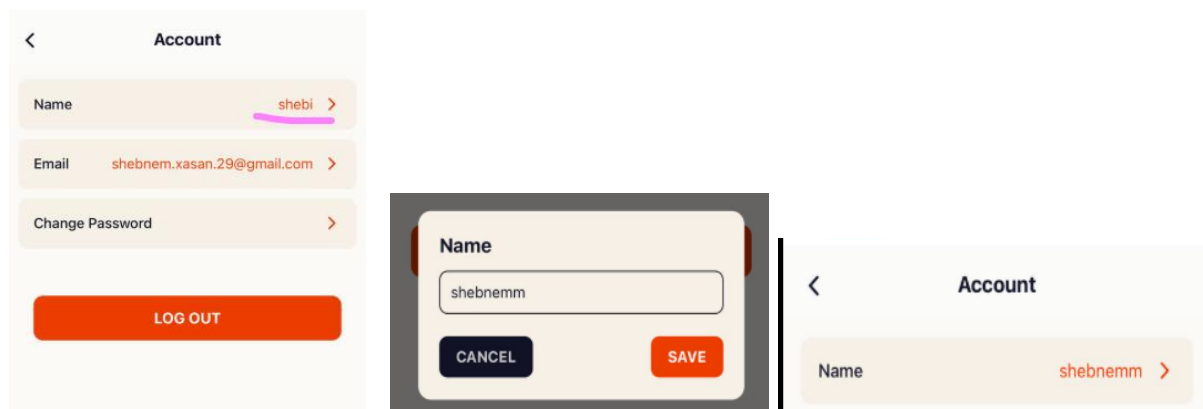
4.2.14. Редактиране на лични данни

Системата позволява на потребителите да преглеждат и редактират основната информация в акаунта си – имейл, име и парола. Когато потребителят отвори този екран, автоматично се извличат текущите му данни чрез `fetchAccountDetails`, която изпраща заявка до сървър и получава името и имейла на потребителя. Тези стойности се визуализират в отделни редове, всеки със заглавие. При натискане върху поле, което подлежи на редакция, се задава кое поле се редактира, подготвя се текущата стойност и се показва модален прозорец `EditFieldModal`, където потребителят може да въведе нова стойност.

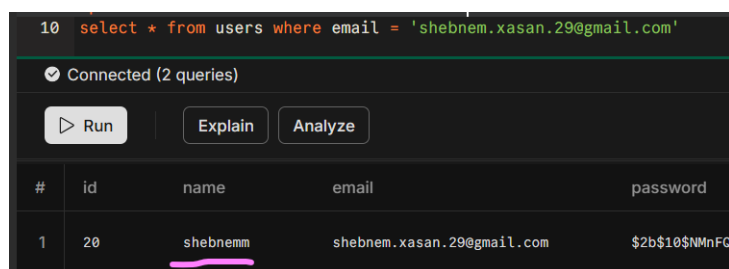
```
const handleEdit = (field) => {  
  setEditingField(field); // 'Name' или 'Email'  
  setEditValue(field === 'Name' ? userName : userEmail);  
  setModalVisible(true); // показва модала};
```

След потвърждение, чрез функцията `updateAccountInfo` се изпраща `PATCH` заявка към сървър, който актуализира съответната информация в базата данни. Промяната се отразява и незабавно на екрана.

За смяна на паролата се използва отделен модален прозорец `ChangePasswordModal`, който приема текущата и новата парола. След това с `updatePassword` се прави заявка към `/api/account/password`. Сървърът валидира текущата парола. Ако е вярна, хешира новата и я записва в базата.



Фигура 45 - редактиране на лични данни



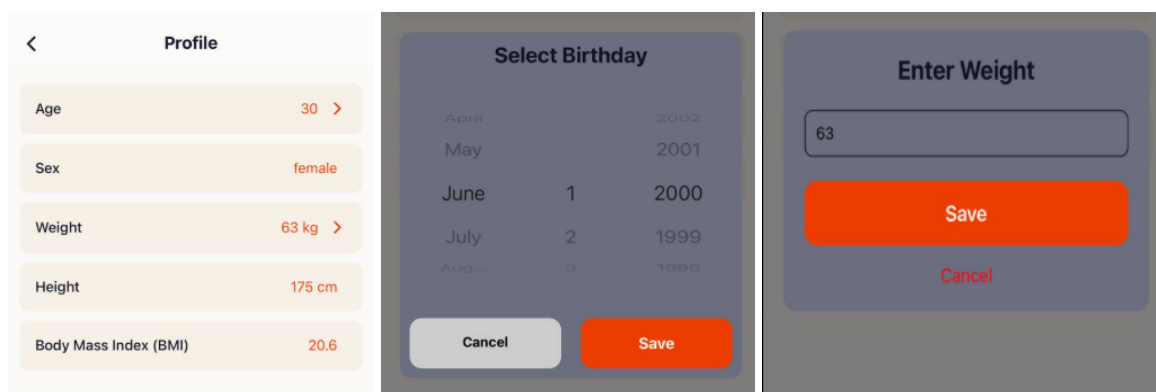
Фигура 46 - редактираните данни в базата

4.2.15. Редактиране на основни физически данни

В този екран потребителят вижда списък с основна информация за своя профил, представена във вид на подредени редове с етикети и стойности. Всеки ред съдържа две основни части – Етикет (Label) – Age, Sex, Weight... и Стойност (Value) – например 25, female,... . В този екран потребителят може и да редактира две основни стойности от своя профил – рождена дата и тегло. Редакцията става чрез модални прозорци, които се отварят при натискане върху съответния ред от списъка, за това се използват компонентите AddOrUpdateAge и AddOrUpdateWeight.

Когато потребителят натисне върху реда „Age“, се отваря модал с избор на ден, месец и година. След потвърждение датата се преобразува във формат YYYY-MM-DD. Изпраща се заявка към сървъра с новата дата. След успешно обновяване, приложението презарежда профилните данни, за да се покаже актуализираната възраст. Възрастта трябва да е по-голяма от 13. Промяната се запазва и в базата.

При натискане върху реда “Weight”, се отваря друг модален прозорец, в който потребителят може да въведе актуално тегло. Потребителят въвежда стойност в килограми и натиска бутона за потвърждение. След това приложението записва новото тегло в базата данни, като актуализира текущото тегло в таблицата users и добавя нов запис в таблицата user_stats със стойност и датата на промяна. Чрез този запис в таблицата user_stats, се предлага възможност за проследяване на напредъка назад във времето. След записа, прозорецът се затваря и данните на екрана се обновяват автоматично.



Фигура 47 - редактиране на потребителски данни

4.2.16. Редактиране на целево тегло

На екрана „Weight Goal“, потребителят вижда текущото си тегло, зададеното целево тегло, тип цел, скорост на целта, прогнозна дата за постигане на целта, калориен таргет и графика с история на теглото. От този интерфейс потребителят може отново да редактира текущото си тегло, както в точка 4.2.15. Също така



Фигура 48 - редактиране на целево тегло

може да редактира и целевото тегло. Когато потребителят натисне върху реда “Weight Goal”, се отваря модален прозорец, където той може да въведе нова стойност.

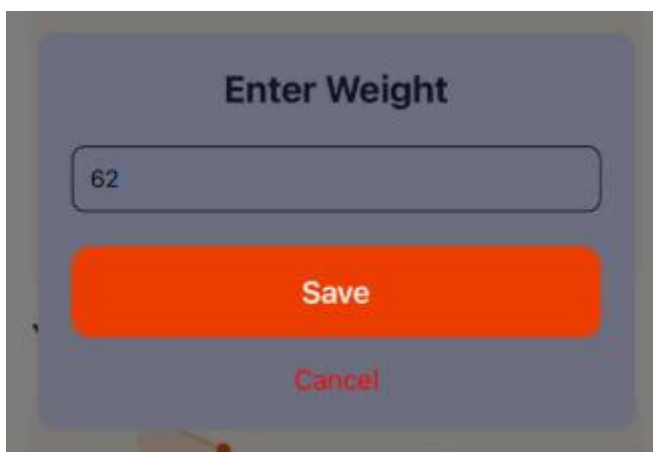
След потвърждение на новото целево тегло се изпраща PATCH заявка с новата стойност. Изчислява се типът цел спрямо текущото тегло. Ако новата цел не е поддържане на тегло, и новата цел се различава от старата по тип, тогава се отваря втори модал, където потребителят задава седмична скорост на целта. (колко килограма на седмица иска да сваля/качва).

Сървърът приема новата стойност, определя типа ѝ, намира ID за съответния тип в таблицата `goal_types`. Вмъква нов запис в таблицата `user_goals` с `user_id`, `goal_weight`, `goal_type_id`, текуща дата, нищо старо не се изтрива, за да се запази историята на целите.

След обновяване на целта, се извиква друг endpoint, който изчислява BMR, TDEE, изчислява дневния калориен таргет спрямо желаната скорост, прогнозира дата за достигане на целта и извежда макроси на база енергийната цел и избрано съотношение.

В началната страница и в екрана с Дневник за консумирани храни всички данни се актуализират автоматично спрямо новата цел.

На фигура 48 текущото тегло на потребителя е 63 а целевото тегло е 64, това означава, че потребителят иска да покачва тегло. Ще сменим целевото тегло от 64 на 62.



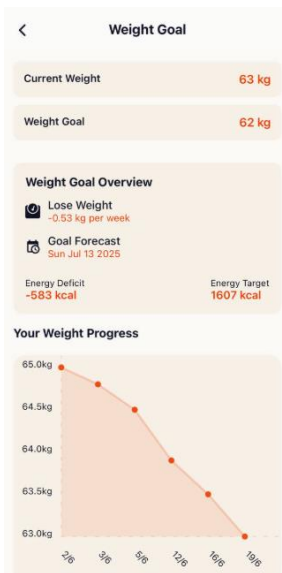
Фигура 50 - въвеждане на целево тегло



Фигура 49 - въвеждане на скорост

Тъй като целта от покачване се променя на сваляне, се появява следващият модален прозорец за избор на скорост. Задаваме, че потребителят иска да отслабне с 0.5 кг на седмица. Според тези стойности се изчислява дефицита и калориите които трябва да приема потребителят.

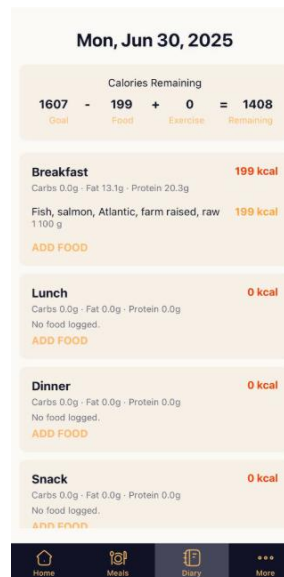
Стойността се актуализира на всички екрани незабавно (Фиг. 51, 52, 53)



Фигура 51 – акт. на данните



Фигура 52 – акт. на Номе



Фигура 53 - акт. на Дневник

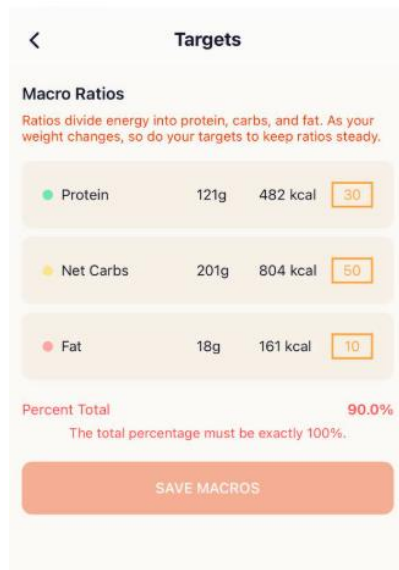
4.2.17. Редактиране на макронутриентно разпределение

Системата предоставя на потребителите редактиране на макронутриентното им разпределение. На екрана „Targets“ потребителят вижда три полета – протеин (Protein), въглехидрати (Net Carbs) и мазнини (Fat), всеки с процентно поле, автоматично изчислени грамажи, калории според въведения процент и общ процент на всички макронутриенти.

При зареждане на екрана в `useEffect()` се извиква функцията `fetchUserMacrosAndEnergy(token)`, която прави заявка към текущото разпределение на потребителя и втора заявка за да вземе текущия калориен таргет. Макросите се преобразуват във формат, подходящ за интерфейса, чрез `formatMacros`.

При промяна на процент в някое поле (`handlePercentChange`), стойността се актуализира локално. Всеки макронутриент изчислява: калории = `energyTarget` x % и грамажи, като протеините и въглехидратите се изчисляват с `kcal/4`, а мазнините – `kcal/9`

Когато потребителят натисне “Save Macros”, се извършва валидация – сборът на процентите трябва да е точно 100%, извиква се updateUserMacros(token, macros) и данните се подготвят по този начин



Macro	Amount	Kcal	Ratio (%)
Protein	121g	482 kcal	30
Net Carbs	201g	804 kcal	50
Fat	18g	161 kcal	10

Percent Total: 90.0%
The total percentage must be exactly 100%.

SAVE MACROS

```
{ protein_ratio: 0.3, fat_ratio: 0.3, carb_ratio: 0.4 }
```

В сървърната част GET заявката /api/user-macros търси макросите на потребителя в таблица user_macros. Ако няма такива – връща грешка.

/api/user-macros PATCH валидира дали сборът на трите стойности е 1.0 или 100%. Ако има вече запис за този потребител – UPDATE, ако няма – INSERT. Записва се и created_at, за да се пази история по време. Новите стойности се актуализират отново във всички екрани където се визуализира макронутриентното разпределение.

Фигура 54 - екран за макронутриентното разпределение

4.2.18. Редактиране на диетичните предпочитания

Потребителите могат да редактират своите диетични предпочитания. На екрана Diet Preferences потребителят вижда три разгъваеми секции:

- Diet Type – избира се само една опция (Веган, Кето, и т.н.)
- Favorite Cuisine – позволяват се множество избори
- Allergies – списък с потенциални алергени, които потребителят може да отбележи

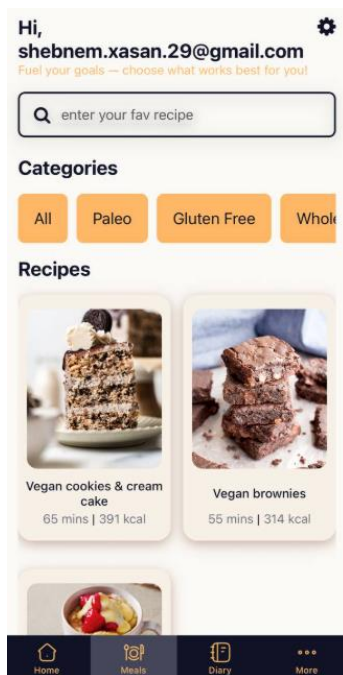
При зареждане на екрана се извиква fetchAllPreferenceData(token), която прави 4 паралелни заявки – към налични видове диети, към кухни по страни или региони, към алергени и вече избрани предпочитания от потребителя. Получените данни се показват в интерфейса чрез разгъваеми компоненти. При всеки избор стойностите се актуализират в състоянието (selectedDiet, selectedCuisines, selectedAllergens). При натискане на Save Preferences, се извиква saveUserPreferences(token, {...}), която изпраща PATCH заявка към api/user/preferences. В тялото на заявката се включват:

```
{ "diet_category_id": 2, "allergen_ids": [1, 5], "cuisine_ids": [3, 4, 6] }
```

Сървърът валидира и записва новите стойности в таблици, свързани със categories, user_allergies и user_favorite_cuisines. Чрез тези избрани опции автоматично се филтрират и рецептите на екрана за рецепти.



Фигура 55 - диетични предпочитания



Фигура 56 - филтрирани рецепти



Фигура 57 - филтрирани рецепти спрямо алергени

Избираме тип диета – Веган и кухня Американска. В страницата с рецепти виждаме съответните филтрирани рецепти. Първите две рецепти имат съставка която може да съдържа соя, в полето за алергени отбелязваме, че потребителят има алергия към соя (Soy), като резултат виждаме, че тези 2 рецепти ги няма вече в препоръките.

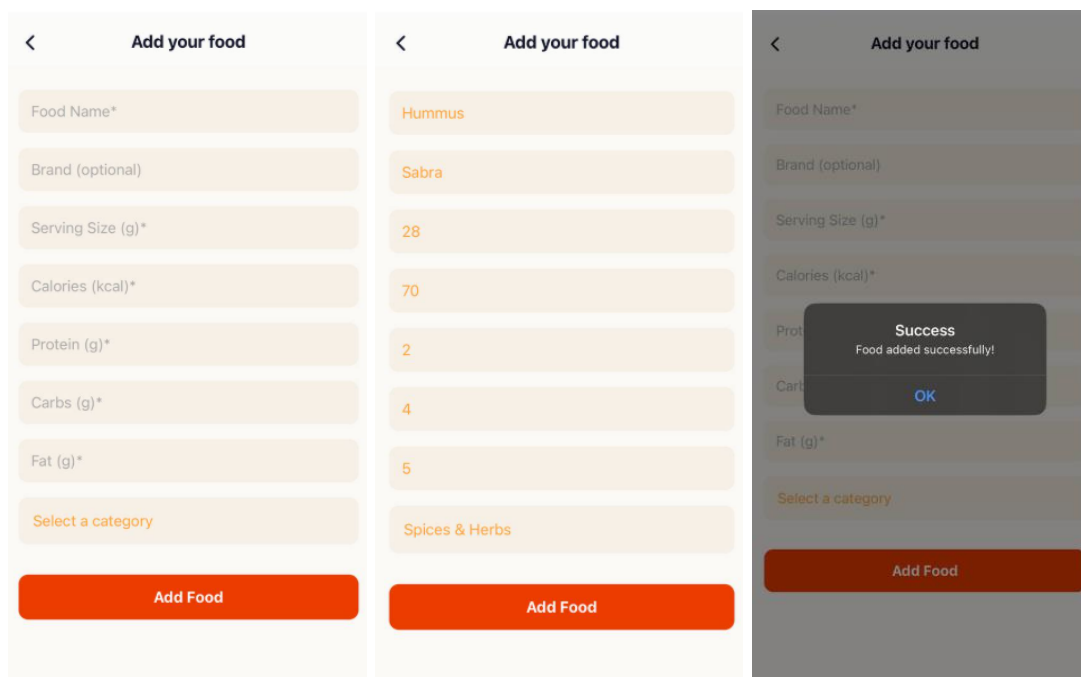
4.2.19. Добавяне на персонализирани хранителни продукти

Добавянето на персонализирани храни в приложението позволява на потребителя да въведе и съхрани храни, които не се намират в предварително зададената база данни. На екрана “Add your food”, потребителят попълва формуляр с основната информация за храната – име на храната (задължително), марка (по избор), грамаж на порция (задължително), калории, протеини, въглехидрати и мазнини (всички са задължителни) и категория, избира се от падащо меню. След попълване се натиска бутонът „Add Food“, който изпраща информацията към сървъра. Кодът използва useState за съхранение на стойности като name, calories, protein, carbs и други. Валидират се всички задължителни полета преди изпращане. При зареждане на екрана се извиква getFoodCategories(token), която зарежда списък с налични категории храни, и го

използва за попълване на Picker меню. Обектът за изпращане изглежда по този начин:

```
const foodData = {  
  name,  
  brand,  
  serving_size_g,  
  nutrients: {  
    Energy: { unit: 'kcal', value: ... },  
    Protein: { unit: 'g', value: ... },  
    Carbohydrate: { ... },  
    Fat: { ... }  
  },  
  source: 'user',  
  category_id,  
};
```

С помощта на `addCustomFood(foodData, token)` се прави заявка към сървър с съдържанието на храната. В сървърната част се приема JSON с хранителни данни. `/customFoods` получава данните и проверява дали задължителните полета са налични и дали `nutrients` е валиден обект. Ако всичко е вярно, в таблицата `foods` се прави запис със следните колони – `name`, `brand`, `serving_size_g`, `nutrients`, `category_id`, `user_id` – ID на текущо автентикирания потребител и `source = 'user'` – за да се разграничи от системните храни. При успешно добавяне сървърът връща обратно JSON с добавения запис, а приложението показва съобщение „Food added successfully“.



Фигура 58 - добавяне на хранителни продукти

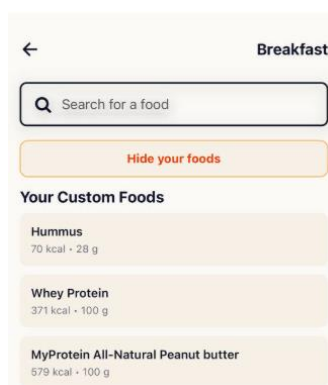
13 SELECT * FROM FOODS WHERE name = 'Hummus' and user_id = 20

Connected (1 query)

Run Explain Analyze 305ms 1 row

#	id	name	brand	serving_size_g	nutrients	source	created_at	category
1	194	Hummus	Sabra	28	{ "Energy": { "unit": "kcal", "value": 70}, "Protein": { "unit": "g", "value": 2}, "Total lipid (fat)": { "unit": "g", "value": 5}, "Carbohydrate":	user	2025-06-30 20:51:57.963355	12

Фигура 59 - запис на добавената храна в базата



Хранителният продукт се визуализира в екрана Diary (Дневник), в поле „Select from your foods“.

Фигура 60 - поле с персонализирани храни на потребителя

4.3. Административен панел. Функционалности на главен админ

4.3.1. Вход в системата за администратори

Администраторският вход в системата е реализиран като уеб-базирана функционалност, достъпна само през браузър.

Потребителят въвежда потребителско име и парола в два текстови полета. След натискане на бутона „Login“, се стартира функцията `handleLogin()`. Чрез `fetch()` се прави заявка към бекенд адреса, в тялото на заявката се изпращат `username` и `password`. Бекендът търси администратор с въведеното потребителско име в базата данни. Ако не е намерен, връща статус 401 със съобщение „Admin not found“. Ако потребителят съществува – се сравнява въведената парола с криптираната парола в базата (чрез `bcrypt.compare`). При неуспешно съвпадение се връща статус 401 и съобщение „Invalid password“. При успешно удостоверяване се генерира JWT токен с валидност 2 часа. Токенът и ролята на администратора се връщат към клиента. Токенът се запазва в `localStorage` под името `adminToken`.

В зависимост от ролята на администратора (поле `role`), потребителят се пренасочва към съответното табло. Тази реализация гарантира, че само упълномощени администратори с валидни данни и роля имат достъп до административната част на системата, при това само чрез уеб интерфейса.

4.3.2. Активни потребители (Active Users – last 5 min)

В това поле от интерфейса се показва броя на потребителите, които са били активни през последните 5 минути. Това е важен индикатор за текущата ангажираност на потребителите в системата.

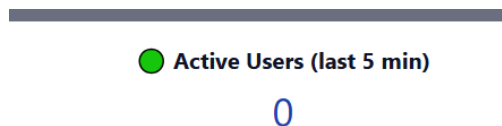
Данните се извличат чрез GET заявка към `/adminDashboard/stats/active-count`. Заявката е защитена с JWT токен, достъпна само за главни администратори (`role: super`).

След първоначално зареждане (`useEffect`), се извършва автоматичен `fetch`. Данните се обновяват автоматично на всеки 2 минути чрез `setInterval`. При зареждане се показва `ActivityIndicator` – въртящ се индикатор. След зареждане се показва числова стойност (брой активни потребители). Броят активни потребители се изчислява по следната логика – извличат се всички потребители от базата, чиято стойност на `last_active_at` е в рамките на последните 5 минути.

```
SELECT COUNT(*) FROM users  
  
WHERE last_active_at > NOW() - INTERVAL '5 minutes';
```

NOW() връща текущата дата и час от системния часовник на базата данни. INTERVAL '5 minutes' е времеви интервал от 5 минути. Резултата е разликата от този интервал от текущото време.

Резултатът се връща като JSON - { "activeCount": 4 }



Фигура 61 - активни потребители в последните 5 мин

4.3.3. Ръст на потребителската база (User Growth)

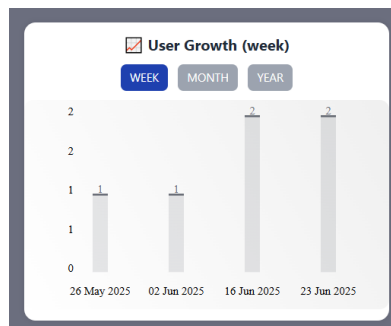
Компонентът показва как се развива потребителската база на системата с времето – тоест колко нови потребители са се регистрирали в платформата по периоди. На екрана се визуализира бар диаграма, която показва общия брой потребители, регистрирани в дадена седмица, месец или година.

При първоначално зареждане и при всяка смяна на времевия период се извиква функцията fetchGrowth(). Заявката се изпраща към бекенд апито. Графиката позволява динамична смяна на времевата грануларност чрез бутони. На графиката се показват брой нови потребители, регистрирани в последните седмици (за краткосрочен поглед), месеци (за средносрочен анализ) и години (за дългосрочни тенденции).

В сървърната част се използва SQL заявка с DATE_TRUNC, за групиране по време. Резултатът връща масив с обекти от типа:

```
[ { "period": "2025-06-02", "user_count": 3 },  
  { "period": "2025-06-09", "user_count": 4 }]
```

Само главния администратор има достъп до тези данни.



Фигура 62 - диаграма за ръст на потребителската база

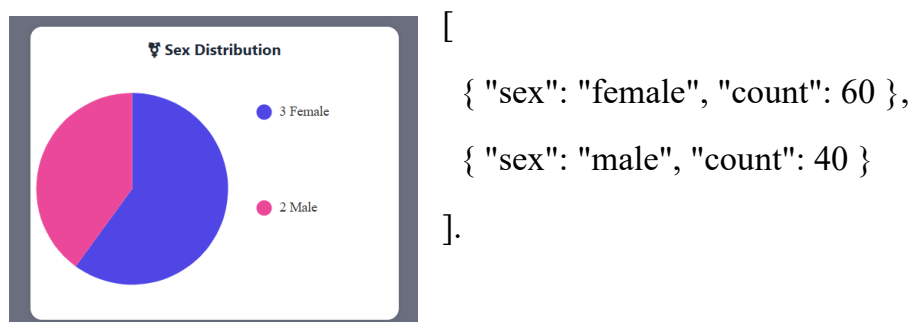
4.3.4. Ръст на потребителската база (User Growth)

Администраторът може да вижда графика за демографската структура на потребителите в платформата според техния пол. Това е важна част от анализа на потребителската база и подпомага вземането на решения, свързани със съдържание, комуникация и здравни насоки.

Използва се кръгова диаграма (PieChart), която показва процентно съотношение между регистрираните полове. Всяка част от графиката е оцветена различно и има легенда за по-лесна ориентация.

При първоначално зареждане се извиква `fetchData()` и се изпраща GET заявка към сървъра. Данните се преобразуват във формат, подходящ за PieChart и се визуализират. При липса на данни се показва съобщение „No data available“.

В бекенда се използва SQL заявка, която групира потребителите според стойността на полето „sex“. Резултатът се връща като JSON масив –



Фигура 63 - диаграма за разпределение на потребителите по пол

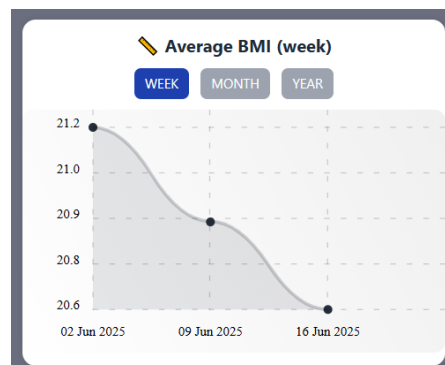
4.3.5. Среден индекс на телесна маса на потребителите (Average BMI)

Администраторът може да вижда как се променя средният индекс на телесна маса (BMI) на потребителите във времето. Това е ключов здравен индикатор, който дава представа за цялостното физическо състояние на потребителската база.

Графиката е от тип Крива линия (LineChart). Визуализира промяната на средния BMI по избран времеви период. Потребителят може динамично да сменя групирането чрез бутони. В компонента където е графиката, се извиква API с параметър `groupBy`

`/adminDashboard/stats/average-bmi?groupBy=month`

Извлича се масив с периоди и средни стойности на BMI. В сървърната част достъпът е ограничен само до главни администратори.



Фигура 64 - среден BMI на потребителите

4.3.6. Управление на администратори (View & Manage Admins)

Главният администратор има достъп до функционалност, която му позволява да преглежда, редактира и изтрива всички останали администратори в системата. Това включва информация за потребителско име, роля и дата на създаване на всеки акаунт.

Потребителският интерфейс представя списък от всички администратори в изглед като отделни карти (ViewListContainer />. Всеки администратор се визуализира в InfoCard, съдържащ username, role и created (дата на създаване). Възможни действия по картите са Edit (Редакция), Delete (Изтриване) и Cancel/Save (Отказ/Запис). Редакцията позволява на админа да променя потребителското име или роля чрез LabeledInput компонента и RolePicker (компонент който визуализира наличните роли и админът трябва да избере от тях). Изтриването на администратор става чрез бутон.

И за двете опции – редактиране или изтриване достъпът е ограничен само до главни администратори. Ако заявката е изпратена от потребител с друга роля, се връща

```
{ "message": "Only super admins can delete/update admins" }
```

За изтриването на администратор се извлича id от параметрите на URL (req.params.id). Изпълнява се SQL заявката

```
DELETE FROM admins WHERE id = $1 RETURNING *
```

Ако rowCount === 0, това означава, че не е намерен такъв администратор – връща се 404 Not Found.

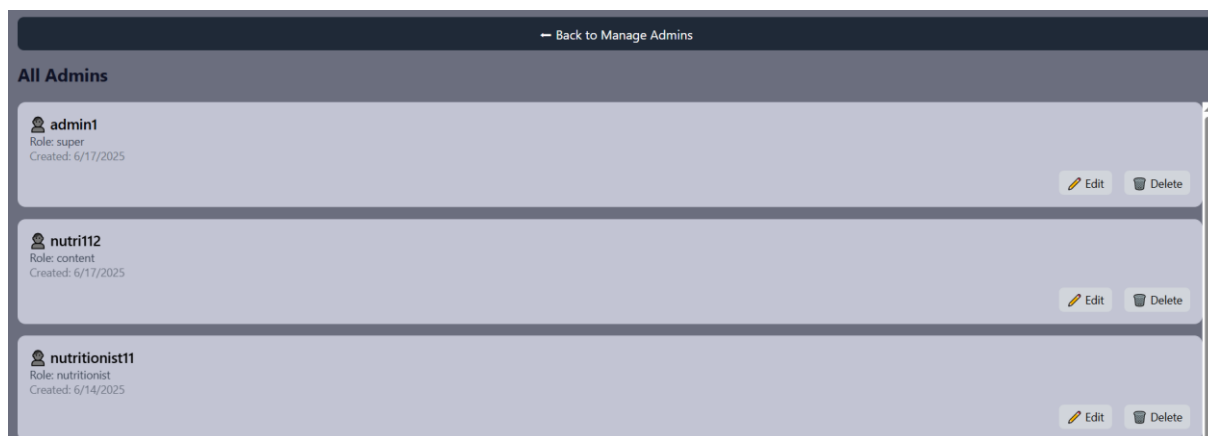
```
При успех сървърът връща отговор { "message": "Admin deleted successfully" }
```

Редактирането на администратор работи по подобен начин, извлича се id от URL и данни от req.body - const { username, role } = req.body;

Проверява се дали и username и role са подадени. Ако не - { "message": "Username and role are required" }

Изпълнява се SQL заявка

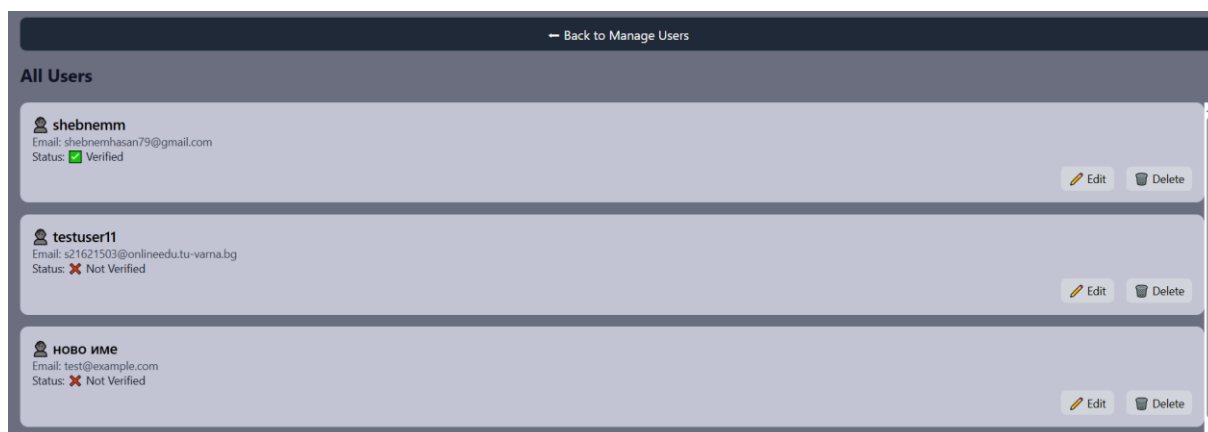
```
UPDATE admins SET username = $1, role = $2 WHERE id = $3 RETURNING id, при успех бекендът връща отговор { "message": "Admin updated successfully" }.
```

Фигура 65 - интерфейс за преглед и управление на администратори

4.3.7. Управление на потребители (View & Manage Users)

Главният администратор има достъп до функционалност, която му позволява да преглежда, редактира и изтрива потребителите в системата. Кодът се реализира по същия начин както в точка 4.3.6. Използвани са общи компоненти и същите функции и заявки, само че за таблица users.



Фигура 66 - интерфейс за преглед и управление на потребители

4.3.8. Създаване на администратор (Create Admins)

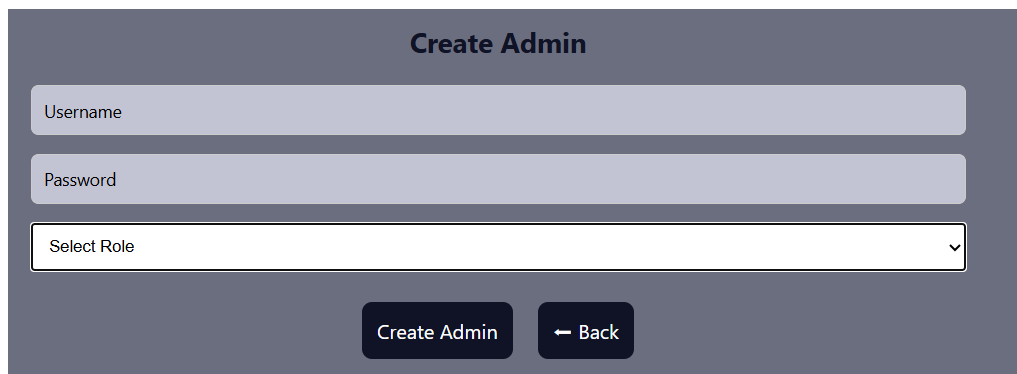
Главният админ с роля super има възможност да създава нови администратори с роля “content” – мениджър съдържание. На екрана админът вижда форма за създаване на админ <CreateForm />. Формата съдържа полетата username, password и role. Всички полета са задължителни.

След попълване на формата и натискане на бутона Create Admin се изпраща POST заявка към сървъра. Прави се проверка на правата – дали заявката е от главния админ. Валидират се входните данни – дали всички задължителни полета са предоставени. Проверява се дали този админ вече не съществува в базата

```
SELECT * FROM admins WHERE username = $1
```

Ако съществува се визуализира грешка - "Username already exists"

Ако всички проверки минат успешно паролата се хешира с `bcrypt.hash(...)` и се записва в базата `INSERT INTO admins (username, password_hash, role) VALUES ($1, $2, $3)`. Връща се отговор `{ "message": "Admin created successfully" }`.



Фигура 67 - форма за създаване на администратор

4.3.9. Търсене и филтриране на потребители (User Search & Filtering)

Главният администратор има възможност да търси потребители по име или имейл, както и да прилага детайлни филтри по различни критерии като пол, ръст, тегло, ниво на активност и физиологично състояние.

За да търси админът желания потребител, въвежда име или имейл в текстово поле. Натискането на бутона `Search` извиква функцията `handleSearch()`. Резултатите се показват в списък чрез компонент `<UserCard />..`

За търсене на потребител се използва линеен метод на търсене (Linear Search). Този метод преглежда всеки запис от базата и ги сравнява по зададени критерии. Търсенето е *case-insensitive* (не различава малки и големи букви), *partial match* (намира съвпадения дори ако терминът е част от стойността).

```
const result = await db.query('SELECT id, name, email, verified FROM users');
const users = result.rows;

let matches = [];

if (name) matches = [...matches, ...linearSearch(users, name, 'name')];
if (email) matches = [...matches, ...linearSearch(users, email, 'email')];
```

Тази част от кода извлича потребителите от базата. Извличат се само определени полета. `Result.rows` съдържа масив с потребители

```
[{ id: 1, name: 'Anna Petrova', email: 'anna@gmail.com', verified: false },
```

```
{ id: 2, name: 'Shebnem Mehmed', email: 'shebnem.xasan.29@gmail.com',  
verified: true }]
```

Създава се празен масив `let matches = []`. В този масив ще се събират потребителите, които отговарят на търсенето.

Функцията `linearSearch(...)` почва да търси частични съвпадения в полетата `name` или `email`. Ако потребителят търси „ann“, `linearSearch(users, “ann”, “name”)` ще върне всички потребители, чието име съдържа „ann“ без значение дали е „Anna“ или “Ioanna”.

`const uniqueMatches = Array.from(new Map(matches.map(u => [u.id, u])).values());` - този ред премахва дублиращи се резултати, ако един и същ потребител съвпада и по име и по имейл, той ще се появи само веднъж в резултатите.

Филтрирането работи по подобен начин на търсенето. Всички потребители се зареждат от базата данни. След това в сървъра се проверяват подадените филтри. За всеки филтър се премахва потребител, който не отговаря на зададените условия. Накрая се връща списък с тези, които отговарят на всички избрани критерии.

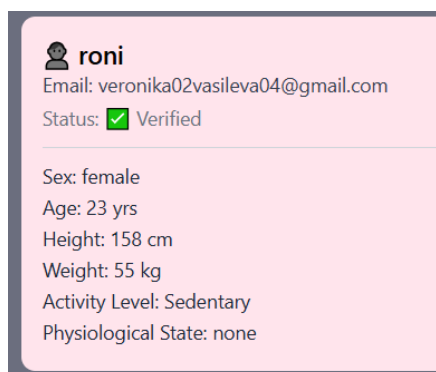
The screenshot shows a web interface with a green header bar containing the text "Found 2 user(s)". Below the header is a section titled "Search Users". It features a search input field with the text "sheb" and a "Search" button. Below the search bar, there are two user cards. Each card displays a user profile icon, the name "shebnemm", the email "Email: shebnemxasan.29@gmail.com", and a status "Status: Verified" with a green checkmark. The cards are styled with a light blue background and a dark blue border.

Фигура 68 - резултат от търсене на потребител

The screenshot shows a web interface with a "Filter By:" section. It contains several filter options: "Verified:" with a dropdown menu showing "Verified", "Activity Level:" with a dropdown menu showing "Sedentary", "Sex:" with a dropdown menu showing "Female", and "Physiological State:" with a dropdown menu showing "None". Below these are input fields for "Min Weight (kg):" (50), "Max Weight (kg):" (80), "Min Height (cm):" (100), and "Max Height (cm):" (170). A "Filter" button is located below the input fields. Below the filter section, there are two user cards. Each card displays a user profile icon, the name "hati", the email "Email: hatimehmed762@gmail.com", and a status "Status: Verified" with a green checkmark. The cards are styled with a light pink background and a dark blue border.

Фигура 69 – филтриране и резултат

<UserCard /> може да се разпъне и админът ще види подробности за потребителя.



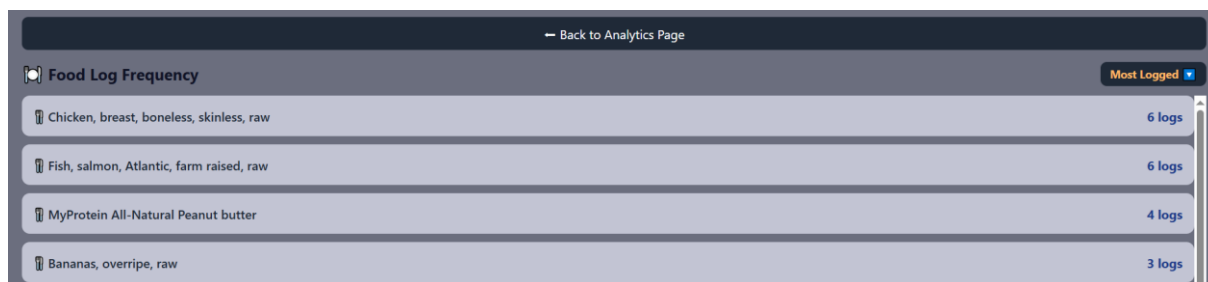
Фигура 70 - детайли за намерения потребител

4.3.10. Честота на въвеждане на храни (Food Log Frequency)

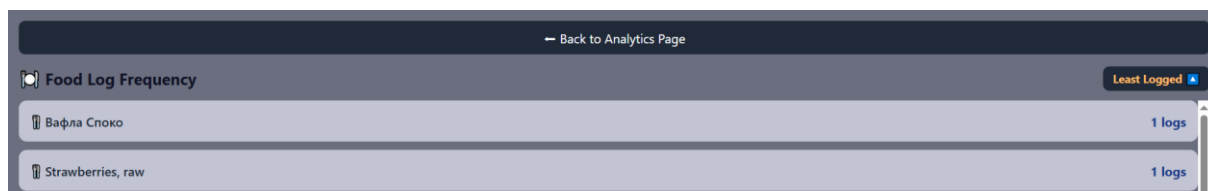
Тази функционалност показва кои храни се регистрират най-често от потребителите в приложението. Главният админ може да види списък с храни и броя пъти, в които всяка е била записана в дневниците.

Функцията `fetchFoodLogFrequency()` извлича списък с храни и полето `log_count` - броя регистрирания на всяка храна. Данните се сортират с функцията `sortFoodLogs()`. В тази функционалност се използва алгоритъм за сортиране чрез вмъкване (`insertion sort`). Това е прост алгоритъм за сортиране, който изгражда сортирания списък елемент по елемент. Има сложност $O(n^2)$ в най-лошия случай, защото сравнява всеки елемент с всички преди него.

Потребителят може да превключва между – `descending` (най-често логвани храни) и `ascending` (най-рядко логвани храни). Списъкът се показва с `FlatList` – всяка карта съдържа име на храната и брой логвания.



Фигура 71 - сортиране на записани храни в низходящ ред



Фигура 72 - филтриране на записани храни във възходящ ред

4.3.11. Използваемост на функционалности (Feature Usage Breakdown)

Тази функционалност показва кои функционалности от приложението се използват най - много от потребителите. Позволява на администратора да натисне върху всяка функционалност и да види кои конкретни потребители са я използвали, както и кога за последно.

Диаграмата е custom компонент <CustomBarChart />. Цветовете на баровете са зададени ръчно с масив barColors.

Сървърът взема данни от таблицата feature_usage_logs, където се съхранява историята на използване на функционалностите. Общият брой потребители се извлича от таблица users. Резултатът е под формата на:

[{ "feature": "LogMeal", "percentage": "78.3" }, { "feature": "BMIChart", "percentage": "51.2" }]. Детайлите за конкретна функционалност се визулизират спрямо следната заявка:

```
SELECT
    u.name,
    u.email,
    MAX(ful.timestamp) AS last_used
FROM feature_usage_logs ful
JOIN users u ON ful.user_id = u.id
WHERE ful.feature = $1
GROUP BY u.id
ORDER BY last_used DESC;
```

Показва се последната дата на използване от всеки потребител.

Данните които се използват за визуализиране на честота на използване на функционалности от потребителите, не се въвеждат ръчно, а се събират автоматично чрез проследяване в кода. След извършване на конкретни действия в приложението, като записване на храна, редакция на профил, обновяване на тегло и отваряне на рецепта се извиква специална функция await trackFeature('log_meal', token);. Тази функция регистрира използването на конкретна функционалност и изпраща информация към сървъра.

```
if (success) {
    alert('Food added to log!');
```

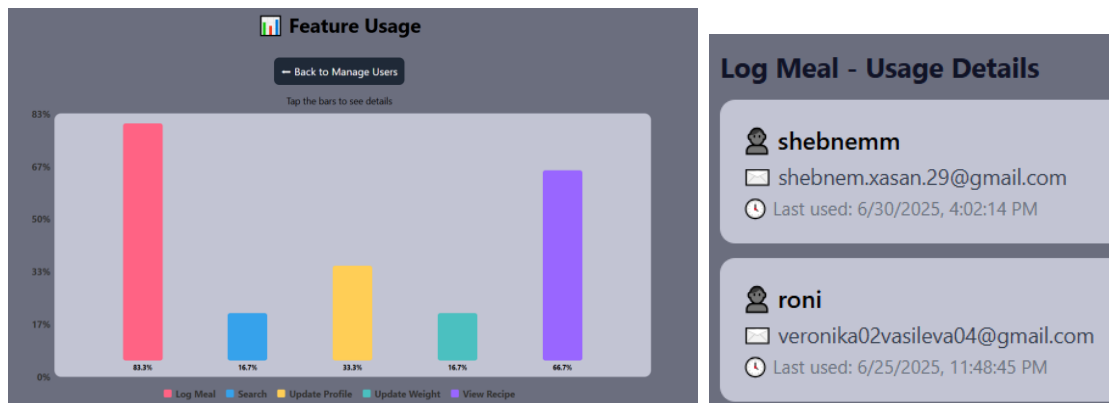
```

    await trackFeature('log_meal', token); // в базата се записва "log_meal"

    router.back();
  }

```

Функцията е асинхронна, защото изпраща HTTP заявка към сървъра, а това е операция, която отнема време и не се изпълнява моментално.



Фигура 73 - диаграма за използвани функционалности

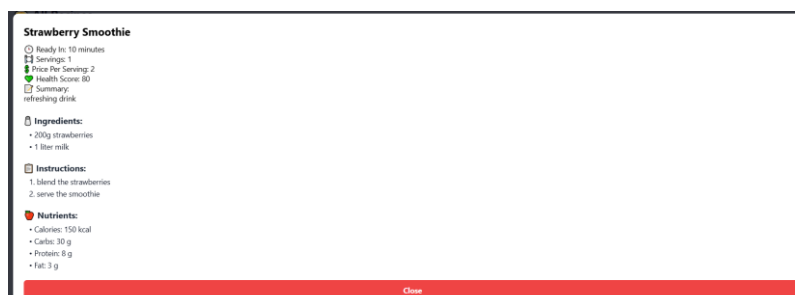
4.4. Административен панел. Функционалности на админ-съдържание

4.4.1. Преглед на рецепти (View Recipes)

Функционалността за преглед на рецепти е аналогична на тази за преглед на администратори в точка 4.3.6. Всички рецепти се зареждат от сървъра и се визуализират в списък, където за всяка от тях могат да се извършват действията редакция, изтриване и детайлен преглед – чрез бутона „See Details“, който отваря модален прозорец с допълнителна информация за рецептата.



Фигура 74 - екран за преглед на всички рецепти



Фигура 75 - модален прозорец за детайли за дадена рецепта

4.4.2. Преглед на хранителни продукти (View Foods)

Функционалността „Преглед на храни“ е аналогична на тази за рецепти, тъй като използва същия компонент `ViewListContainer` за визуализиране, редакция и изтриване на записи. Разликата тук е, че в тази секция има добавено поле за търсене, което позволява на админа по-бързо и лесно да намира конкретна храна по име. Това е важно, защото в базата данни се съдържат много хранителни продукти и ръчното преглеждане би било неефективно. Също така админът може да сортира резултатите по име, което допълнително улеснява управлението на голямото количество данни.



Фигура 76 - преглед на хранителни

продукти

4.4.3. Преглед на категории (View Categories)

Функционалността „Преглед на категории“ е изградена по същия начин, както и другите списъчни изгледи. На екрана се визуализира списък с всички съществуващи категории. За всяка категория са налични две основни действия – редакция и изтриване.



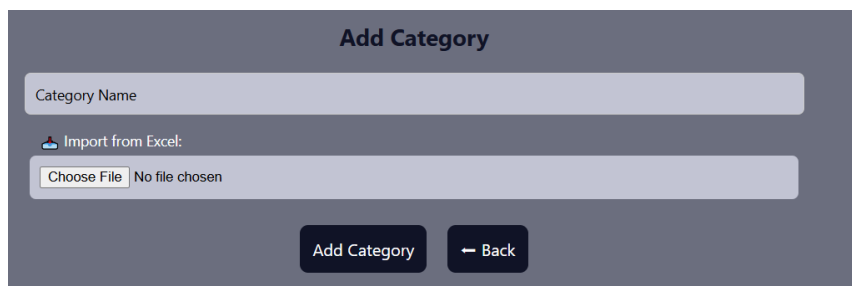
Фигура 77 - преглед на категории

4.4.4. Добавяне на категория (Add New Category)

Админът може да добавя нови категории в базата. Компонентът `AddCategory` използва готовия компонент `CreateForm`, който визуализира поле за попълване на име на категория (задължително поле). При натискане на бутона за изпращане, се извиква функцията `createCategory()`, която изпраща POST заявка към сървъра с въведеното име. Ако заявката е успешна, категорията се добавя и потребителят може да бъде върнат обратно към списъка.

Освен ръчно, потребителят може да импортира множество категории наведнъж от Excel. При избиране на файл от системата, функцията

handleImportExcel извлича първия избран файл (e.target.files[0]). Подготвя го за изпращане чрез importCategoriesFromExcel(file). Сървърната страна прочита Excel файла, обработва редовете и записва категориите в базата. След успех, се показва съобщение “Imported X categories”.



Фигура 78 - добавяне на

категории

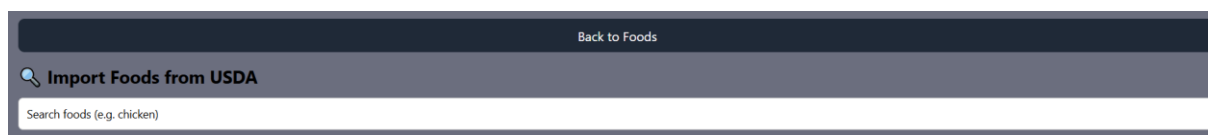
4.4.5. Добавяне на хранителни продукти от външен източник

Импортирането на данни от външно API в приложението позволява на админите бързо и удобно да добавят храни с подробна хранителна информация.

В приложението външният източник е USDA (United States Department of Agriculture). USDA предоставя публично API, чрез което може да се търси и получава информация за хиляди храни и техните хранителни стойности. Това е надежден и официален източник на хранителни данни, който включва описание на продукта, бранд, макронутриенти и стандартна порция.

В екрана за добавяне на храна потребителят въвежда ключова дума. При натискане на Enter се извиква функцията searchUsdaFoods(query), която прави заявка към USDA API. Получените резултати се показват в списък – всяка храна е в отделна карта с данни.

До всяка храна има чекбокс, с който потребителят избира кои храни иска да импортира. Избраните храни се записват в масив selectedIds. При натискане на бутона Import X Selected, се извиква importUsdaFoods(selectedIds), която изпраща избраните ID-та към сървърната част. Сървърът прави допълнителна заявка към USDA API, за да изтегли пълни данни за всяка храна. Храните се записват в базата на приложението. След успешен импорт се показва съобщение “Successfully imported X foods”.



Фигура 79 - добавяне на хранителни продукти от външен източник

4.4.6. Добавяне на рецепти

Админът може да добавя рецепти към системата. Добавянето на рецепта в приложението представлява богата и интерактивна форма, която позволява на админа да въведе подробно описание на ястието. Админът попълва заглавие, изображение, резюме, време за приготвяне, брой порции, цена, здравословна оценка, категории, видове ястие, кухня, списък със съставки, инструкции, хранителни стойности. След попълване се натиска бутон за добавяне. Този бутон извиква функцията `handleSubmit()`.

Преди да се изпрати заявката към сървъра, се правят няколко задължителни проверки, за да се гарантира, че формата е коректно попълнена. Всички текстови полета са задължителни, числовите полета се валидират дали са числа. Изисква се поне една категория, съставка и една стъпка от инструкции. При липса на някое от горните, се показва грешка в `AlertBox`.

При избор на съставки админът може да избере от наличните в базата (зарездат се от `fetchIngredients()`). Ако желае да добави нова съставка, админът натиска бутон, който активира поле за въвеждане на име на съставката, количество и мерна единица и дали съдържа алергени (избират се от списък). Системата съхранява връзка между новата съставка и свързаните алергени, като се използва обект `ingredientAllergenMap`.

След успешна проверка всички данни се форматират в обект `payload`. Съставките се трансформират с `ingredient_id`, `amount`, `unit`. Инструкциите се номерират (`step_number`) и форматират. Изпраща се POST заявка към сървъра. Ако заявката е успешна се получава съобщение за успешно добавяне на рецептата. Ако възникне грешка – показва се съобщение в червено с описанието ѝ.

Back to Manage Recipes

Add New Recipe

Title

Image URL

Summary

Ready in Minutes

Servings

Price per Serving

Health Score

Category

Select category

Dish Type

Select dish type

Cuisines

Select cuisines

Фигура 80 - добавяне на рецепта - част 1

Ingredients

+ Add New Ingredient

Select Ingredient

Amount Unit

Original (e.g. '200g chicken')

Add Ingredient

Instructions

Add Step

Nutrients

Select Nutrient Add

Submit Recipe

Фигура 81 - добавяне на рецепта - част 2

Глава 5: Тестове и резултати

5.1. Тестване на софтуерна система

Софтуерното тестване е процесът на изследване на софтуер с цел получаване на оценка на качеството, функционалността и пригодността на софтуерния продукт или система. Тестовите се използват за откриване на грешки, дефекти и недостатъци в софтуерната система и за уверяване, че тя отговаря на спецификациите и потребителските изисквания.

Методите за тестване на софтуер традиционно се разделят на две групи – метод на отворената кутия (white-box) и метод на затворената кутия (black-box). Те се различават по фокуса и подхода към процеса на тестване – дали разработчикът се интересува повече как работи софтуерът, или дали работи според изискванията. При метода на отворената кутия се получава пълен достъп до вътрешността на софтуера – изследва се всеки отделен компонент и кода на програмата. При метода на затворената кутия тестовите се изпълняват на базата на външното поведение на софтуера, с цел проверка на функционалността му.

5.2. Използвани методи при тестване

5.2.1. Тестване с Mocha и Chai

В процеса на разработка на системата е използван подход за автоматизирано тестване чрез инструментите Mocha и Chai, които са широко разпространени в JavaScript екосистемата. Целта на този подход е гарантиране на надеждността и коректността на основната логика, чрез създаване и изпълнение на модулни тестове (unit tests).

Mocha представлява тестов фреймуърк, който предоставя структура за писане и организиране на тестове, докато Chai е библиотека за логически проверки (assertions), която позволява лесно валидиране на резултатите. Двата инструмента работят съвместно и са особено подходящи за white-box тестване, тъй като позволяват директно тестване на отделни функции и логически компоненти от кода.

Автоматизираните тестове, реализирани с Mocha и chai, обхващат основни функционалности на системата като:

- Проверка на регистрация и вход с валидни и невалидни данни
- Добавяне на консумирана храна
- Обобщена информация за нутриентите приети от потребителя за определен ден
- Търсене на храна по id
- Търсене на храна по име

- Списък с всички храни
- Списък с категории на храни
- Търсене на потребители, филтриране на потребители по критерии
- Изтриване и възстановяване на потребители

Тези тестове допринасят за:

- Бърза идентификация на грешки още в ранните етапи;
- Улеснено регресионно тестване при бъдещи промени;

* По-високо качество на кода, чрез системна проверка на логиката.

5.2.1.1. Резултати от тестването

Изпълнението на автоматизирани тестове е осъществено чрез команда `npm test`, при което са стартирани всички файлове в папката `tests`.

```
POST /api/auth/register
✓ връща 400 при липсващи задължителни полета (120ms)
✓ връща 400 ако потребителят вече е регистриран и верифициран (3139ms)
✓ връща 200 ако потребителят съществува, но не е верифициран (1218ms)
✓ връща 201 при нова успешна регистрация (872ms)

4 passing (5s)
```

Фигура 82 - автоматични тестове за регистрация на потребители

```
POST /login
✓ връща 400 при липсващи данни (94ms)
✓ връща 401 при невалиден имейл (2104ms)
✓ връща 401 при грешна парола (2695ms)
✓ връща 403 при неверифициран акаунт (309ms)
✓ връща 200 и токен при успешен login (428ms)

5 passing (6s)
```

Фигура 83 - автоматични тестове за вход в системата

```
POST /api/food-logs/
✓ връща 201 при валидни данни (1588ms)
✓ връща 500 ако липсва unit (1411ms)

2 passing (6s)
```

Фигура 84 - Автоматични тестове за добавяне на консумирана храна

```
GET /api/food-logs/nutrients-summary
✓ връща 200 и нутриентен обобщен отчет за валидна дата (349ms)
✓ връща 401 при липсващ токен

2 passing (4s)
```

Фигура 85 - обобщена информация за нутриентите (макронутриенти и микронутриенти), приети от потребителя за избрана дата

```
/api/foods
GET /api/foods/:id
✓ връща 200 и обект за валидно ID (208ms)
✓ връща 404 за невалидно ID (147ms)
GET /api/foods?search=chicken
✓ връща 200 и списък от храни (148ms)
✓ връща 400 при прекалено кратък search низ
GET /api/foods/food-categories
✓ връща 200 и списък с категории (153ms)

5 passing (3s)
```

Фигура 86 - резултати от тестове за функционалностите, свързани с храни

```
/api/users – обикновен потребител
GET /me
✓ връща профила на текущия потребител с възраст (158ms)
PATCH /update-profile
✓ успешно обновява профил с валидни данни (149ms)
✓ връща 400 при невалидни стойности
GET /users-with-goals
✓ връща потребители с последна цел (151ms)
GET /:id/goals-history
✓ връща история на целите за потребител (142ms)
GET /:id/stats
✓ връща статистики за потребител (148ms)

6 passing (3s)
```

Фигура 87 - Успешно преминали клиентски тестове: профил, обновяване, цели и статистики

```
/api/users – супер админ
PUT /:id
  ✓ обновява потребител по ID (153ms)
DELETE + undo
  ✓ изтрива и възстановява потребител (354ms)
GET /search
  ✓ търси потребител по име (155ms)
GET /filter
  ✓ филтрира потребители по ръст (145ms)

/api/users – обикновен потребител
GET /me
  ✓ връща профила на текущия потребител с възраст (139ms)
PATCH /update-profile
  ✓ успешно обновява профил с валидни данни (151ms)
  ✓ връща 400 при невалидни стойности
GET /users-with-goals
  ✓ връща потребители с последна цел (151ms)
GET /:id/goals-history
  ✓ връща история на целите за потребител (145ms)
GET /:id/stats
  ✓ връща статистики за потребител (139ms)

10 passing (4s)
```

Фигура 88 - Успешни тестове: Админски действия с потребители – търсене, филтриране, редакция и изтриване

5.2.2. Тестване чрез черна кутия

Тества се поведението на API-то без да се знае вътрешната реализация. Тестващият се интересува само от: Какво подава (вход), Какво получава (изход), Какво поведение се очаква.

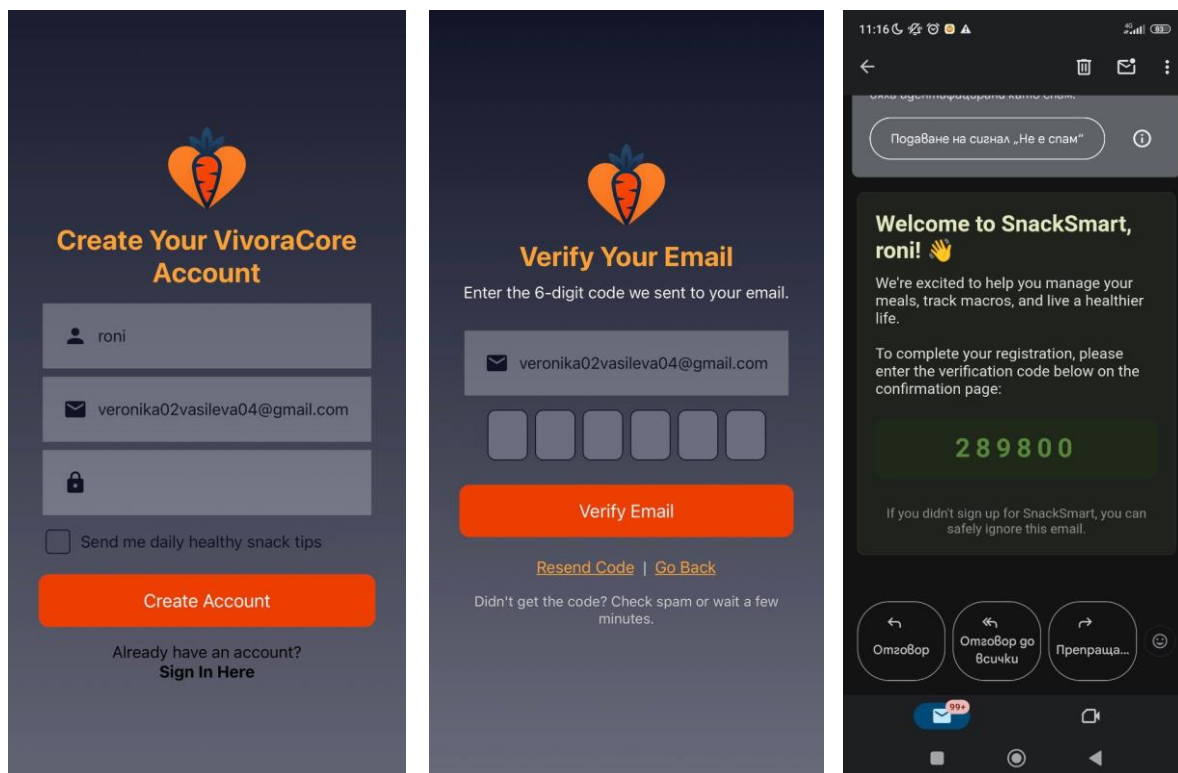
5.2.2.1. Резултати от тестването – Тест 1 – Регистрация в системата

Стъпки:

- Отваряне на екрана за регистрация
- Въвеждане на име, имейл
- Въвеждане на парола
- Натискане на бутона „Create Account“

Очакван резултат: Препратка към екран за въвеждане на код за потвърждение на въведения имейл.

Действителен резултат: Като очаквания.



Фигура 89 - изпълнение и резултат на тест 1

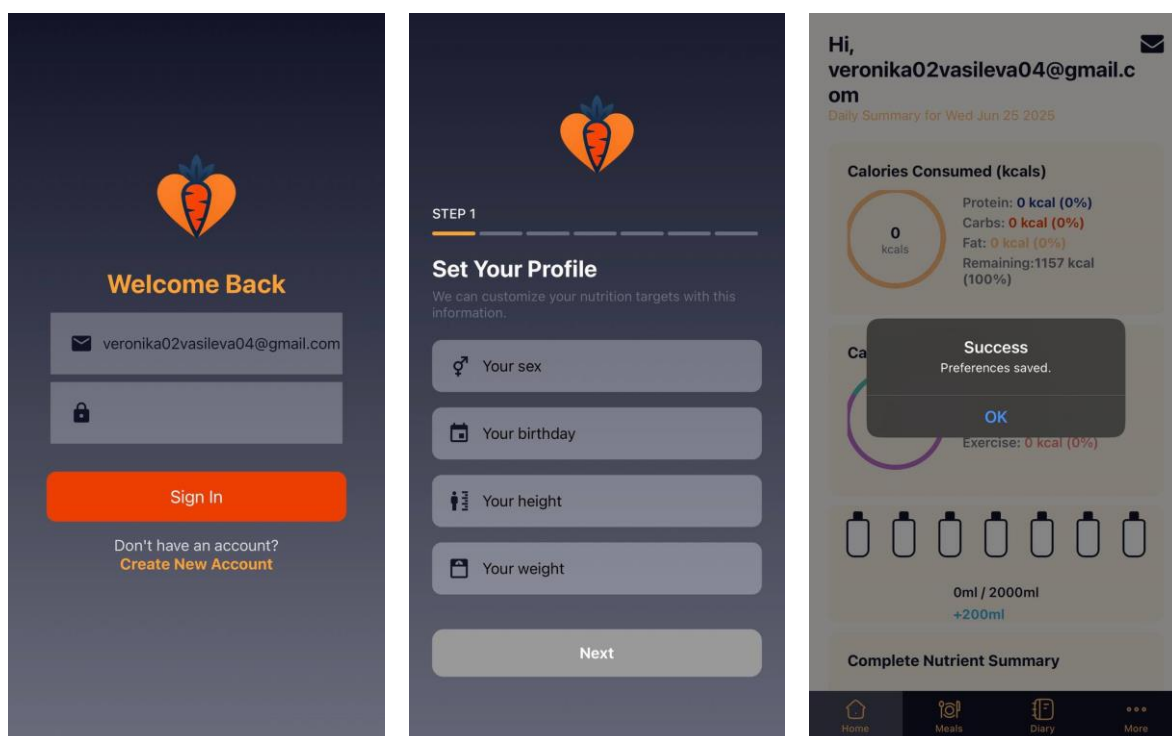
5.2.2.2. Резултати от тестването – Тест 2 – Вход в системата

Стъпки:

- Отваряне на екрана за вход в системата
- Въвеждане на имейл
- Въвеждане на парола
- Натискане на бутона „Sign in“

Очакван резултат: При първото влизане на потребителя в системата след регистрация, трябва да се препрати към екран за настройване на профила. След преминаване на всички стъпки за настройване на профила, има препратка към началната страница на системата (Home Page).

Действителен резултат: Като очаквания.



Фигура 90 - Изпълнение и резултат на тест 2

5.2.2.3. Резултати от тестването – Тест 3 – Добавяне на храна към хранене

Цел: Да се провери дали потребителят може успешно да добави храна и тя да се визуализира със съответните нутриенти и калории.

Стъпки:

- Влизаме в системата и навигираме към Diary от менюто;
- Избираме хранене, например закуска (Breakfast), и натискаме Add Food;
- Въвеждаме име на храна в полето за търсене (пример: chicken) или избираме от „Select from your foods“;
- Избираме храна от резултатите
- Задаваме брой порции и хранене (например 100g)
- Натискаме бутона за потвърждение

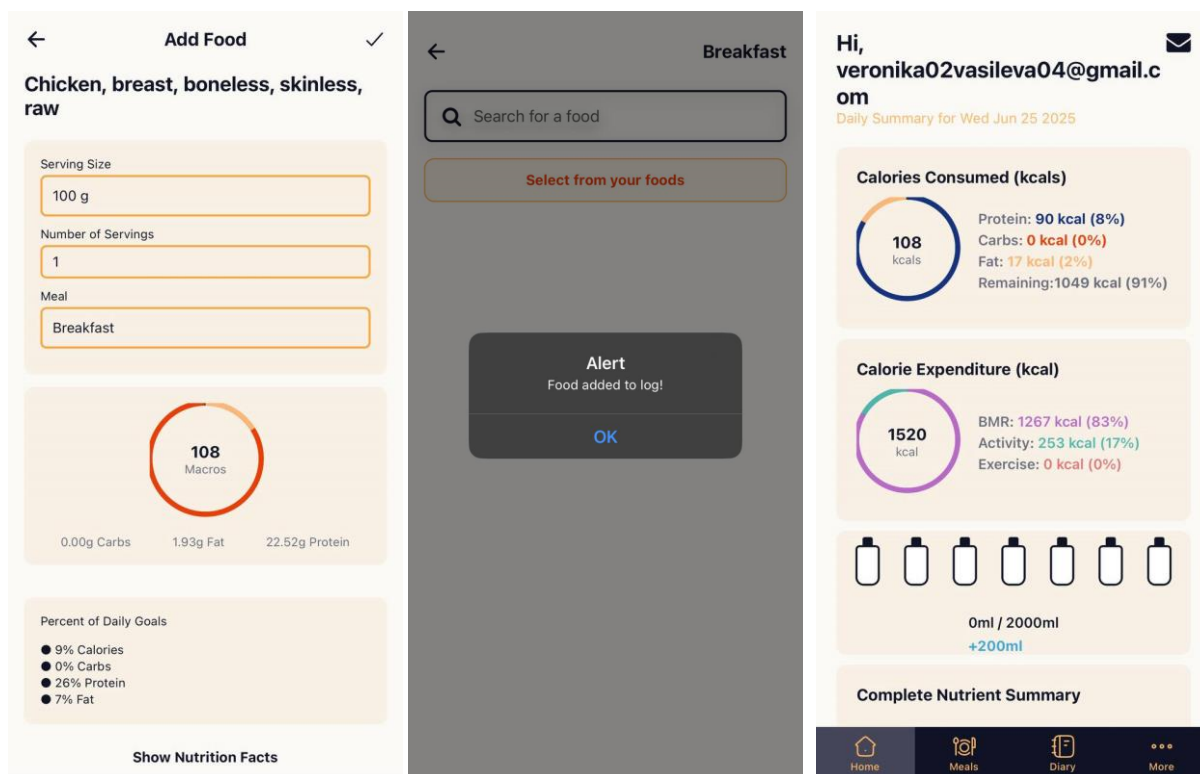
Очакван резултат:

- Храната се появява в съответния раздел (Breakfast) със съответните калории и макронутриенти;
- Калориите и макронутриентите са добавени в обобщената статистика в главната страница (Home page)

Действителен резултат:

- Храната се появява в списъка със закуската.

- Изчислени са калориите и макронутриентите
- Дневната статистика се актуализира правилно (Calories Consumed и Remaining)
- Не се появява грешка или предупреждение.



Фигура 91 - Изпълнение и резултат от Тест 3

5.2.2.3. Резултати от тестването – Тест 4 – Търсене и филтриране на потребители от админски панел

Стъпки:

- Отваряме администраторския интерфейс за потребители;
- Въвеждаме име или имейл в полето „Name or Email“ и натискаме бутона Search;

Очакван резултат:

- Очакваме: да се визуализират всички съвпадащи потребители или съобщение, че няма намерени;
- Пробваме различни филтри като пол, ниво на активност, ръст и т.н.
- Натискаме Filter и наблюдаваме резултатите.

Действителен резултат: като очаквания.

Filter By:

Verified:	Activity Level:
Unverified	Any
Sex:	Physiological State:
Any	Any
Min Weight (kg):	Min Height (cm):
50	150
Max Weight (kg):	Max Height (cm):
90	190

Filter

 **testuser11**
Email: s21621503@onlineedu.tu-varna.bg
Status:  Not Verified

Фигура 92 - изпълнение и резултат от тест 4

Направените тестове потвърждават, че системата реагира адекватно при валидни и невалидни входни данни, спазва бизнес логиката и визуализира правилно резултатите в потребителския интерфейс. Всички успешно преминали тестови случаи доказват стабилността на приложението при нормална употреба, както и при гранични ситуации.

Глава 6: Заключение и бъдещо развитие

6.1. Изводи и заключение

След изпълнението на разработката и провеждането на тестване на системата за управление на здравословно хранене, можем да потвърдим, че тя успешно изпълнява очакваната функционалност, съобразена с предварително дефинираните изисквания и спецификации.

След анализ на вече съществуващи решения в областта на хранителните дневници и приложения за следене на здравословен начин на живот, се установиха редица пропуски, като липса на персонализирано изчисление на калориите според физиологичното състояние, недостатъчно прецизно управление на потребителски данни и ограничена администраторска функционалност. Разработената система адресира тези липси, като комбинира добрите практики с нови решения, предлагащи цялостна и лесна за използване платформа както за обикновени потребители, така и за администратори.

Приложението използва модерни уеб технологии и утвърдени архитектурни модели, което го прави гъвкаво, мащабируемо и сигурно. Интерфейсът е изграден с мисъл за крайния потребител – интуитивен, минималистичен и достъпен.

Системата:

- Улеснява потребителите в дневното проследяване на калории, макронутриенти и цели за тегло;
- Осигурява динамични препоръки, съобразени с възраст, пол, ръст, тегло, активност и физиологично състояние;
- Позволява анализ на прогреса чрез визуализации и статистики;
- Предоставя на администраторите пълен контрол върху потребителската база – търсене, филтриране, редакция, изтриване и възстановяване на данни.

В заключение системата за управление на здравословно хранене успешно постига основните си цели – да улесни проследяването на храненето и напредъка на потребителите, да повиши информираността относно балансираното хранене и да предостави ефективни инструменти за администрация и анализ. Приложението е стабилно, разширяемо и готово за реална употреба или бъдещо надграждане с модули.

6.2. Бъдещо развитие

Системата за управление на здравословно хранене и рецепти предоставя стабилна основа за проследяване на калориен прием, цели, тегло и напредък, както и за ефективно администриране на потребители. Въпреки това, с оглед на бъдещи подобрения и повишаване на стойността за крайните потребители, могат да бъдат реализирани редица допълнителни функционалности и оптимизации.

За бъдещо развитие и подобрене на текущата система са предвидени следните функционалности за внедряване:

- 1) Синхронизация с фитнес гривни и смарт часовници за автоматично добавяне на физическа активност и разход на калории;
- 2) Възможност за добавяне на упражнения, избор на тренировки и отчитане на изгорени калории;
- 3) Интелигентни препоръки чрез машинно обучение – персонализирани предложения за храни, цели и режими на хранене на базата на анализ на поведението и данните на потребителя;
- 4) Въвеждане на социални функции като създаване на общности, споделяне на прогрес, хранителни планове и мотивация между потребителите;
- 5) Многоезичност и локализация – добавяне на опции за използване на различни езици с цел по-широка достъпност;
- 6) Интегриране на система за известия и напомняния – известия за добавяне на храна, воден прием, седмични отчети и препоръки.

Тези насоки предлагат естествено надграждане на съществуващата функционалност и биха превърнали приложението в цялостен здравословен асистент. Реализирането им би повишило потребителската ангажираност, би подобрило точността на събраните данни и би създадо още по-персонализирано преживяване.

Списък с фигури

Фигура 1 - Изглед на MyFitnessPal - система за следене на здравословно хранене	8
Фигура 2 - Изглед на Cronometer - система за следене на здравословно хранене	10
Фигура 3 - Архитектура на системата	20
Фигура 4 - Use Case диаграма за потребителската част	26
Фигура 5 - Use case диаграма за админ - мениджър съдържание	27
Фигура 6 - Use case диаграма за главния админ (Всички административни use case-ове изискват валиден JWT токен. Проверката се извършва автоматично при всяка заявка, дори ако не е визуално свързана в диаграмата.)	28
Фигура 7 - Flowchart диаграма за регистрация, вход и първоначална настройка на профила	29
Фигура 8 - Flowchart диаграма за работния процес на отделните страници	31
Фигура 9 - Flowchart диаграма за Други Опции (More)	31
Фигура 10 - Flowchart диаграма за функционалностите на главния администратор	34
Фигура 11 - Flowchart диаграма за функционалностите на администратор - мениджър съдържание	34
Фигура 12 - wireframe за вход Фигура 13 - wireframe за регистрация Фигура 14 - wireframe за потв. на имейл	35
Фигура 15 - wireframe за начална стр. Фигура 16 - wireframe за дневник	
Фигура 17 - wireframe за рецепти	36
Фигура 18 - концептуален модел на БД	41
Фигура 19 - релационна база от данни	43
Фигура 20 - страница за регистрация	66
Фигура 21 - страница за потвърждение на имейл	66
Фигура 22 -стр. за вход в системата	67
Фигура 23 - стъпка 1 Фигура 24 - стъпка 2 Фигура 25 - стъпка 3	
Фигура 26 - стъпка 4	71
Фигура 27 - стъпка 5 Фигура 28 - стъпка 6 Фигура 29 - стъпка 7	
Фигура 30 - начална страница	71
Фигура 31 - интерфейс за добавяне на храна	72
Фигура 32 - интерфейс за хранителния дневник	73
Фигура 33 - визуализация на календар	74
Фигура 34 - изтриване на храни	74
Фигура 35 - оставащи калории	75
Фигура 36 - филтрирани рецепти Фигура 37 - страница с рецепти Фигура 38 - търсене на рецепта	76
Фигура 39 - страници с детайли за рецепта	77
Фигура 40 - екран с любими рецепти	78
Фигура 41 - предупреждение	79
Фигура 42 - начален екран Фигура 43 - начален екран - обобщение на нутриенти и микронутриенти	80
Фигура 44 - добавяне на вода	80

Фигура 45 - редатиране на лични данни	81
Фигура 46 - редактираните данни в базата	81
Фигура 47 - редактиране на потребителски данни	82
Фигура 48 - редактиране на целево тегло	83
Фигура 50 - въвеждане на скорост.....	83
Фигура 49 - въвеждане на целево тегло	83
Фигура 51 – акт. на данните Фигура 52 – акт. на Home Фигура 53 - акт. на Дневник	84
Фигура 54 - екран за макронутриентното разпределение	85
Фигура 55 - диетични предпочитания Фигура 56 - филтрирани рецепти	86
Фигура 57 - филтрирани рецепти спрямо алергени	86
Фигура 58 - добавяне на хранителни продукти.....	88
Фигура 59 - запис на добавената храна в базата	88
Фигура 60 - поле с персонализирани храни на потребителя	88
Фигура 61 - активни потребители в последните 5 мин	90
Фигура 62 - диаграма за ръст на потребителската база.....	90
Фигура 63 - диаграма за разпределение на потребителите по пол.....	91
Фигура 64 - среден BMI на потребителите	91
Фигура 65 - интерфейс за преглед и управление на администратори	93
Фигура 66 - интерфейс за преглед и управление на потребители	93
Фигура 67 - форма за създаване на администратор	94
Фигура 68 - резултат от търсене на потребител.....	95
Фигура 69 – филтриране и резултат	95
Фигура 70 - детайли за намерения потребител	96
Фигура 71 - соритане на записани храни в низходящ ред.....	96
Фигура 72 - филтриране на записани храни във възходящ ред	96
Фигура 73 - диаграма за използвани функционалности.....	98
Фигура 74 - екран за преглед на всички рецепти	98
Фигура 75 - модален прозорец за детайли за дадена рецепта.....	98
Фигура 76 - преглед на хранителни продукти.....	99
Фигура 77 - преглед на категории	99
Фигура 78 - добавяне на категории	100
Фигура 79 - добавяне на хранителни продукти от външен източник	100
Фигура 80 - добавяне на рецепта - част 1	101
Фигура 81 - добавяне на рецепта - част 2	102
Фигура 82 - автоматични тестове за регистрация на потребители.....	104
Фигура 83 - автоматични тестове за вход в системата	104
Фигура 84 - Автоматични тестове за добавяне на консумирана храна	104
Фигура 85 - обобщена информация за нутриентите (макронутриенти и микронутриенти), приети от потребителя за избрана дата	105
Фигура 86 - резултати от тестове за функционалностите, свързани с храни.....	105
Фигура 87 - Успешно преминали клиентски тестове: профил, обновяване, цели и статистики.....	105

Фигура 88 - Успешни тестове: Админски действия с потребители – търсене, филтриране, редакция и изтриване.....	106
Фигура 89 - изпълнение и резултат на тест 1	107
Фигура 90 - Изпълнение и резултат на тест 2	108
Фигура 91 - Изпълнение и резултат от Тест 3	109
Фигура 92 - изпълнение и резултат от тест 4	111

Използвани източници

- [1] „Node.js v22.3.0 documentation”, <https://nodejs.org/docs/latest/api>
- [2] „React Native Documentation”, <https://reactnative.dev/docs/getting-started>
- [3] „React Documentation – Learn React”, <https://react.dev/learn>
- [4] „Express - Node.js web application framework”, <https://expressjs.com/>
- [5] „PostgreSQL: Documentation”, <https://www.postgresql.org/docs/>
- [6] „Render Documentation”, <https://render.com/docs>
- [7] „Neon Docs – Serverless PostgreSQL”, <https://neon.tech/docs>
- [8] „GitHub Docs – Product Documentation”, <https://docs.github.com/>
- [9] „World Health Organization – Healthy diet”, <https://www.who.int/news-room/fact-sheets/detail/healthy-diet>
- [10] „Harvard T.H. Chan School of Public Health – The Nutrition Source”, <https://www.hsph.harvard.edu/nutritionsource/>
- [11] „USDA MyPlate – Dietary Guidelines for Americans”, <https://www.myplate.gov/>
- [12] „National Health Service (UK) – Eat well”, <https://www.nhs.uk/live-well/eat-well/>
- [13] „Centers for Disease Control and Prevention (CDC) – Healthy Eating for a Healthy Weight”, https://www.cdc.gov/healthyweight/healthy_eating/index.html
- [14] „Mayo Clinic – Nutrition and healthy eating”, <https://www.mayoclinic.org/healthy-lifestyle/nutrition-and-healthy-eating>
- [15] „National Institutes of Health – Tracking Your Diet and Activity”, <https://www.niddk.nih.gov/health-information/weight-management/track-your-diet-physical-activity>
- [16] „Healthline – How to Count Calories: A Beginner’s Guide”, <https://www.healthline.com/nutrition/how-to-count-calories>
- [17] „Eat This Much – Automatic Meal Planner”, <https://www.eatthismuch.com/>
- [18] „Verywell Fit – Healthy Recipes and Nutrition Tips”, <https://www.verywellfit.com/>

Приложение – исходен код

➤ signUp.jsx

```
import { View, StyleSheet, Text } from "react-native";
import { Link } from "expo-router";
import { SafeAreaView } from "react-native-safe-area-context";
import AuthHeader from "../components/AuthHeader";
import SignUpForm from "../components/signUpForm";
import { LinearGradient } from 'expo-linear-gradient';
import * as Animatable from 'react-native-animatable';
```

```
export default function SignUp() {
  return (
    <View style={{ flex: 1 }}>
      <LinearGradient
        colors={['#6b6e7e', '#101326']}
        style={styles.animatedGradient}
      />
      <Animatable.View
        animation="fadeIn"
        duration={7000}
        iterationCount="infinite"
        direction="alternate"
        style={styles.animatedGradient}
      >
        <LinearGradient
          colors={['#101326', '#6b6e7e']}
          style={styles.animatedGradient}
        />
      </Animatable.View>
```

```

    <SafeAreaView style={styles.content}>
      <AuthHeader title="Create Your VivoraCore Account" />
      <SignUpForm />
      <Text style={styles.signInPrompt}>Already have an account?</Text>
      <Link href="/auth/pages/signIn">
        <Text style={styles.signInLink}>Sign In Here</Text>
      </Link>
    </SafeAreaView>
  </View>
);
}
const styles = StyleSheet.create({
  animatedGradient: {
    ...StyleSheet.absoluteFillObject,
  },
  content: {
    flex: 1,
    width: "100%",
    padding: 30,
    justifyContent: "center",
    alignItems: "center",
  },
  signInPrompt: {
    textAlign: "center",
    fontSize: 16,
    marginTop: 15,
  },
  signInLink: {

```

```

    textAlign: "center",
    fontSize: 16,
    marginTop: 5,
    fontWeight: "bold",
  },
});

```

➤ signIn.jsx

```

import { View, StyleSheet } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
import { LinearGradient } from 'expo-linear-gradient';
import * as Animatable from 'react-native-animatable';
import AuthHeader from "../components/AuthHeader";
import SignInForm from "../components/signInForm";

```

```

export default function SignIn() {
  return (
    <View style={{ flex: 1, position: 'relative' }}>
      { /* Base Gradient */ }
      <LinearGradient
        colors={['#6b6e7e', '#101326']}
        style={styles.animatedGradient}
      />

```

```

      { /* Animated Gradient Overlay */ }
      <Animatable.View
        animation="fadeIn"
        duration={7000}
        iterationCount="infinite"
        direction="alternate"

```

```

        style={styles.animatedGradient}
      >
      <LinearGradient
        colors={['#101326', '#6b6e7e']}
        style={styles.animatedGradient}
      />
    </Animatable.View>

    { /* Content */ }
    <SafeAreaView style={styles.content}>
      <AuthHeader title="Welcome Back" logoStyle={{ width: 160, height: 140, marginTop:
40 }} />
      <SignInForm />
    </SafeAreaView>
  </View>
);
}

const styles = StyleSheet.create({
  animatedGradient: {
    ...StyleSheet.absoluteFillObject,
  },
  content: {
    marginTop: 40,
    justifyContent: "center",
    alignItems: "center",
    paddingHorizontal: 40,
  },
});

```

➤ authService.js

```
export const login = async (email, password) => {
  const response = await fetch(
    "https://dietary-management-studio.onrender.com/api/auth/login",
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    }
  );

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || "Login failed.");
  }

  return data; // contains the token and user info
};

export const register = async ({ name, email, password, tipsOptIn }) => {
  const response = await fetch(
    "https://dietary-management-studio.onrender.com/api/auth/register",
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name, email, password, tipsOptIn }),
    }
  );
};
```

```

const data = await response.json();
if (!response.ok) {
  throw new Error(data.message || "Registration failed.");
}
return data;
};

export const verifyEmailCode = async (email, code) => {
  const response = await fetch(
    'https://dietary-management-studio.onrender.com/api/auth/verify-code',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, code }),
    }
  );
  const data = await response.json();
  return { ok: response.ok, data };
};

export const resendVerificationCode = async (email) => {
  const response = await fetch(
    'https://dietary-management-studio.onrender.com/api/auth/resend-code',
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email }),
    }
  );

```

```

);

const data = await response.json();

return { ok: response.ok, data };

};

export const updateEmail = async (oldEmail, newEmail) => {

  const response = await fetch(

    'https://dietary-management-studio.onrender.com/api/auth/update-email',

    {

      method: 'PATCH',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify({ oldEmail, newEmail }),

    }

  );

  const data = await response.json();

  return { ok: response.ok, data };

};

```

➤ Backend/auth.js

```

const db = require('../db');

const sgMail = require('@sendgrid/mail');

const router = express.Router();

const { authenticateToken } = require('../middleware/auth');

const JWT_SECRET = process.env.JWT_SECRET;;

sgMail.setApiKey(process.env.SENDGRID_API_KEY);

router.post('/register', async (req, res) => {

  const { name, email, password, tipsOptIn } = req.body;

```



```

if (!name || !email || !password) {
  return res.status(400).json({ message: 'All fields are required' });
}

try {
  const result = await db.query('SELECT * FROM users WHERE email = $1', [email]);
  const existingUser = result.rows[0];

  // If user already exists
  if (existingUser) {
    if (existingUser.verified) {
      return res.status(400).json({ message: 'Email already registered' });
    } else {
      // Resend verification code
      const newVerificationCode = Math.floor(100000 + Math.random() * 900000).toString();
      await db.query('UPDATE users SET email_verification_code = $1 WHERE email = $2',
        [newVerificationCode, email]);

      await sgMail.send({
        to: email,
        from: 'snacksmartapp@gmail.com',
        templateId: process.env.SENDGRID_TEMPLATE_ID,
        dynamic_template_data: {
          name: existingUser.name,
          code: newVerificationCode
        },
      });
    }
  }
}

```

```

    return res.status(200).json({ message: 'User already registered but not verified. New
verification code sent.' });
  }
}

```

```

//  Register new user

```

```

const hashedPassword = await bcrypt.hash(password, 10);
const verificationCode = Math.floor(100000 + Math.random() * 900000).toString();

```

```

await db.query(
  'INSERT INTO users (name, email, password, tips_opt_in, email_verification_code)
VALUES ($1, $2, $3, $4, $5)',
  [name, email, hashedPassword, tipsOptIn || false, verificationCode]
);

```

```

await sgMail.send({
  to: email,
  from: 'snacksmartapp@gmail.com',
  templateId: process.env.SENDGRID_TEMPLATE_ID,
  dynamic_template_data: {
    name,
    code: verificationCode
  },
});

```

```

res.status(201).json({ message: 'User registered. Please verify your email.' });

```

```

} catch (error) {
  console.error(error);
}

```

```

    res.status(500).json({ message: 'Internal server error' });
  }
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: 'Email and password are required' });
  }

  try {
    const result = await db.query(
      `SELECT
        u.id, u.email, u.password, u.verified, u.sex, u.birthday, u.height, u.weight,
        u.activity_level_id,
        g.goal_weight, g.weekly_rate_kg,
        m.protein_ratio, m.fat_ratio, m.carb_ratio
      FROM users u
      LEFT JOIN LATERAL (
        SELECT goal_weight, weekly_rate_kg
        FROM user_goals
        WHERE user_id = u.id
        ORDER BY created_at DESC
        LIMIT 1
      ) g ON true
      LEFT JOIN user_macros m ON u.id = m.user_id
      WHERE u.email = $1
      `, [email]);

```

```

const user = result.rows[0];

if (!user) {
  return res.status(401).json({ message: 'Invalid credentials' });
}

const [prefResult, allergyResult, cuisineResult] = await Promise.all([
  db.query("SELECT 1 FROM user_preferences WHERE user_id = $1", [user.id]),
  db.query("SELECT 1 FROM user_allergies WHERE user_id = $1 LIMIT 1", [user.id]),
  db.query("SELECT 1 FROM user_favorite_cuisines WHERE user_id = $1 LIMIT 1",
[user.id]),
]);

if (!user.verified) {
  return res.status(403).json({ message: 'Please verify your email before logging in.' });
}

const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) {
  return res.status(401).json({ message: 'Invalid credentials' });
}

const token = jwt.sign({ id: user.id, email: user.email }, JWT_SECRET, { expiresIn: '1d'
});

res.status(200).json({
  token,
  sex: user.sex,

```

```

    birthday: user.birthday,
    height: user.height,
    weight: user.weight,
    activity_level_id: user.activity_level_id,
    physiological_state: user.physiological_state,
    goal_weight: user.goal_weight,
    weekly_rate_kg: user.weekly_rate_kg,
    macro_set:
      user.protein_ratio !== null &&
      user.fat_ratio !== null &&
      user.carb_ratio !== null,
    hasPreferences: prefResult.rowCount > 0,
    hasAllergies: allergyResult.rowCount > 0,
    hasCuisines: cuisineResult.rowCount > 0,
  });
} catch (err) {
  console.error(err);
  res.status(500).json({ message: 'Server error' });
}
});
router.post('/verify-code', async (req, res) => {
  const { email, code } = req.body;

  try {
    const result = await db.query('SELECT email_verification_code FROM users WHERE
email = $1', [email]);
    const user = result.rows[0];

    if (!user) return res.status(404).json({ message: 'User not found' });
  }

```

```

    if (user.email_verification_code === code) {

      await db.query('UPDATE users SET verified = true, email_verification_code = NULL
WHERE email = $1', [email]);

      return res.status(200).json({ message: ' Email verified!' });

    } else {

      return res.status(400).json({ message: ' Invalid verification code' });

    }

  } catch (err) {

    console.error(err);

    res.status(500).json({ message: 'Server error' });

  }

});

router.post('/resend-code', async (req, res) => {

  const { email } = req.body;

  const code = Math.floor(100000 + Math.random() * 900000).toString();

  try {

    const result = await db.query('SELECT * FROM users WHERE email = $1 AND verified
= false', [email]);

    const user = result.rows[0];

    if (!user) return res.status(404).json({ message: 'User not found or already verified' });

    await db.query('UPDATE users SET email_verification_code = $1 WHERE email = $2',
[code, email]);

    await sgMail.send({

      to: email,

      from: 'snacksmartapp@gmail.com',

```

```

    templateId: process.env.SENDGRID_TEMPLATE_ID,
    dynamic_template_data: {
      name: user.name,
      code
    },
  });

  res.status(200).json({ message: 'Verification code resent.' });
} catch (err) {
  console.error(err);
  res.status(500).json({ message: 'Server error' });
}
});

```

➤ AddRecipe.jsx

```

import React, { useState, useEffect } from 'react';
import {
  Text,
  StyleSheet,
  ScrollView,
} from 'react-native';
import RecipeFormInputs from '.././.././shared/Form/RecipeFormImports';
import InstructionSteps from '.././.././shared/List/InstructionList';
import IngredientManager from '.././.././shared/List/IngredientManager';
import CategorySelector from '.././.././shared/List/CategorySelector';
import SubmitButton from '.././../././components/shared/SubmitButton';
import NutrientList from '.././.././shared/List/NutrientList';
import AlertBox from '.././../././components/shared/AlertBox';
import BackButton from '.././../././components/shared/BackButton';

```

```

import { fetchCategories, fetchDishTypes, fetchAllergens, fetchIngredients, fetchCuisines, }
from '.././.././../services/ViewService';

export default function AddRecipes() {

  const [form, setForm] = useState({

    title: "",

    image: "",

    summary: "",

    readyInMinutes: "",

    servings: "",

    pricePerServing: "",

    healthScore: "",

    categories: [],

    dishTypes: [],

    cuisineTypes: [],

    ingredients: [],

    instructions: [],

    nutrients: [],

  });

  const [selectedNutrient, setSelectedNutrient] = useState("");
  const [showNewIngredientField, setShowNewIngredientField] = useState(false);
  const [newIngredientName, setNewIngredientName] = useState("");
  const [newIngredientAllergens, setNewIngredientAllergens] = useState([]);
  const [ingredientAllergenMap, setIngredientAllergenMap] = useState({});
  const [categoryOptions, setCategoryOptions] = useState([]);
  const [dishTypeOptions, setDishTypeOptions] = useState([]);
  const [cuisineTypeOptions, setCuisineTypeOptions] = useState([]);
  const [allergenOptions, setAllergenOptions] = useState([]);
  const [availableIngredients, setAvailableIngredients] = useState([]);

```



```
const [alert, setAlert] = useState({ type: "", message: "" });
```

```
const [newIngredient, setNewIngredient] = useState({  
  name: "",  
  amount: "",  
  unit: "",  
  original: "",  
});
```

```
useEffect(() => {  
  const loadData = async () => {  
    try {  
      const [  
        categories,  
        dishTypes,  
        cuisines,  
        allergens,  
        ingredients  
      ] = await Promise.all([  
        fetchCategories(),  
        fetchDishTypes(),  
        fetchCuisines(),  
        fetchAllergens(),  
        fetchIngredients()  
      ]);  
      setCategoryOptions(categories);  
      setDishTypeOptions(dishTypes);  
      setCuisineTypeOptions(cuisines);  
      setAllergenOptions(allergens);
```

```

        setAvailableIngredients(ingredients);
    } catch (err) {
        console.error('Failed to load metadata:', err);
    }
};

loadData();
}, []);
useEffect(() => {
    if (alert.message !== "") {
        const timer = setTimeout(() => setAlert({ type: "", message: "" }), 4000);
        return () => clearTimeout(timer);
    }
}, [alert]);
const handleInputChange = (key, value) => {
    setForm({ ...form, [key]: value });
};

const handleAddInstruction = () => {
    const id = Date.now();
    setForm(prev => ({
        ...prev,
        instructions: [...prev.instructions, { id, stepNumber: "", description: "" }],
    }));
};

const handleAddNutrient = () => {
    const id = Date.now();
    setForm(prev => ({
        ...prev,

```

```

        nutrients: [...prev.nutrients, { id, name: selectedNutrient, amount: "", unit: "" }],
    }));
};

const getIngredientIdByValue = (value) => {
    const match = availableIngredients.find(i => i.value === value);
    return match ? match.id : null;
};

const validateForm = () => {
    const errors = [];

    if (!form.title.trim()) errors.push("Title is required.");
    if (!form.image.trim()) errors.push("Image URL is required.");
    if (!form.summary.trim()) errors.push("Summary is required.");
    if (!form.readyInMinutes || isNaN(form.readyInMinutes)) errors.push("Ready in minutes must be a number.");
    if (!form.servings || isNaN(form.servings)) errors.push("Servings must be a number.");
    if (!form.pricePerServing || isNaN(form.pricePerServing)) errors.push("Price per serving must be a number.");
    if (!form.healthScore || isNaN(form.healthScore)) errors.push("Health score must be a number.");
    if (form.categories.length === 0) errors.push("At least one category must be selected.");
    if (form.ingredients.length === 0) errors.push("At least one ingredient must be added.");
    if (form.instructions.length === 0) errors.push("At least one instruction step is required.");

    return errors;
};

const handleSubmit = async () => {
    const validationErrors = validateForm();

```

```

if (validationErrors.length > 0) {
  showAlert({ type: 'error', message: validationErrors[0] });
  return;
}

try {
  const payload = {
    title: form.title,
    image: form.image,
    summary: form.summary,
    readyInMinutes: parseInt(form.readyInMinutes),
    servings: parseInt(form.servings),
    pricePerServing: parseFloat(form.pricePerServing),
    healthScore: parseFloat(form.healthScore),
    categories: form.categories,
    dishTypes: form.dishTypes,
    cuisineTypes: form.cuisineTypes,
    ingredients: form.ingredients.map(ing => ({
      ingredient_id: getIngredientIdByValue(ing.name),
      amount: parseFloat(ing.amount),
      unit: ing.unit,
      original: ing.original,
    })),
    instructions: form.instructions.map(s => ({
      step_number: parseInt(s.stepNumber),
      description: s.description,
    })),
    nutrients: form.nutrients.map(n => ({
      name: n.name,
      amount: parseFloat(n.amount),

```

```
        unit: n.unit,
      )))
};
```

```
const res = await fetch('https://dietary-management-
studio.onrender.com/recipes/recipes', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${localStorage.getItem('adminToken')}`,
  },
  body: JSON.stringify(payload)
});
```

```
const data = await res.json();
```

```
if (res.ok) {
  showAlert({ type: 'success', message: `Recipe added successfully! ID:
${data.recipeId}` });
  setForm({
    title: "",
    image: "",
    summary: "",
    readyInMinutes: "",
    servings: "",
    pricePerServing: "",
    healthScore: "",
    categories: [],
    dishTypes: [],
```

```

        cuisineTypes: [],
        ingredients: [],
        instructions: [],
        nutrients: [],
    });
} else {
    console.error('Submit failed:', data);
    showAlert({ type: 'error', message: data.message || 'Failed to add recipe' });
}
} catch (err) {
    console.error('Submit error:', err);
    showAlert({ type: 'error', message: 'Something went wrong while submitting the recipe.'
});
}
};

return (
    <ScrollView contentContainerStyle={styles.container}>
        <BackButton to={"/admin/dashboard/content/?tab=manageRecipes"} label={"Back to
Manage Recipes"} />
        <Text style={styles.title}>Add New Recipe</Text>
        {alert.message !== " && <AlertBox type={alert.type} message={alert.message} />}

        <RecipeFormInputs form={form} onChange={handleInputChange} />

        <CategorySelector
            categoryOptions={categoryOptions}

```

```

dishTypeOptions={dishTypeOptions}
cuisineTypeOptions={cuisineTypeOptions}
selectedCategories={form.categories}
setCategories={(val) => setForm({ ...form, categories: val })}
selectedDishTypes={form.dishTypes}
setDishTypes={(val) => setForm({ ...form, dishTypes: val })}
selectedCuisineTypes={form.cuisineTypes}
setCuisineTypes={(val) => setForm({ ...form, cuisineTypes: val })}
/>

```

<IngredientManager

```

newIngredient={newIngredient}
setNewIngredient={setNewIngredient}
form={form}
setForm={setForm}
showNewIngredientField={showNewIngredientField}
setShowNewIngredientField={setShowNewIngredientField}
newIngredientName={newIngredientName}
setNewIngredientName={setNewIngredientName}
newIngredientAllergens={newIngredientAllergens}
setNewIngredientAllergens={setNewIngredientAllergens}
allergenOptions={allergenOptions}
availableIngredients={availableIngredients}
setAvailableIngredients={setAvailableIngredients}
ingredientAllergenMap={ingredientAllergenMap}
setIngredientAllergenMap={setIngredientAllergenMap}
/>

```

```

<InstructionSteps
  instructions={form.instructions}
  onAdd={handleAddInstruction}
  onUpdate={(index, key, value) => {
    const updated = [...form.instructions];
    updated[index][key] = value;
    setForm({ ...form, instructions: updated });
  }}
/>

<NutrientList
  nutrients={form.nutrients}
  selectedNutrient={selectedNutrient}
  onSelectNutrient={setSelectedNutrient}
  onAddNutrient={handleAddNutrient}
  onUpdateNutrient={(index, key, value) => {
    const updated = [...form.nutrients];
    updated[index][key] = value;
    setForm({ ...form, nutrients: updated });
  }}
/>

<SubmitButton onPress={handleSubmit} />

</ScrollView>

);
}

```



```
const styles = StyleSheet.create({
  container: {
    backgroundColor: '#6b6e7e',
    padding: 20,
  },
  title: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 20,
    color: '#101326',
  },
});
```